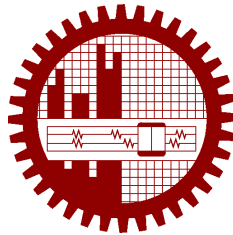


M.Sc. Engg. (CSE) Thesis

**Agentic Graph Reasoning: Autonomous Knowledge Graph
Navigation for Fact Verification and Hallucination
Mitigation in Large Language Models**

Submitted by
Md. Sakif Khan
0421052099

Supervised by
Dr. Sadia Sharmin



Submitted to
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of
Master of Science in Computer Science and Engineering

June 2026

Candidate’s Declaration

I, do, hereby, certify that the work presented in this thesis, titled, “Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models”, is the outcome of the investigation and research carried out by me under the supervision of Dr. Sadia Sharmin, Associate Professor, Department of CSE, BUET.

I also declare that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Md. Sakif Khan

0421052099

The thesis titled “**Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models**”, submitted by Md. Sakif Khan, Student ID 0421052099, Session April 2021, to the Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, has been accepted as satisfactory in partial fulfilment of the requirements for the degree of Master of Science in Computer Science and Engineering and approved as to its style and contents on June 6, 2026.

Board of Examiners

- | | | |
|----|--|--------------------------|
| 1. | <hr style="border: 0; border-top: 1px solid black; margin-bottom: 5px;"/> Dr. Sadia Sharmin
Associate Professor
Department of CSE, BUET, Dhaka | Chairman
(Supervisor) |
| 2. | <hr style="border: 0; border-top: 1px solid black; margin-bottom: 5px;"/> Dr. Tanzima Hashem
Professor and Head
Department of CSE, BUET, Dhaka | Member
(Ex-Officio) |
| 3. | <hr style="border: 0; border-top: 1px solid black; margin-bottom: 5px;"/> Dr. M. Kaykobad
Professor
Department of CSE, BUET, Dhaka | Member |
| 4. | <hr style="border: 0; border-top: 1px solid black; margin-bottom: 5px;"/> Dr. Abu Sayed Md. Latiful Hoque
Professor
Department of CSE, BUET, Dhaka | Member |
| 5. | <hr style="border: 0; border-top: 1px solid black; margin-bottom: 5px;"/> Dr. Honorable External Examiner
Professor
Department of CSE
University of External, Dhaka | Member
(External) |

Acknowledgement

I am thankful to

Finally,

Dhaka

June 6, 2026

Md. Sakif Khan

0421052099

Contents

Candidate’s Declaration	i
Board of Examiners	ii
Acknowledgement	iii
List of Figures	xii
List of Tables	xiii
List of Algorithms	xvi
Abstract	xvii
1 Introduction	1
1.1 Hallucination in Large Language Models	1
1.1.1 Where Hallucination Originates: Training Data, Model, and Prompt . .	1
1.1.2 Why Multi-Hop Questions Are the Hard Case	2
1.2 From Retrieval Augmentation to Graph-Structured Grounding	2
1.2.1 Vector Retrieval and Its Structural Blind Spot	2
1.2.2 Knowledge Graphs and Static GraphRAG	3
1.2.3 Agentic Retrieval: Reasoning as Navigation	3
1.3 Limitations of Existing Agentic KGQA Systems	4
1.3.1 No Check on What the Final Answer Asserts	4
1.3.2 Unquantified Contribution of Individual Agentic Mechanisms	4
1.3.3 Accuracy Reported Without Cost	4
1.4 Problem Statement	5
1.5 Research Questions	5
1.5.1 RQ1: Does Agentic Navigation Improve Multi-Hop Factual Accuracy?	5
1.5.2 RQ2: What Does Pre-Generation Verification Contribute Beyond Graph Navigation?	5
1.5.3 RQ3: Which Components Contribute What, at What Token Cost? . . .	5
1.6 Our Contribution	6

1.6.1	The AGR Framework and Its Verification Layer	6
1.6.2	A Component-Level Ablation of Agentic Mechanisms	6
1.6.3	Stratum-Dependent Decomposition: When Planning Hurts	6
1.6.4	The Echo Attractor as a Named Failure Mode	6
1.6.5	Quantified Benchmark Defect Rates for WebQSP and CWQ	6
1.6.6	An Evaluation Protocol with Pre-Registered Thresholds	7
1.7	Scope and Delimitations	7
1.8	Thesis Organization	7
2	Background and Preliminaries	8
2.1	Knowledge Graphs	8
2.1.1	Triples, Entities, and Relations	8
2.1.2	Freebase: MIDs, Schema, and Mediator Nodes	8
2.1.3	Property Graphs, Neo4j, and Cypher	9
2.2	Knowledge Graph Question Answering	9
2.2.1	Multi-Hop Questions and Constraint Satisfaction	9
2.2.2	Evaluation Conventions: Hits@1 and F1	10
2.3	Large Language Models as Reasoning Engines	10
2.3.1	In-Context Learning and Chain-of-Thought Prompting	10
2.3.2	Tool Use and the ReAct Loop	10
2.3.3	A Working Definition of Hallucination for This Thesis	11
2.4	Retrieval-Augmented Generation	11
2.4.1	Dense Retrieval and Sentence Embeddings	11
2.4.2	Graph-Based Retrieval	11
2.5	Search over Graphs	12
2.5.1	Best-First and Beam Search	12
2.5.2	Backtracking and Dead-End Detection	12
2.5.3	Relation to Monte Carlo Tree Search: A Terminological Clarification	12
2.6	Statistical Tools Used in This Thesis	13
3	Related Work	14
3.1	Static Graph-Augmented Generation	14
3.1.1	GraphRAG and Query-Focused Summarization	14
3.1.2	HippoRAG and Memory-Structured Retrieval	14
3.2	Path-Retrieval and Semantic-Parsing KGQA	15
3.2.1	Reasoning-on-Graphs	15
3.2.2	StructGPT and KG-Agent	15
3.3	Agentic Graph Exploration	16
3.3.1	Think-on-Graph	16

3.3.2	Plan-on-Graph	16
3.3.3	Generate-on-Graph and Reasoning with Trees	16
3.3.4	KBQA-o1 and Genuine Tree Search	17
3.4	Verification and Self-Correction in Language Models	17
3.4.1	Chain-of-Verification	17
3.4.2	Ontology-Guided and Self-Correcting Graph RAG	18
3.5	Measuring Factuality	18
3.5.1	Claim-Decomposition Metrics	18
3.5.2	Factuality Benchmarks and Their Limits	18
3.6	Comparative Summary of Agentic KGQA Systems	18
3.7	Positioning of This Work	20
4	The Knowledge Environment	22
4.1	Design Goals	22
4.1.1	Decoupling the Graph Source from the Question Sets	22
4.1.2	Why a Curated Subgraph Rather Than LLM-Extracted Triples	22
4.2	Source Data	23
4.2.1	The Freebase Snapshot	23
4.2.2	WebQSP and ComplexWebQuestions as Question Sets	23
4.3	Graph Construction	24
4.3.1	Union of Per-Question Subgraphs from the RoG Distribution	24
4.3.2	Relation Filtering and Noise Removal	24
4.3.3	Treatment of Mediator Nodes and Its Effect on Hop Counts	25
4.4	Graph Store Construction	25
4.4.1	Property-Graph Schema	25
4.4.2	Bulk Import into Neo4j	26
4.4.3	Graph Statistics	26
4.5	The Vector Index	26
4.5.1	Entity Embeddings	26
4.5.2	Relation-Vocabulary Embeddings	27
4.5.3	Scope: Linking and Candidate Ranking, Never Ground Truth	27
4.6	Entity Resolution and Surface-Form Linking	27
4.6.1	The Exact, Lexical, and Vector Cascade	27
4.6.2	Threshold Selection on Development Data	28
4.7	Environment Validation Gate	28
4.7.1	Answer-Reachability Protocol	28
4.7.2	Coverage Results and the Induced Accuracy Ceiling	29
4.7.3	Analysis of Linking Misses and Gate Failures	29
4.7.4	What the Environment Cannot Express	30

5	The AGR Framework: Architecture and Navigation	31
5.1	Architectural Overview	31
5.1.1	The Controlled-Agent Pattern	31
5.1.2	The State Machine	32
5.2	The Graph Tool API	33
5.2.1	Rationale: Constrained Tools Instead of Free-Form Cypher	33
5.2.2	The Five Operations	33
5.2.3	Mediator Traversal and Relation-Agnostic Adjacency	34
5.3	Agent State	34
5.4	The Planner	35
5.4.1	Decomposition into Ordered Sub-Objectives	35
5.4.2	Plan Validation and Degenerate-Plan Handling	36
5.5	The Explorer	36
5.5.1	Frontier Construction and the Single Scoring Pass	36
5.5.2	The Hybrid Scoring Function	37
5.6	The Evaluator and Routing Policy	38
5.6.1	Sub-Objective Completion Judgment	38
5.6.2	The Groundedness Filter on Resolutions	38
5.6.3	Backtracking Triggers and the Backtrack Stack	38
5.6.4	Routing	39
5.7	The Answerer	40
5.8	Budgets and Termination Guarantees	40
5.9	Instrumentation and Reproducibility	41
5.10	Design Validation on the Development Set	41
5.11	The Complete Navigation Algorithm	43
6	The Structural Verification Layer	45
6.1	Motivation: Verifying Before Emitting, Not After	45
6.2	Claim Decomposition of the Draft Answer	46
6.3	Two-Stage Claim Checking	47
6.3.1	The Structural Check	47
6.3.2	The Entailment Check	48
6.3.3	Why the Structural Check Runs First	49
6.4	Repair Policies for Unsupported Claims	49
6.4.1	Targeted Re-Exploration Under Remaining Budget	49
6.4.2	Answer Rewriting and Hedging	50
6.4.3	Iteration Cap and the Draft-Only Fallback	51
6.5	Entity Filtering of the Final Answer	51
6.6	Output Contract: Answer Paired with Supporting Triples	53

6.7	A Worked Example	54
6.8	Failure Modes by Construction: Wrongful Rejection and Wrongful Acceptance	55
7	Experimental Setup	58
7.1	Mapping Experiments to Research Questions	58
7.1.1	Pre-Registration and the No-Tuning Policy	59
7.1.2	Metrics Proposed but Not Run	59
7.2	Test Sets	60
7.2.1	WebQSP and ComplexWebQuestions	60
7.2.2	Stratified Sampling, Seeds, and Published Question IDs	60
7.2.3	Hop-Count Stratification and the Accuracy Ceiling	61
7.3	Backbone Model Selection and Qualification	61
7.3.1	Trajectory Stability Under Temperature-Zero Decoding	62
7.3.2	Response Caching as the Reproducibility Backstop	63
7.4	Baseline Systems	63
7.4.1	No-Retrieval LLM (Parametric-Memory Control)	63
7.4.2	Vector-RAG over Verbalised Triples	64
7.4.3	Static GraphRAG	64
7.4.4	Think-on-Graph (Agentic Baseline)	64
7.4.5	Variables Held Constant Across All Systems	65
7.4.6	Baseline Certification Protocol	65
7.5	Metrics	65
7.5.1	Answer Accuracy: Hits@1 and F1	65
7.5.2	Accounting for Hedges and Abstentions	66
7.5.3	Two-Tier Groundedness and Hallucination Rate	66
7.5.4	Efficiency: Tokens, LLM Calls, and Wall-Clock Latency	67
7.6	The Groundedness Judge and Its Validation	67
7.6.1	Independence of the Judge from the Answering System	67
7.6.2	Human Annotation Protocol	67
7.6.3	Agreement Between Judge and Human Annotator	68
7.7	Gold-Answer Quality Control	69
7.7.1	The Gold-Noise Problem in WebQSP and CWQ	69
7.7.2	Consensus Pre-Pass and Per-Item Adjudication	69
7.7.3	Two Evidence Signatures of Label Error	69
7.7.4	Census-Based Exclusions and the Dual-Reporting Policy	70
7.8	Ablation Conditions	71
7.9	Hyperparameter Selection on the Development Set	71
7.9.1	Development Set Construction and Coverage	71
7.9.2	The α - τ Sweep Protocol	72

7.10	Implementation, Environment, and Reproducibility	72
8	Results	73
8.1	Development-Set Tuning Outcomes	73
8.2	Main Results on WebQSP	74
8.3	Main Results on ComplexWebQuestions	76
8.4	Accuracy Broken Down by Hop Count	76
8.5	Groundedness and Hallucination Rate Across Systems	78
8.5.1	Tier 1: Structural Hallucination Is a Property of the Paradigm	78
8.5.2	Tier 2: A Narrow Band and an Underpowered Comparison	79
8.6	Efficiency and Cost	79
8.6.1	Token and Call Budgets per System	79
8.6.2	The Accuracy–Cost Frontier	80
8.7	Ablation Study	80
8.7.1	Without the Planner: A Stratum-Dependent Effect	81
8.7.2	Without Backtracking	82
8.7.3	Without the Verification Layer	82
8.7.4	Embedding-Only Scoring	82
8.7.5	Paired Significance Testing and Statistical Power	83
8.8	Findings Against RQ1, RQ2, and RQ3	83
9	Error Analysis and Discussion	85
9.1	Annotation Protocol and the Failure Taxonomy	85
9.1.1	Category and Subtype Schema	85
9.1.2	Population, Not Sample	86
9.2	Distribution of Failure Categories Across Datasets	86
9.2.1	The Shape Flip, and What It Is Not	86
9.2.2	Hedging Where It Should	87
9.3	Decomposition, Drafting, and Answer-Selection Errors	88
9.3.1	The Entity-Extraction Bug	88
9.3.2	Right Candidate Retrieved, Wrong One Drafted	89
9.3.3	Failing to Commit: Six Cases of Hedging on a Verified Fact	89
9.3.4	Context-Stripping: When Decomposition Discards the Question	90
9.4	Relation-Selection and Navigation Errors	91
9.4.1	The Evaluator Abandons a Good Relation	91
9.5	Verifier Rejection and Acceptance Errors	92
9.5.1	Defining the Error Polarities	92
9.5.2	Wrongly Rejected: Semantically Correct, Structurally Unmatched	92
9.5.3	Wrongly Accepted: One Specimen and a Boundary Case	93

9.5.4	A Rate for One Polarity, and a Blank for the Other	93
9.6	Knowledge-Graph Gaps: Literals, Temporal Qualifiers, and Ordinals	94
9.7	The Echo Attractor: Answering with the Topic or an Intermediate Entity	95
9.8	Benchmark Defects: Gold Noise and Ambiguous Questions	95
9.8.1	The Confirmed Defects	96
9.8.2	Where Bad Gold Comes From: Flattened Multi-Valued Relations	97
9.9	Discussion	97
9.9.1	What Agency Buys, and What It Costs	97
9.9.2	When Verification Helps and When It Over-Hedges	98
9.9.3	Threats to Validity	98
10	Conclusion	100
10.1	Summary of Contributions	100
10.2	Limitations	102
10.3	Future Work	103
10.3.1	Repairs the Census Earned	103
10.3.2	Logging Accepted Claims to Make Wrongful Acceptance Measurable	103
10.3.3	Adaptive Decomposition Gating	103
10.3.4	Semantic-Level Verification	104
10.3.5	A Set-Intersection Operator for Compound Questions	104
10.3.6	Ordinal and Literal-Value Support	104
10.3.7	Path Fidelity Against Gold SPARQL Relation Chains	104
10.3.8	Beyond This Environment	105
10.4	Closing Remark	105
	References	106
	Index	111
A	Prompt Templates	112
A.1	Planner	112
A.2	Relation Scorer	114
A.3	Evaluator	114
A.4	Drafter and Claim Decomposer	115
A.5	Entailment Check	116
A.6	Rewriter	116
A.7	Answerer	117
A.8	Baseline Prompts	117
B	Tool API Specification and Cypher Templates	119

B.1	search_entity	119
B.2	get_relations	120
B.3	get_neighbors	120
B.4	verify_triple	121
B.5	verify_connection	121
C	Sampled Question IDs and Run Manifests	123
C.1	WebQSP test sample ($n = 400$)	123
C.2	CWQ test sample ($n = 400$)	124
C.3	Development set ($n = 80$)	128
D	Annotation Instructions and Label Schema	130
D.1	Judge Instructions	130
D.2	Failure Census Label Schema	131
D.3	Label Provenance and Regeneration	131
E	Implementation Notes	134
E.1	Records Self-Describe	134
E.2	Frozen Artifacts Are Committed; Regenerable Ones Are Not	134
E.3	Cache-Identity Verification	135
E.4	Config Injection and the Partial-Edit Hazard	136
E.5	Routers Read, Nodes Write	136
E.6	Instrumentation Decides What Can Be Concluded	136

List of Figures

- 5.1 The AGR state machine. Two cycles exist: Explorer $\leftrightarrow$ Evaluator for iterative search, and Verifier $\rightarrow$ Explorer for verification-driven re-exploration. Both are bounded by the budgets of Section 5.8 rather than by the graph structure. 32

List of Tables

3.1	Mechanism comparison of the six closest prior systems. No system performs claim-level verification of the final answer against retrieved triples before responding.	19
3.2	Reported evaluation settings and Hits@1 as published. These figures are <i>not</i> directly comparable to one another: backbones differ, some papers evaluate on sampled subsets, and the backbone accounts for much of the variation.	20
4.1	The knowledge environment as constructed and imported.	26
4.2	Entity resolution over all topic-entity mentions in both test sets. A mention counts as resolved only when the returned node’s name equals the mention exactly.	28
4.3	Environment coverage over the full test sets, at a bound of four raw hops. The reachability column is the Hits@1 ceiling for every system evaluated in this thesis.	29
5.1	The graph tool API. All operations are deterministic parameterised Cypher; none consults an embedding except <code>search_entity</code> , which delegates to the resolver of Section 4.6.	33
5.2	The agent state. Fields marked <i>append-only</i> accumulate across node executions through reducers; all others are overwritten by whichever node returns them.	35
5.3	Budget configuration. The configuration is hashed and the hash recorded in every run record, so results can always be attributed to the configuration that produced them.	40
5.4	Root causes of failure in the first end-to-end run, and the mechanism each one motivated. Every fix is described in the section indicated.	42
6.1	The complete attribution census: every development-set question whose answer differed between claim checking and its ablation, at $\alpha = 0.7$, $\tau = 0.20$. Three questions, one run per condition. “Pre-fix” is the original design in which the rewrite call regenerated the entity list; “post-fix” is deterministic filtering.	52
7.1	The two certified test samples. The unreachable stratum is retained deliberately; the ceiling is the share of questions with at least one gold answer reachable within four hops, and is the upper bound on Hits@1 for every system evaluated here.	61

7.2	Gold-label adjudication. Rows and distinct questions are separate units because the pass emits one row per consensus answer. Exclusions are the union of <code>gold_wrong</code> and <code>ambiguous_question</code> questions.	70
8.1	The development-set sweep, $n = 80$. Hits, hedges, and wrong answers partition the set. <code>Verify</code> indicates claim verification enabled; the two draft-only rows disable it. All rows are from the sweep as run, before the answer-entity filter of Section 6.5 was corrected; the frozen condition’s post-correction figures are given in the text.	74
8.2	Main results, $n = 400$ per dataset. Brackets are 95% bootstrap intervals. P and R are mean per-question precision and recall; Hedge is the share of questions on which the system asserted nothing. The Hits@1 ceilings are 97.0% and 99.2% (Table 7.1).	75
8.3	Paired McNemar tests, AGR against each baseline, on per-question correctness over the same 400 questions. Every comparison rejects.	76
8.4	Hits@1 and F1 per hop stratum. Stratum sizes are those of Table 7.1. WebQSP’s <code>h3plus</code> has $n = 4$ and is not interpreted.	77
8.5	Two-tier groundedness. Tier 1 is deterministic over every asserted entity in all 4,000 records; Tier 2 is a language-model entailment judgement over a stratified sample of 60 assertions per system per dataset, validated at $\kappa = 0.6995$ (Section 7.6).	78
8.6	Cost per question. Latency is computed over cold-cache records only. Clip rate is the share of questions on which the shared 25-call budget was exhausted mid-search.	80
8.7	Ablation conditions on stratified halves. The reference row is the frozen system restricted to the same questions, at zero additional cost. Latency is unmeasured for no-verifier because every record in that condition is a full cache replay. . . .	81
8.8	Component attribution, RQ3. $\Delta F1$ is <i>ablated minus reference</i> , so positive means removing the component improved F1. Only the planner condition reaches significance.	83
9.1	Census provenance. Stage D is the main reading over the full-matrix failures; Stage A is the separate reading of the questions where removing the planner flipped a failure into a success (Section 8.7); the single migrated row is a Stage D finding later promoted to a formal benchmark-defect exclusion (Section 9.8). . . .	86
9.2	The merged failure histogram, $n = 259$. Wrong and hedge are kept separate. Generated by <code>scripts/synthesize_census.py</code> over the three label sources of Table 9.1. . . .	87

9.3	The entity-extraction bug. In each case the drafted answer text names every gold value correctly and the scored entity list holds only the sentence’s grammatical subject.	88
C.1	The three question sets. The test samples are stratified by hop count, the development set by question shape.	123
D.1	Failure categories and their subtypes. One category per packet.	132
D.2	Gold-defect and question-defect subtypes, with the counts observed during adjudication across both datasets.	133
D.3	Where each label lives, and what regenerating will do to it.	133

List of Algorithms

1	AGR navigation (verification layer abstracted; see Chapter 6)	44
2	Structural verification of a draft	48

Abstract

Large language models answer factual questions fluently. When they do not hold the fact, they get it wrong just as fluently. Retrieval-augmented generation grounds them in retrieved text, but it retrieves once, before reasoning begins. A multi-hop question must then be answered from whatever that single query returned.

This thesis presents Agentic Graph Reasoning (AGR), which interleaves retrieval with reasoning instead. An agent navigates a knowledge graph through a constrained, deterministic tool API. It breaks the question into sub-objectives, expands a scored frontier, and backtracks when a branch stops paying. Its distinguishing component is a Structural Verification Layer. The layer splits the draft answer into atomic claims and checks each one against the triples the agent actually traversed, before the answer is emitted. Claims that cannot be grounded are re-explored or withdrawn. The system therefore hedges rather than asserts, and every answer arrives with the triples that support it.

AGR is evaluated on a Freebase-derived environment of 2,592,892 entities and 8,309,194 triples, over 400 certified questions from each of WebQSP and ComplexWebQuestions. Five systems are compared: parametric answering, vector retrieval, GraphRAG, an agentic-navigation baseline, and AGR. All five run on one frozen backbone under the same budget, so that architecture rather than model capacity accounts for the differences.

AGR reaches Hits@1 of 0.755 and F1 of 0.642 on WebQSP, and 0.522 and 0.469 on ComplexWebQuestions. That is ahead of every baseline, including the agentic one at 0.660 and 0.435, at half the language-model calls. AGR asserts no ungrounded entity across 1,709 assertions, against 27.1% for parametric answering. The thesis attributes that property to graph navigation itself rather than to the verification layer. On ComplexWebQuestions, AGR’s accuracy rises with hop count while every other system decays.

A four-condition ablation with paired significance testing finds one component with a detectable accuracy effect, and its sign is not the expected one. Removing the planner *improves* WebQSP by 0.083 F1 at $p = 0.006$ and cuts tokens by 30%. On ComplexWebQuestions it trends the other way. Decomposition should therefore be gated on question structure rather than applied unconditionally. A census of all 259 remaining failures names the characteristic error of a graph-navigating system. It is not invention but the *echo attractor*: a real, grounded entity one hop from the answer, which any grounding check passes because it is true. The census also finds 59 questions where the benchmark was wrong rather than the system. Consensus against gold is dominated by that same attractor rather than by annotation error. Rescoring on majority agreement would hide this failure rather than correct it.

Chapter 1

Introduction

Large language models answer factual questions fluently. Often enough, they answer them incorrectly. The failure is not random noise. It is systematic and confident, and it is hard to detect from the output alone, because a fabricated answer is written in the same register as a correct one. The rate rises sharply when the answer requires composing several facts rather than recalling one. This thesis closes that gap in two steps. It gives the model a symbolic knowledge graph to walk through. It then checks what the model is about to assert, before it asserts it.

This chapter motivates the problem and surveys why the obvious remedies are incomplete. It then states the problem and the research questions, summarises the contributions, and sets out how the rest of the document is organised.

1.1 Hallucination in Large Language Models

A *hallucination* is a generated statement that is fluent and syntactically well formed, but that no source the model can point to supports. Frequently it is simply false. Huang et al. [1] survey the phenomenon at length. They treat it as an intrinsic consequence of how these models are built, not as an incidental defect to be patched away. That framing matters here. If hallucination were a bug, the response would be to fix it. Because it is structural, the response must be architectural.

1.1.1 Where Hallucination Originates: Training Data, Model, and Prompt

It is useful to separate three loci at which a hallucination can originate, because they admit very different interventions.

The first is the *training data*. A model trained on a corpus containing errors, staleness, or under-representation of a domain will reproduce those defects. Correcting them means curating the corpus and retraining, which is outside the reach of anyone who is not the model’s producer.

The second is the *model* itself — its parameterisation and decoding procedure. Knowledge is stored diffusely across weights with no explicit index, so the model has no reliable internal signal distinguishing a fact it has memorised from a plausible continuation it is constructing. Interventions here mean changing the architecture or the training objective, again largely out of reach.

The third is the *prompt*. This is the locus the thesis addresses. If the context supplied alongside the question contains the facts required to answer it, the model need not fall back on parametric recall. Structured context can override defects inherited from training data without touching the model. That is the premise behind retrieval augmentation [2], and behind the present work. The contribution lies in *how* that context is assembled, and in *what is checked* before the answer is emitted.

1.1.2 Why Multi-Hop Questions Are the Hard Case

Single-fact questions are comparatively benign. “Where was Ada Lovelace born?” requires one lookup, and either the model knows it or it does not.

Multi-hop questions — “Who directed the film that won Best Picture in 1998?” — require composing an ordered chain of lookups, each conditioned on the previous result. Two properties make these the hard case. First, errors compound. A wrong resolution at the first hop is not usually detected downstream, so the remainder of the chain proceeds confidently from a false premise. Second, the intermediate results are exactly what a question-level correctness check cannot see. A system that answers correctly by accident and a system that answers correctly by sound reasoning are indistinguishable from the final string alone.

Multi-hop questions also carry *constraints* that must be satisfied jointly rather than sequentially, whether temporal, categorical, or ordinal. Dropping one silently is among the most common ways such questions are answered wrongly rather than not at all.

1.2 From Retrieval Augmentation to Graph-Structured Grounding

1.2.1 Vector Retrieval and Its Structural Blind Spot

Retrieval-Augmented Generation [2] attaches the model to an external corpus. The question is embedded, semantically similar passages are retrieved by vector similarity, and the model answers from them. Where the answer resides in a single retrievable passage, this works well.

Its weakness is structural. Vector retrieval treats a corpus as flat chunks of text and ranks them by similarity to the question. For a multi-hop question that signal is misleading. The passage

holding the *intermediate* entity may bear little lexical or semantic resemblance to the original question, and the system does not yet know which entity to look for. Relationships between entities are not represented at all. They are at best implicit in adjacency within a retrieved chunk, and they are precisely what a compositional question traverses. Surveys of the intersection between graphs and language models identify this as the central limitation of flat retrieval [3].

1.2.2 Knowledge Graphs and Static GraphRAG

A *knowledge graph* makes relationships explicit. Facts are stored as typed edges between identified entities, so composing facts becomes traversal rather than inference over prose. GraphRAG [4] exploits this by building a graph over a corpus and retrieving structured context rather than passages. Related systems extend the idea to long-term memory [5] and to temporally-qualified facts [6].

These systems nonetheless retrieve *statically*. A subgraph around the linked entities is fetched in one shot and handed to the model. The retrieval step does not know what the reasoning step will need, because it happens first and only once. Consider a question whose second hop depends on the answer to its first. A fixed-radius neighbourhood is either too small to contain the answer, or so large that the relevant edges are buried among thousands of irrelevant ones.

1.2.3 Agentic Retrieval: Reasoning as Navigation

The natural response is to interleave retrieval with reasoning. The model takes one step, observes what it found, and decides where to look next. Retrieval becomes *navigation*. The system becomes an agent operating on the graph rather than a pipeline reading from it.

Think-on-Graph [7] established the pattern with LLM-guided beam search over the graph. Plan-on-Graph [8] added sub-objective decomposition and reflection-driven backtracking, and Reasoning-on-Graphs [9] generates grounded relation paths as explicit plans. Further systems add tool abstractions [10], handle incomplete graphs [11], or generalise across structured sources [12]. Recent surveys treat this convergence of language models and knowledge graphs as a distinct research programme [13, 14].

Agentic navigation is the right shape for the problem, and this thesis adopts it. The question is what remains unsolved once it is adopted.

1.3 Limitations of Existing Agentic KGQA Systems

1.3.1 No Check on What the Final Answer Asserts

Agentic systems ground their answers in a traversed subgraph, and this thesis’s own measurements confirm that they do so effectively. The entities they assert exist in the graph, and are reachable along the paths they walked (Section 8.5). The deficit is not fabrication.

The deficit is *precision*. Every system surveyed in Chapter 3 emits whatever its final generation call produces from the traversed context. Nothing checks that each asserted entity actually answers the question that was asked. An entity can be present in the retrieved subgraph, correctly retrieved, and still be the wrong answer. It may be an intermediate node from the reasoning chain, a container where a member was requested, or one of several plausible neighbours selected arbitrarily. It may also be one of many entities asserted where the question admits few, which inflates recall at the cost of precision. Both failures are invisible to a system that verifies retrieval but never verifies assertion. They surface as a measurable precision gap against a system that does check (Section 8.2).

Chain-of-Verification [15] and multi-agent debate [16] establish that post-hoc self-checking reduces factual error in free-form generation. A self-correcting agentic graph pipeline has been demonstrated in a clinical setting [17]. What is missing is a claim-level check against the retrieved triples, performed *before* the answer is emitted, inside a graph-navigating agent.

1.3.2 Unquantified Contribution of Individual Agentic Mechanisms

Agentic systems are assemblies of mechanisms: decomposition, scoring, backtracking, self-correction. They are reported as wholes. When such a system outperforms a static baseline, the literature rarely establishes which mechanism produced the gain, or whether some contribute nothing at all. Without that attribution, subsequent work inherits the entire assembly on faith, and its cost along with it. Component-level ablation is the standard remedy. It is conspicuously underused in this literature.

1.3.3 Accuracy Reported Without Cost

Navigation is expensive. Each exploration step costs at least one model call, and agentic systems routinely consume an order of magnitude more tokens per question than a single-shot baseline. Accuracy tables that omit this make agentic methods look uniformly preferable, when the honest picture is a trade-off. That trade-off is exactly what a practitioner deciding whether to deploy such a system needs to see.

1.4 Problem Statement

This thesis addresses the following problem. Given a natural-language question q and a knowledge graph $G = (E, R, T)$ with entities E , relation types R , and triples $T \subseteq E \times R \times E$, produce an answer set $A \subseteq E$ together with a supporting triple set $S \subseteq T$, such that every entity in A is justified by S , and S consists only of triples the system actually traversed in reaching A . Three properties distinguish this from standard knowledge graph question answering. First, the output is a *pair*, not an answer alone. The supporting triples are part of the contract, which makes the reasoning auditable rather than asserted. Second, the support must be *traversed*, not merely retrievable after the fact. That forecloses post-hoc justification of an answer arrived at some other way. Third, the justification is checked *before* the answer is emitted. Unsupported content can then be removed or hedged rather than merely flagged.

1.5 Research Questions

1.5.1 RQ1: Does Agentic Navigation Improve Multi-Hop Factual Accuracy?

Does interleaving retrieval with reasoning over a knowledge graph produce more accurate answers than static retrieval, and does the advantage grow with the number of hops a question requires? Answering this needs a no-retrieval control, since the standard benchmarks predate current models and may be partly memorised.

1.5.2 RQ2: What Does Pre-Generation Verification Contribute Beyond Graph Navigation?

Given a system that already navigates the graph, what does a claim-level check against traversed triples add? The question is posed as *what does it contribute*, not *does it reduce hallucination*, because the answer turns out to have several parts that a yes-or-no framing would obscure (Section 8.8).

1.5.3 RQ3: Which Components Contribute What, at What Token Cost?

Decomposition, backtracking, verification, and learned candidate scoring each cost model calls. Which of them measurably improve accuracy or groundedness, by how much, and at what price in tokens and latency?

1.6 Our Contribution

1.6.1 The AGR Framework and Its Verification Layer

We present Agentic Graph Reasoning (AGR), a knowledge-graph question-answering framework built as an explicit state machine over a constrained, deterministic graph tool API. It carries a *Structural Verification Layer* that decomposes the draft answer into atomic claims and checks each against the traversed subgraph, before the answer is emitted. Unsupported claims trigger targeted re-exploration, or removal from the answer. The system returns the answer paired with its supporting triples.

1.6.2 A Component-Level Ablation of Agentic Mechanisms

We ablate four components: the planner, the backtracking mechanism, the verification layer, and the learned component of candidate scoring. For each condition we report accuracy, groundedness, and token cost, with paired significance testing. This addresses the attribution gap identified in Section 1.3.

1.6.3 Stratum-Dependent Decomposition: When Planning Hurts

Question decomposition is treated in the literature as straightforwardly beneficial. We find it is not: removing the planner *improves* accuracy on the shallower benchmark while trending toward harming it on the deeper one. The asymmetry, not the pooled average, is the result.

1.6.4 The Echo Attractor as a Named Failure Mode

We identify and characterise a recurring failure in which multiple independent systems converge on the same wrong answer. It is typically the question’s own topic entity, an intermediate node from the reasoning chain, or a coarser-granularity container. Because it is shared across systems, it is invisible to any evaluation that treats systems as independent.

1.6.5 Quantified Benchmark Defect Rates for WebQSP and CWQ

We quantify label defects in the two standard benchmarks through a consensus pre-pass followed by per-item adjudication. Two kinds of label error are separated, because they have opposite evidence signatures. We report results both with and without the affected items.

1.6.6 An Evaluation Protocol with Pre-Registered Thresholds

Three decision thresholds were fixed before test data was touched: the judge-agreement bar, the baseline-certification diagnostic, and the scoring hyperparameters. We report outcomes against all three, including the one that was narrowly missed.

1.7 Scope and Delimitations

The knowledge environment is a static, curated subset of Freebase [18]. The thesis does not address graph construction from unstructured text, knowledge-base completion, or the temporal evolution of facts. Questions are English and factoid, drawn from WebQSP [19] and ComplexWebQuestions [20]. Long-form generation and conversational settings are out of scope. All systems share one backbone model, held constant so that architectural differences are not confounded with model capability. The thesis therefore reports how these architectures compare on a single backbone, not how they scale across backbones. Finally, the system is a research prototype evaluated offline. Latency is reported as a cost measure, not as evidence of production readiness.

1.8 Thesis Organization

Chapter 2 establishes the background required for the technical chapters. It covers knowledge graphs and their Freebase-specific idiosyncrasies, knowledge graph question answering and its evaluation conventions, language models as reasoning engines, retrieval augmentation, graph search, and the statistical tools used throughout. Chapter 3 surveys prior work and positions AGR against it. It culminates in a comparison of agentic knowledge graph question answering systems, which identifies the gap this thesis addresses and justifies the choice of baselines.

Chapter 4 describes the construction and validation of the knowledge environment, including the answer-reachability gate that bounds the accuracy attainable in it. Chapter 5 specifies the AGR framework: the tool API, the state machine, the planner, the scoring function, the backtracking policy, and the budgets that guarantee termination. Chapter 6 treats the Structural Verification Layer in detail, as the thesis’s principal architectural contribution.

Chapter 7 sets out the experimental design: test sets, baselines, metrics, judge validation, gold-answer quality control, and the pre-registration policy. Chapter 8 reports results and answers the three research questions. Chapter 9 analyses the failures that remain, including the benchmark defects and the shared-attractor pattern, and discusses threats to validity. Chapter 10 summarises the contributions, states the limitations, and identifies future work.

Chapter 2

Background and Preliminaries

This chapter establishes the concepts and notation the technical chapters assume. It is deliberately compact: only material actually used later is included, and standard results are stated rather than derived.

2.1 Knowledge Graphs

2.1.1 Triples, Entities, and Relations

A *knowledge graph* $G = (E, R, T)$ consists of a set of entities E , a set of relation types R , and a set of triples $T \subseteq E \times R \times E$. A triple (h, r, t) asserts that relation r holds from head entity h to tail entity t — for example, (Ada Lovelace, place_of_birth, London). Relations are directed and typed, and both directions carry meaning: the inverse of place_of_birth is a legitimate but distinct relation.

Two properties matter for what follows. Triples are *atomic*: each asserts exactly one fact, so a claim can be checked against the graph by testing for the existence of specific edges. And they are *composable*: a chain $(e_0, r_1, e_1), (e_1, r_2, e_2), \dots$ expresses a multi-step relationship that no single triple states, which is what makes a knowledge graph the natural substrate for multi-hop questions.

2.1.2 Freebase: MIDs, Schema, and Mediator Nodes

Freebase [18] is a large collaboratively-built knowledge base, frozen since 2016, and the substrate for the standard knowledge graph question answering benchmarks. Three of its characteristics affect any system built on it.

Entities are identified by opaque *machine identifiers* (MIDs) such as m.02mjmr, with human-readable names attached as a separate property. Names are neither unique nor stable, so entity

identity and entity surface form must be handled separately.

Relations are namespaced paths such as `people.person.place_of_birth`, encoding domain, type, and property. The vocabulary is large — tens of thousands of relation types — and includes many that carry no factual content, such as internal type and key predicates.

Most consequentially, Freebase represents n -ary facts through *mediator* or *compound value type* (CVT) nodes. A fact with more than two participants — an actor playing a particular character in a particular film — is not a single edge but an anonymous intermediate node linking the participants. A relationship a user would describe as one hop is therefore two edges in the graph. Any system reporting hop counts over Freebase must state how it treats these nodes, since the choice changes every depth measurement; systems that ignore them lose the facts they encode [11].

2.1.3 Property Graphs, Neo4j, and Cypher

The triple model above is the RDF view. An alternative is the *property graph* model, in which nodes and edges are both typed and may carry arbitrary key–value properties. This is the model implemented by Neo4j [21] and queried by Cypher [22], a declarative language whose `MATCH` clause expresses graph patterns directly. A triple store maps onto it naturally: entities become nodes carrying their MID and name, and relations become typed edges. Traversal patterns that would require recursive joins in a relational schema are expressed as literal path patterns, which is the practical reason for choosing a graph store here.

2.2 Knowledge Graph Question Answering

2.2.1 Multi-Hop Questions and Constraint Satisfaction

Knowledge graph question answering (KGQA) takes a natural-language question and a graph, and returns the entity or entities that answer it. A question is *k-hop* if answering it requires traversing k edges from the entities named in the question — its *topic entities* — to the answer. Beyond hop count, questions vary in *constraints*: conditions that filter candidates rather than extending the chain. “Which of Tolstoy’s novels was published after 1875?” is one hop plus a temporal constraint. Constraints must be satisfied jointly with the chain, and a system that traverses correctly while silently dropping a constraint returns a superset of the answer — a failure that hop-count analysis alone does not expose.

Two families of approach have historically divided this field. *Semantic parsing* methods translate the question into a formal query (SPARQL or equivalent) and execute it, giving exact and interpretable results but requiring the parse to be exactly right [19, 23]. *Information retrieval* methods retrieve a relevant subgraph and rank candidate answers within it, degrading more

gracefully but sacrificing transparency. The agentic systems considered in this thesis are closer to the second family, but recover much of the first family’s transparency by logging the traversal itself.

2.2.2 Evaluation Conventions: Hits@1 and F1

Two metrics are conventional. *Hits@1* is the proportion of questions for which the system’s top-ranked answer appears in the gold answer set; it is a per-question binary correctness measure and the headline number in most of this literature.

Because many questions admit multiple correct answers, Hits@1 alone rewards a system that names one correct entity and ignores the rest. *F1* therefore complements it, computed per question between the predicted answer set A and the gold set A^* :

$$P = \frac{|A \cap A^*|}{|A|}, \quad R = \frac{|A \cap A^*|}{|A^*|}, \quad F_1 = \frac{2PR}{P + R},$$

and averaged over questions. The distinction between P and R is load-bearing in this thesis: a system that asserts many entities can achieve high recall while its precision falls, and Section 1.3 argues that precisely this failure goes unmeasured when only Hits@1 is reported.

2.3 Large Language Models as Reasoning Engines

2.3.1 In-Context Learning and Chain-of-Thought Prompting

Sufficiently large language models perform tasks specified entirely in their prompt, without gradient updates — *in-context learning* [24]. Prompting the model to produce intermediate reasoning steps before its final answer, *chain-of-thought* prompting [25], substantially improves performance on multi-step problems. Both are used here: the agent’s planner, scorer, evaluator, and verifier are all prompted components of a single frozen model, and no component is fine-tuned. This matters for comparability, since several systems surveyed in Chapter 3 are fine-tuned and therefore not directly comparable to prompting methods.

2.3.2 Tool Use and the ReAct Loop

A language model can be given *tools*: named operations it may invoke, whose results re-enter its context [26]. The *ReAct* pattern [27] interleaves reasoning and action in a loop — the model reasons about what to do, acts, observes the result, and reasons again — which is the control structure underlying agentic graph navigation. Where the tools are graph operations, each

iteration extends a traversal, and the sequence of tool calls is a complete record of how the answer was reached.

2.3.3 A Working Definition of Hallucination for This Thesis

“Hallucination” is used loosely in the literature. This thesis adopts a narrow, operational definition tied to the knowledge environment: an asserted entity is *ungrounded* if it does not exist in the graph, or exists but is not reachable from the question’s topic entities within the traversal budget. A claim is *unsupported* if the traversed triples do not entail it.

Two consequences of this definition should be stated plainly. It is relative to the environment: an entity absent from the graph is ungrounded even if true in the world. And groundedness is not correctness — an answer can be perfectly grounded, every entity real and reachable, and still be the wrong answer to the question. Chapter 8 shows that this gap is not hypothetical, and Section 1.3 is built on it.

2.4 Retrieval-Augmented Generation

2.4.1 Dense Retrieval and Sentence Embeddings

Dense retrieval encodes queries and documents into a shared vector space and retrieves by nearest neighbour, capturing semantic similarity where lexical matching fails [28]. Sentence-level encoders such as Sentence-BERT [29] produce embeddings suitable for direct cosine comparison. In this thesis, embeddings serve two bounded purposes — linking question phrases to graph entities, and pre-ranking relation candidates during exploration. They never establish that a fact is true; that role belongs exclusively to the symbolic triples.

2.4.2 Graph-Based Retrieval

Graph-based retrieval replaces flat passages with structured context [4, 5]. The retrieved unit is a subgraph rather than a document, which preserves relationships explicitly. The distinction that matters for this thesis is not graph versus vector but *static versus interleaved*: whether retrieval happens once before reasoning begins, or repeatedly as reasoning proceeds.

2.5 Search over Graphs

2.5.1 Best-First and Beam Search

Best-first search maintains a frontier of candidate nodes ordered by a heuristic estimate of promise and expands the most promising first [30, 31]. *Beam search* bounds the frontier to the b highest-scoring candidates at each depth, trading completeness for a fixed memory and computation budget. With an expensive scoring function — here, a language model call — the beam width is also the principal cost control.

2.5.2 Backtracking and Dead-End Detection

Bounded search commits to choices that may prove wrong. *Backtracking* recovers by abandoning the current branch and resuming from a previously unexplored alternative, which requires maintaining a stack of such alternatives and a trigger condition for when to pop it. Trigger conditions and the stack discipline used here are specified in Section 5.6.

2.5.3 Relation to Monte Carlo Tree Search: A Terminological Clarification

The original proposal for this work described its navigation strategy as “inspired by Monte Carlo Tree Search”. That description is withdrawn here, and the reason is worth stating explicitly.

Monte Carlo Tree Search [32, 33] is defined by four phases: selection, expansion, *simulation* (a rollout to a terminal state), and *backpropagation* of the rollout’s value to the ancestors of the expanded node. The two phases that give MCTS its statistical character are simulation and backpropagation, which together let node values be estimated from sampled outcomes rather than from a fixed heuristic.

The method used in this thesis has neither. There are no rollouts, and no value is propagated back up the search tree; each candidate is scored once, by a fixed function combining embedding similarity with a language-model judgement, and that score is never revised in light of what is found later. What the method is, is *guided best-first search with a bounded beam and backtracking*. It is described that way throughout.

The contrast is not hypothetical. Genuine tree search has been applied to this exact task: KBQA-01 [34] drives an agentic knowledge base question answering process with full MCTS — policy and reward models, rollouts, and value backpropagation — over Freebase, and Reasoning with Trees [35] likewise builds an explicit search tree. That those systems exist is precisely why the terminology matters here: describing a single-pass scoring heuristic as MCTS-inspired would invite comparison with machinery this framework does not implement.

2.6 Statistical Tools Used in This Thesis

Three tools recur, each chosen for a specific property of the data.

Bootstrap confidence intervals [36] quantify sampling uncertainty by resampling the evaluated questions with replacement and recomputing the metric; the 2.5th and 97.5th percentiles of the resulting distribution give a 95% interval. They are used because the test sets are samples, and because they require no distributional assumption about metrics such as per-question F1.

McNemar’s test [37] assesses whether two systems differ in accuracy when both are evaluated on the *same* questions. It considers only the discordant pairs — questions one system answers correctly and the other does not — and tests whether the split between the two discordant counts departs from chance. Paired testing is essential here because all systems are run on identical question samples, so an unpaired test would discard the pairing and lose power.

Cohen’s κ [38] measures agreement between two annotators on a categorical judgement, correcting for agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is observed agreement and p_e is the agreement expected if both annotators labelled independently at their observed marginal rates. It is used to validate the automatic groundedness judge against human annotation. Conventional interpretation follows Landis and Koch [39], on whose scale values between 0.61 and 0.80 denote substantial agreement.

Chapter 3

Related Work

This chapter surveys the work AGR builds on and competes with. It proceeds from static graph-augmented generation, through path-retrieval and agentic exploration, to verification and factuality measurement, and closes with a structured comparison of the six closest systems. That comparison serves two purposes: it locates the gap this thesis addresses, and it justifies the choice of baselines in Section 7.4.

3.1 Static Graph-Augmented Generation

3.1.1 GraphRAG and Query-Focused Summarization

GraphRAG [4] constructs a knowledge graph from a text corpus using a language model, partitions it into communities, and pre-computes summaries at each level, so a query can be answered from structured context rather than retrieved passages. Its strength is global questions — those whose answer depends on the corpus as a whole rather than any single passage — which flat retrieval handles poorly.

The relevant limitation is that retrieval remains a single step preceding generation. The context is assembled before the model has begun reasoning, so it cannot be conditioned on intermediate findings. GraphRAG also builds its graph by language-model extraction, which is appropriate for unstructured corpora but introduces extraction error into the reference itself — a consideration that shapes the environment design in Section 4.1.

3.1.2 HippoRAG and Memory-Structured Retrieval

HippoRAG [5] draws on the hippocampal indexing theory of human memory, combining a knowledge graph with a personalised PageRank retrieval step so that a single query can integrate evidence distributed across many passages. It substantially improves multi-hop retrieval over flat baselines at low cost.

Its retrieval is nonetheless a single graph-propagation step, not an agent-directed traversal: the propagation is fixed and the model does not choose where to look next. Temporal extensions such as T-GRAG [6] address a different axis — conflicting and redundant facts across time — which is orthogonal to the navigation question and is noted here for completeness rather than compared against.

3.2 Path-Retrieval and Semantic-Parsing KGQA

3.2.1 Reasoning-on-Graphs

Reasoning-on-Graphs (RoG) [9] adopts a planning–retrieval–reasoning pipeline: a fine-tuned model generates relation paths grounded in the graph schema, those paths are used to retrieve matching instantiations, and the model reasons over the retrieved paths. Because answers are produced from retrieved paths, RoG is faithful by construction and can present the path as an explanation.

Two properties distinguish it from the present work. It is *fine-tuned*, which places its reported numbers outside direct comparison with prompting methods. And its plan is generated in one shot before retrieval: if the generated relation path does not instantiate in the graph, there is no mechanism to revise it mid-course. The pre-generation verification question does not arise, because grounding is guaranteed structurally rather than checked.

3.2.2 StructGPT and KG-Agent

StructGPT [12] provides a general interface for reasoning over structured data — knowledge graphs, tables, and databases — through an invoke–linearise–generate loop, in which specialised interfaces extract evidence that is linearised into text for the model. It is deliberately general rather than graph-specific, and performs no open-ended graph search.

KG-Agent [10] is closer to the agentic setting: a multifunctional toolbox, a knowledge-graph executor, and a dynamic memory, driven by an instruction-tuned model that autonomously selects tools across iterations. It demonstrates that a small fine-tuned model with good tool abstractions is competitive with much larger prompted ones. It plans implicitly through tool selection rather than committing to explicit sub-objectives, provides no backtracking mechanism, and performs no verification of the final answer.

3.3 Agentic Graph Exploration

3.3.1 Think-on-Graph

Think-on-Graph (ToG) [7] established the template for training-free agentic KGQA. The model performs iterative beam search over the graph: at each step it is shown candidate relations and entities and prunes them itself, with beam width and maximum depth both set to three. Exploration terminates when the model judges the retrieved paths sufficient, at which point it answers directly.

ToG is the most directly comparable prior system to AGR — training-free, prompting-based, operating on Freebase, evaluated on the same benchmarks — and is therefore the agentic baseline in Section 7.4. Its limitations are instructive. There is no explicit decomposition, so multi-constraint questions are handled implicitly. Low-scoring branches are pruned within the beam, but there is no mechanism to return to an abandoned node once the beam has moved on. And the transition from exploration to answering is a single model judgement: nothing checks the answer that follows.

3.3.2 Plan-on-Graph

Plan-on-Graph (PoG) [8] is the closest system to AGR in ambition. It decomposes the question into sub-objectives including constraint conditions, explores adaptively with a breadth that varies by sub-objective, and adds Guidance, Memory, and Reflection mechanisms. Reflection decides both whether to self-correct and which entity to backtrack to, making PoG the one surveyed system with genuine backtracking.

PoG’s self-correction operates on the *search process*: it reconsiders where to explore. It does not decompose the draft answer into claims and check each against the retrieved triples before responding. The distinction is the gap this thesis occupies, and it is stated precisely in Section 3.7.

A naming hazard must be recorded here. A second, unrelated system is also abbreviated PoG — Paths-over-Graph [40] — and reports considerably higher figures. Survey tables that cite “PoG” without disambiguation are a known source of contaminated comparisons, and this thesis always writes Plan-on-Graph or Paths-over-Graph in full.

3.3.3 Generate-on-Graph and Reasoning with Trees

Generate-on-Graph (GoG) [11] targets the incomplete-graph setting through a thinking–searching–generating loop: when the graph lacks a required fact, the model generates candidate triples from its parametric knowledge and continues. It also expands subgraphs dynamically to handle the mediator nodes described in Section 2.1, which ToG tends to skip. The trade-off

is directly opposed to this thesis’s: GoG deliberately admits model-generated facts into the evidence, whereas AGR admits only graph triples.

Reasoning with Trees [35] applies explicit tree search to knowledge graph question answering. It is cited in Section 2.5.3 as an example of genuine tree-search machinery, in contrast with the guided best-first search implemented here.

3.3.4 KBQA-o1 and Genuine Tree Search

KBQA-o1 [34] is the closest system to the search strategy the present work’s proposal originally described, and therefore the most important one to distinguish from it. It performs agentic knowledge base question answering with full Monte Carlo Tree Search — a policy model proposing actions, a reward model scoring them, and rollouts whose values propagate back through the tree — wrapped around a ReAct-style agent that generates logical forms step by step against a Freebase environment. The exploration additionally generates annotations used for incremental fine-tuning, so the search improves the model that drives it.

Two aspects bear on this thesis. First, KBQA-o1 is what the withdrawn “MCTS-inspired” description of Section 2.5.3 would have claimed: it has the rollouts and value backpropagation that AGR does not, and naming the difference honestly is what keeps the two separable.

Second, its ablation is directly relevant to the methodological argument of Section 3.7. Removing MCTS from KBQA-o1 while leaving every other component intact drops overall GrailQA F1 from 78.5 to 48.5 — a thirty-point attribution to a single mechanism, obtained precisely because the authors ablated components rather than reporting the assembly as a whole. That is the same instrument this thesis applies in Section 8.7, and it is evidence that the instrument is informative. KBQA-o1 is not a baseline here. It is fine-tuned rather than prompted, evaluates under a low-resource few-shot protocol, and does not report ComplexWebQuestions, so no controlled comparison against it is available in this environment.

3.4 Verification and Self-Correction in Language Models

3.4.1 Chain-of-Verification

Chain-of-Verification (CoVe) [15] has the model draft a response, plan verification questions about it, answer those independently, and regenerate a corrected response. It establishes the core premise of this thesis’s verification layer — that decomposing a draft and checking its parts reduces factual error — but performs the checks against the model’s own knowledge rather than an external symbolic source. Multi-agent debate [16] achieves a related effect by having several model instances critique one another.

This distinction matters given evidence that language models cannot reliably self-correct reasoning without external feedback [41]. A verification step whose ground truth is the same model that produced the error is limited by construction. AGR’s verification is grounded in traversed triples, making the check external to the generator.

3.4.2 Ontology-Guided and Self-Correcting Graph RAG

A self-correcting agentic graph RAG system for clinical decision support in hepatology [17] demonstrates the combination of agentic retrieval with a correction loop in a high-stakes domain, and ontology-guided reasoning constrains an agent to logically valid relation use. These substantiate the practical motivation for verification, though in domain-specific settings rather than on the open-domain benchmarks used here. Reflexion-style verbal self-improvement [42] provides the general agentic pattern of which these are graph-specific instances.

3.5 Measuring Factuality

3.5.1 Claim-Decomposition Metrics

FActScore [43] evaluates long-form generation by decomposing text into atomic facts and verifying each against a reference source, reporting the supported proportion. This decomposition-then-verify protocol is the basis for the groundedness metric in Section 7.5.3, with the knowledge graph serving as the reference. The essential methodological requirement, adopted here, is that the verifying judge must be independent of the system that produced the text.

3.5.2 Factuality Benchmarks and Their Limits

FactBench [44] provides a dynamic benchmark for in-the-wild factuality evaluation. The original proposal for this thesis intended to use it both as a source for the knowledge environment and as the hallucination evaluator. That plan was abandoned, and the reason is methodological rather than practical: if the evaluator’s facts are contained in the graph the system retrieves from, a low hallucination rate follows by construction rather than by merit. The evaluation in Chapter 7 therefore measures claim grounding against the graph with a human-validated judge, keeping the evaluator’s ground truth outside the system’s evidence.

3.6 Comparative Summary of Agentic KGQA Systems

Table 3.1 compares the six closest systems on the mechanisms relevant to this thesis, and Table 3.2 records their evaluation settings and published accuracy.

Table 3.1: Mechanism comparison of the six closest prior systems. No system performs claim-level verification of the final answer against retrieved triples before responding.

System	Exploration strategy	Decomposes query?	Backtracks?	Verifies before answering?
ToG [7]	Training-free LLM-guided iterative beam search; beam width and depth both 3	No	No — prunes within beam, cannot return	No
PoG [8]	Self-correcting adaptive planning with Guidance, Memory, Reflection	Yes, into sub-objectives with constraints	Yes — Reflection selects the entity to return to	Partial — corrects the search, not the answer
RoG [9]	Planning–retrieval–reasoning over generated relation paths	Partial — relation-path plans	No	Faithful by construction; no explicit check
KG-Agent [10]	Toolbox with KG executor and dynamic memory; autonomous tool selection	Implicit, via tool calls	No	No
GoG [11]	Thinking–searching–generating; generates triples when the KG is incomplete	Yes	No — expands subgraph instead	No
StructGPT [12]	Iterative reading–then–reasoning via invoke–linearise–generate interfaces	No	No	No
KBQA-o1 [34]	Monte Carlo Tree Search with policy and reward models over a ReAct agent; fine-tuned	Implicit, stepwise logical form	Via MCTS tree revisiting	No
AGR (this work)	Guided best-first beam search with explicit backtracking	Yes, into ordered sub-objectives	Yes — threshold, dead end, or evaluator veto	Yes — claim-level, pre-generation

Three caveats attach to Table 3.2 and are carried into every comparison in this thesis.

First, the figures are drawn from the original publications and reflect different backbones. RoG and KG-Agent are fine-tuned, and their numbers are not comparable with prompting methods at all. Among the prompting methods, the backbone accounts for much of the spread — the same methods drop sharply when run on smaller open-weight models. This is the direct motivation for holding the backbone constant across every system in Chapter 7: the literature does not, so a cross-paper table cannot settle which architecture is better.

Second, evaluation protocols differ. Some papers evaluate on sampled test subsets, and answer-matching conventions are not uniform across implementations.

Third, the Plan-on-Graph figures are marked as approximate. They are consistent with values reported in follow-up work, but should be confirmed against the primary tables before being relied upon in any quantitative claim. No claim in this thesis depends on them; they appear here

Table 3.2: Reported evaluation settings and Hits@1 as published. These figures are *not* directly comparable to one another: backbones differ, some papers evaluate on sampled subsets, and the backbone accounts for much of the variation.

System	Backbone	Datasets	WebQSP	CWQ
ToG	GPT-3.5	CWQ, WebQSP, GrailQA [45], QALD10-en, SimpleQuestions, WebQuestions, T-REx, Zero-Shot RE, Creak	76.2	58.9
	GPT-4		82.6	69.5
PoG	GPT-3.5	CWQ, WebQSP, GrailQA	≈82.0	≈63.2
	GPT-4		≈87.3	≈75.0
RoG	Fine-tuned LLaMA2-7B	WebQSP, CWQ	85.7	62.6
KG-Agent	Fine-tuned 7B	WebQSP, CWQ, plus other KBs	83.3	72.2
GoG	GPT-3.5	WebQSP, CWQ (complete and incomplete KG)	78.7	55.7
	GPT-4		84.4	75.2
StructGPT	GPT-3.5	WebQSP, MetaQA, TableQA, Text-to-SQL	72.6	—
KBQA-o1	Llama-3.1-8B	GrailQA, WebQSP, GraphQ; 100-shot low-resource protocol	59.8 [†]	—
	Llama-3.3-70B		67.0 [†]	—

[†] KBQA-o1 reports **F1**, not Hits@1, and evaluates under a 100-shot low-resource protocol; these cells are therefore *not* comparable with the rest of the column and are shown only to situate the system. It does not evaluate CWQ.

to situate the design space.

3.7 Positioning of This Work

Table 3.1 makes the gap explicit. Decomposition is well established — PoG and GoG both do it. Backtracking exists, in PoG. Faithful grounding by construction exists, in RoG. But in the final column, *no surveyed system decomposes its draft answer into atomic claims and checks each against the retrieved triples before responding*. PoG’s Reflection corrects the search while it is running; RoG’s answers are grounded because of how they are produced, not because anything examined them. The slot is open.

AGR’s contribution is therefore positioned as follows.

The **primary** contribution is the Structural Verification Layer: a claim-level check against traversed triples, executed before the answer is emitted, with repair policies that either resume exploration or remove unsupported content. Chapter 6 specifies it.

The **secondary** contribution is a systematic component ablation quantifying what each agentic mechanism actually contributes, in accuracy and in tokens. As Table 3.1 shows, these mechanisms are typically reported as bundles; which of them earns its cost is not established in this literature.

A deliberate consequence of this positioning is that the thesis does not depend on beating prior

systems on raw Hits@1. The comparison that matters is against the same architecture with the verification layer removed, on the same graph, with the same backbone — which is why the ablation in Section 8.7, rather than Table 3.2, carries the argument. Where AGR is compared against a prior system, that comparison is run under controlled conditions in this thesis’s own environment rather than quoted across papers.

Chapter 4

The Knowledge Environment

Everything the agent can know, it knows from the graph. The environment therefore bounds the accuracy any system in this thesis can attain, and a result reported without that bound is uninterpretable. This chapter describes how the environment was built, what it contains, and — equally important — what it cannot express.

4.1 Design Goals

4.1.1 Decoupling the Graph Source from the Question Sets

The original proposal for this work described ingesting WebQSP and FactBench into a graph database. That description conflated two distinct roles, and the correction is worth stating because it shaped everything downstream.

WebQSP [19] is not a knowledge graph. It is a set of questions annotated with semantic parses over Freebase [18]; the graph is Freebase, and WebQSP supplies questions and gold answers against it. The two roles are kept strictly separate here: Freebase-derived triples constitute the environment, while WebQSP and ComplexWebQuestions [20] supply questions and gold answers and contribute no facts to it.

FactBench [44] was removed entirely. It had been intended both as a source of graph content and as the hallucination evaluator, which is circular: if the evaluator’s facts are inside the graph the system retrieves from, a low hallucination rate follows by construction. Groundedness is instead measured against the graph by an independent judge, as described in Section 7.6.

4.1.2 Why a Curated Subgraph Rather Than LLM-Extracted Triples

The proposal also specified a language-model entity-relationship extractor to build the graph. This is appropriate when the source is unstructured text, and it is what GraphRAG does [4]. It is

self-defeating here. The graph is the reference against which model output is verified; building that reference with a language model would introduce the very error class the thesis measures, and no claim about hallucination reduction would survive the observation that the ground truth was itself generated.

The environment is therefore assembled exclusively from curated Freebase triples. Language models are used to *navigate* the graph and to *judge* answers, never to populate it.

4.2 Source Data

4.2.1 The Freebase Snapshot

Freebase has been frozen since 2016, which makes it stable and reproducible while also dating it — a property that cuts both ways and motivates the no-retrieval control in Section 7.4. Two routes to a working Freebase environment were considered.

The first is a full SPARQL endpoint: the community-standard preprocessed Virtuoso image, which ships a database of over 53 GB and whose maintainers recommend on the order of 100 GB of RAM for acceptable query latency. Its advantage is a complete and realistic search space, including the hub entities of very high degree that make naive traversal explode.

The second is the per-question subgraph distribution published alongside Reasoning-on-Graphs [9], in which each record pairs a question with its topic entities, gold answers, and the triples of a multi-hop neighbourhood around those topic entities.

The second route was chosen, for a reason that should be recorded plainly: the hardware for the first was not available. The consequence is a scoped environment rather than all of Freebase, and Section 4.7 quantifies what that costs.

4.2.2 WebQSP and ComplexWebQuestions as Question Sets

WebQSP [19] contains questions whose answers lie predominantly one or two hops from the topic entity; it descends from WebQuestions [23]. ComplexWebQuestions [20] is constructed by composing WebQSP queries into deeper chains with conjunctions, superlatives, and comparatives, and is the genuine multi-hop benchmark of the two. Reporting both is deliberate: WebQSP establishes comparability with prior work, and CWQ is where the multi-hop claim is actually tested.

4.3 Graph Construction

4.3.1 Union of Per-Question Subgraphs from the RoG Distribution

The environment is the *union* of every per-question subgraph in the RoG distribution, across both datasets and all three splits. Each record’s triple list is read, whitespace-normalised, and inserted into a global deduplicated set; entity and relation strings are interned to integers so that the union and its adjacency structure fit in memory.

Unioning is not incidental. A per-question subgraph in isolation is a near-oracle: it was extracted to contain the answer, so an agent searching it faces almost no distractors. Merging every question’s neighbourhood into one global graph restores distractor mass, since the agent exploring one question’s topic entity encounters edges contributed by unrelated questions. WebQSP contributes 3,791,302 unique triples over 1,298,304 entities; adding CWQ brings the union to 8,309,194 triples over 2,592,892 entities.

Two consequences must be disclosed. First, the RoG subgraphs were pre-extracted so as to guarantee answer reachability, which makes the resulting environment friendlier than raw Freebase — fewer irrelevant hub explosions, and a higher answer-reachability rate than a BFS extraction from the full store would produce. This is a threat to external validity, revisited in Section 9.9.3. The mitigating argument is that *every system compared in this thesis navigates this identical environment*, so relative comparisons remain sound even where absolute numbers are optimistic.

Second, the distribution’s triples are keyed by human-readable entity names rather than by Freebase machine identifiers. The node identifier is therefore the name string, which means distinct real-world entities sharing a surface form are merged into a single node. This is inherent to name-keying and is also revisited in Section 9.9.3.

4.3.2 Relation Filtering and Noise Removal

An extraction from raw Freebase would require aggressive predicate filtering: administrative namespaces, type and key predicates, and above all `type.type.instance`, which contributes millions of edges per type node. Degree capping would likewise be mandatory, since uncapped breadth-first expansion from a hub entity explodes at the second hop.

No such filtering was applied here, and the reason is structural rather than an oversight: the RoG distribution ships subgraphs that have already been scoped around topic entities, so the administrative predicates and hub explosions that filtering exists to remove are largely absent from the source. The only transformation applied to relation names is sanitisation into Cypher-legal identifiers, described in Section 4.4. What survives is a vocabulary of 7,058 distinct relation types, which is the relation search space the agent faces at every exploration step.

4.3.3 Treatment of Mediator Nodes and Its Effect on Hop Counts

Freebase reifies n -ary facts through mediator nodes, as described in Section 2.1. In the name-keyed distribution these surface distinctively: named entities appear as their names, while unnamed mediators survive as raw machine identifiers of the form `m.0d05w3` or `g.11b62t84jq`. Nodes are flagged as mediators by matching that pattern, and the flag is stored as an `is_cvt` property.

Mediators are *retained*, not collapsed into hyper-edges. Retaining them preserves the facts they encode — systems that skip mediators lose exactly those facts [11] — at the cost of inflating raw hop counts, since one logical hop through a mediator is two graph edges. Every hop budget in this thesis is therefore stated in *raw* hops, and the conversion is roughly two raw hops per logical hop.

The scale of this decision is easy to understand: **1,652,618 of the 2,592,892 nodes — 63.7% — are mediator-form nodes carrying no human-readable name.** Nearly two thirds of the environment is connective tissue rather than nameable entities, which is why the tool layer in Section 5.2 must traverse through mediators transparently rather than present them to the agent as candidate answers.

4.4 Graph Store Construction

4.4.1 Property-Graph Schema

Entities become `:Entity` nodes carrying an integer `id`, the name string, and the boolean `is_cvt` flag. Triples become typed relationships whose type is the sanitised relation name — non-alphanumeric characters replaced by underscores, since Freebase predicates contain dots that are illegal in unquoted Cypher relationship types — with the original dotted string retained as an `fb_name` property.

Native relationship types were chosen over a single generic type carrying a name property. The alternative simplifies import but slows filtered expansion, and filtered expansion is the agent’s hottest operation: every exploration step retrieves the relations of an entity and then the neighbours along one of them.

Integer identifiers were chosen over names as the import key. Entity names contain commas, quotation marks, and unicode, all of which invite escaping bugs and silent collisions in bulk-import CSVs. The name survives as an ordinary property and is what the entity resolver matches against.

Table 4.1: The knowledge environment as constructed and imported.

Property	Value
Entities (nodes)	2,592,892
Triples (relationships)	8,309,194
Node and relationship properties	16,087,870
Distinct relation types	7,058
Mediator-form nodes (is_cvt)	1,652,618 (63.7%)
Triples contributed by WebQSP subgraphs	3,791,302
Entities after WebQSP alone	1,298,304
Rows skipped during import	0
Import wall-clock time	36.4 s
Peak import memory	1.09 GiB
Neo4j version	Community 5.26.28

4.4.2 Bulk Import into Neo4j

Import used the offline bulk importer of Neo4j Community 5.26.28, which operates on a stopped database and is orders of magnitude faster than transactional loading at this scale. Duplicate nodes and bad relationships were configured to be skipped rather than to abort the import.

After import, a uniqueness constraint on Entity.id and a full-text index on Entity.name were created; the latter is what the resolver’s lexical tier queries.

4.4.3 Graph Statistics

Table 4.1 records the environment as built. These figures, together with the dataset versions and the hop cap, constitute the reproducibility record for the environment.

The zero skipped rows deserve emphasis, because the import was deliberately configured to tolerate skips. Node and relationship counts in the database match the CSV row counts exactly, so the count-reconciliation check of Section 4.7 passes without any unexplained gap to account for.

4.5 The Vector Index

4.5.1 Entity Embeddings

Entity names were embedded with all-MiniLM-L6-v2, a 384-dimensional sentence encoder [29] chosen for CPU throughput at this node count. Mediator nodes were skipped: embedding a string such as m.0y5k79m produces noise, and by Section 4.3 that is almost two thirds of the graph, so skipping them is also the difference between a tractable and an intractable embedding job. Vectors were written back into Neo4j and indexed with a cosine-similarity vector index.

4.5.2 Relation-Vocabulary Embeddings

The scoring function of Section 5.5 compares candidate relations against the current sub-objective in embedding space, so the 7,058 relation names are embedded once and held in memory rather than embedded per step.

Two details make this work. Relation names are *verbalised* before encoding: `people.person.place_of_birth` becomes “people person place of birth”, with consecutive duplicate tokens removed. The encoder was trained on natural language, and the verbalised phrase lands near a question like “where was X born” in embedding space where the raw dotted path does not. And relations are embedded with the *same* model as entities and sub-objectives, since vectors from different models occupy unrelated spaces and their cosine similarity is meaningless.

The resulting artefact is a 7058×384 float32 matrix with a parallel name list. Because vectors are L2-normalised at encoding time, similarity against the whole vocabulary is a single matrix–vector product.

A functional check confirms the verbalisation is doing its job. Encoding the probe “where was the person born” and ranking the vocabulary against it returns `people.person.place_of_birth` first at cosine 0.709 and `location.location.people_born_here` — the inverse relation — second at 0.675, with the remaining top-five entries plausible near-misses drawn from birth-related and fictional-character namespaces. A degenerate verbaliser would have produced an unordered top five here.

4.5.3 Scope: Linking and Candidate Ranking, Never Ground Truth

The role of embeddings in this thesis is deliberately narrow. They resolve question phrases to graph entities, and they pre-rank relation candidates so the language-model scorer can be applied to a short list rather than to thousands of relations. They never establish that a fact holds. Every factual check — whether a triple exists, whether an entity is reachable, whether a claim is supported — is answered by symbolic graph queries. Nothing in the verification layer of Chapter 6 consults an embedding.

4.6 Entity Resolution and Surface-Form Linking

4.6.1 The Exact, Lexical, and Vector Cascade

Mapping a question’s surface mention to a graph node uses a three-stage cascade, each stage attempted only if the previous one fails.

Exact matching queries for a node whose name equals the normalised mention. Because the environment is name-keyed and the benchmarks supply topic entities as names, this resolves

Table 4.2: Entity resolution over all topic-entity mentions in both test sets. A mention counts as resolved only when the returned node’s name equals the mention exactly.

Dataset	Mentions	Exact stage	Unresolved
WebQSP	1,661	1,661 (100%)	0
CWQ	5,488	5,478 (99.8%)	10
Total	7,149	7,139 (99.86%)	10 (0.14%)

nearly everything.

Lexical matching queries the full-text index, handling typographical variation and partial names. Mentions are escaped for Lucene syntax before the query: benchmark entity names contain parentheses, colons, and slashes, all of which are Lucene operators that would otherwise crash or silently distort the query.

Vector matching encodes the mention and queries the entity vector index, handling paraphrases and misspellings that survive the first two stages.

The cascade is referred to by these names throughout, never as “tiers”, which this thesis reserves for the two-tier groundedness metric of Section 7.5.3.

4.6.2 Threshold Selection on Development Data

The lexical stage accepts its top hit only when the full-text score exceeds 1.0, falling through to vector matching otherwise. The threshold exists because the full-text index returns a ranked list for almost any input, including inputs that match nothing meaningfully; without a floor, the lexical stage would confidently return the least-bad token overlap and the vector stage would never be reached.

Resolution was measured over every topic-entity mention in both test sets, and Table 4.2 reports the outcome.

Entity linking is therefore not a bottleneck in this environment, and it can be excluded as an explanation for downstream failures — a claim the error analysis of Chapter 9 relies on.

4.7 Environment Validation Gate

No experiment was run until the environment was certified. The gate has three parts of increasing strength.

4.7.1 Answer-Reachability Protocol

For every test question, two properties are computed over the union graph. *Existence* asks whether the gold answer entities appear as nodes at all. *Reachability* asks whether at least one

Table 4.3: Environment coverage over the full test sets, at a bound of four raw hops. The reachability column is the Hits@1 ceiling for every system evaluated in this thesis.

Dataset	Test questions	All answers exist	≥ 1 exists	≥ 1 reachable
WebQSP	1,628	94.7%	97.3%	97.3%
CWQ	3,531	99.4%	99.7%	99.7%

gold answer is connected to at least one topic entity within a bounded number of hops.

Reachability is computed by breadth-first search treating edges as *undirected*, because the agent’s neighbour-retrieval tool looks in both directions and the gate must model the agent’s actual traversal semantics. The bound is four raw hops, which by Section 4.3 is approximately two logical hops through mediators — enough for all of WebQSP and most of CWQ. A question whose topic entity is itself a gold answer counts as reachable at zero hops.

This number is the **accuracy ceiling**: if a gold answer is unreachable, no navigating system can find it, however well designed.

4.7.2 Coverage Results and the Induced Accuracy Ceiling

Table 4.3 reports the gate.

Three observations follow. Every question in both datasets has at least one topic entity present in the graph, so no question is lost at the entry point.

More strikingly, *existence and reachability are identical* — 1,584 of 1,628 for WebQSP and 3,519 of 3,531 for CWQ. Every gold answer that exists in this environment is reachable within four hops. Reachability is therefore never the binding constraint; when an answer is unavailable it is because the entity is absent altogether. This is a direct consequence of the construction in Section 4.3: the subgraphs were extracted around topic entities precisely to guarantee reachability, and the measurement confirms that the property survived unioning.

Finally, CWQ’s coverage exceeds WebQSP’s, which inverts the expected ordering — the deeper benchmark has the better-covered answers. The explanation is again constructional rather than semantic: CWQ contributes far more questions and therefore far more subgraphs to the union, so its own answers are better represented. The residual WebQSP shortfall is dominated by answers that are not entities at all, which Section 4.7.4 takes up.

4.7.3 Analysis of Linking Misses and Gate Failures

The ten unresolved mentions of Table 4.2 are not a random residue. Every one is a bare machine identifier of the form g.123852dg appearing as a CWQ topic entity — an unnamed mediator node that the benchmark supplies where a named entity was expected. Exact matching fails because no node carries that string as a *name*, and the lexical stage then returns a meaningless

single-character match. The correct characterisation is that these questions are anchored on nodes with no surface form, not that resolution is unreliable.

The strongest gate check re-verifies the Python-computed reachability against the live database, confirming that nothing was lost between the CSV files and the imported graph. A random sample of 400 questions flagged reachable, drawn under a fixed seed, was re-tested with a shortest-path query in Neo4j. The pass criterion is strict — these were already known to be reachable, so anything below 100% would indicate dropped edges. **All 400 verified; the failure set is empty.** Together with the zero skipped import rows and the exact count reconciliation, the environment is certified.

4.7.4 What the Environment Cannot Express

Certifying reachability establishes that answers which *are* entities can be found. It says nothing about answers that are not entities, and this is where the environment’s real limits lie.

The import schema of Section 4.4 gives every node a single `:Entity` label. There is no typed literal node, no datatype, and no distinction between an entity and a value. A date or a number can therefore exist in the graph only incidentally, as an entity whose *name* happens to be a numeric string. Measured over all 2,592,892 nodes:

- **Dates** are effectively absent. Only 77 nodes carry a full YYYY-MM-DD name and 175 a bare four-digit year — together under 0.01% of the graph, and untyped, so they cannot be compared, ordered, or filtered by range.
- **Numeric values** appear on 12,622 nodes (0.49%), but the population is dominated by identifiers such as ISBNs rather than by measurable quantities.
- **Ordinals and superlatives** are not expressible at all. This is a property of the tool layer rather than the data: Section 5.2 exposes no aggregation, sorting, or comparison operator, so “the smallest” or “the earliest” cannot be computed even when every candidate is present in the graph.

This defines a class of questions that is *unanswerable in this environment*: any question whose gold answer is a date, a measured quantity, or a superlative selection over candidates. Such a question is not a reasoning failure when the system misses it — the environment could not have supplied the answer. The class is referenced by name in Section 9.6, where it accounts for a substantial share of the residual failures, and it is a known consequence of a design decision: modelling literals as typed nodes was considered and not adopted, in exchange for a uniform schema and a simpler tool API.

Chapter 5

The AGR Framework: Architecture and Navigation

This chapter specifies the Agentic Graph Reasoning framework: how the agent is structured, what operations it may perform on the graph, how it decides where to look next, how it recovers from wrong turns, and what guarantees bound its cost. The verification layer, which is the thesis’s principal contribution, is separated into Chapter 6; everything here is the machinery that produces the traversal it verifies.

5.1 Architectural Overview

5.1.1 The Controlled-Agent Pattern

There are two ways to build a graph-navigating agent. The first hands the model a set of tools through native function-calling and lets it drive the loop itself, ReAct-style [27]. The second inverts control: the *program* orchestrates the loop, calls the graph tools deterministically, and consults the model only at decision points, through constrained prompts that must return JSON. AGR uses the second, and the choice is deliberate enough to be worth defending, because it determines what the word “agentic” means in this thesis. Four consequences follow:

- **Every graph operation is logged by construction.** The traversal is not reconstructed from the model’s narration of what it did; it is the literal sequence of tool calls the program made.
- **Budgets are enforced in code, not requested in prompts.** A prompt asking the model to stop after four hops is a suggestion. A counter checked in a router is a guarantee (Section 5.8).

START → **Planner** → **Explorer** → **Evaluator**

Evaluator routes on `route_after_eval`:

- continue → **Explorer**
- backtrack → **Backtracker** → **Explorer**
- answer → **Verifier**

Verifier routes on `route_after_verify`:

- grounded → **Answerer**
- retry → **Explorer**
- give_up → **Answerer**

Answerer → END

Figure 5.1: The AGR state machine. Two cycles exist: Explorer ↔ Evaluator for iterative search, and Verifier → Explorer for verification-driven re-exploration. Both are bounded by the budgets of Section 5.8 rather than by the graph structure.

- **Failures are reproducible.** A malformed model response degrades a decision; it cannot corrupt the control flow.
- **Each model call is small, single-purpose, and cacheable**, which is what makes the response cache of Section 5.9 viable.

The agency in AGR therefore lives in the *decision policy* — which sub-objectives to pursue, which edges look promising, whether the current path is a dead end, whether the draft answer is supported — and not in unconstrained tool invocation. Prior systems in this space make the same choice [7, 8]; stating it explicitly forecloses the objection that AGR is merely a prompt around a function-calling loop.

5.1.2 The State Machine

The framework is a directed graph of six nodes over a shared typed state, built with LangGraph. Figure 5.1 gives the topology.

Two properties of this topology matter. It contains *cycles*, which is why a state-machine framework is used rather than a linear pipeline: exploration loops until the evaluator is satisfied, and verification can send the agent back to explore. And the verifier sits *between* the decision to answer and the answer itself, which is the structural expression of this thesis’s contribution — there is no path from evaluator to answerer that does not pass through verification.

Table 5.1: The graph tool API. All operations are deterministic parameterised Cypher; none consults an embedding except `search_entity`, which delegates to the resolver of Section 4.6.

Operation	Returns	Role
<code>search_entity(s, k)</code>	up to k candidate entities with id, name, score, and resolver stage	anchor seeding; the only entry point from text to graph
<code>get_relations(e)</code>	relation types incident to e in <i>both</i> directions, each with its fanout count	candidate generation for the scorer
<code>get_neighbors(e, r, d)</code>	entities reached from e along r in direction d , each with the via chain that reached it	frontier expansion
<code>verify_triple(h, r, t)</code>	{direct, via_cvt, supported}	claim checking (Section 6.3.1)
<code>verify_connection(a, b)</code>	{direct, via_cvt, connected}	relation-agnostic claim checking (Section 5.2.3)

5.2 The Graph Tool API

The five operations are specified in full, with the Cypher each executes, in Chapter B.

5.2.1 Rationale: Constrained Tools Instead of Free-Form Cypher

An obvious alternative is to let the model write Cypher directly. This was rejected. Generated-query approaches introduce a failure mode that is indistinguishable, in the results, from a reasoning failure: when a question is answered incorrectly, one cannot tell whether the agent reasoned badly or merely emitted invalid syntax. Since the thesis’s central claims are about reasoning and verification, a confound that contaminates every error category is unacceptable.

The tools are therefore parameterised Cypher templates (Section 7.10), written once and unit-tested. They accept entity identifiers and return names alongside them, so the model always sees human-readable text while the program tracks identity. Every invocation is appended to a JSONL log with its arguments, result, and latency.

5.2.2 The Five Operations

Table 5.1 lists the operations. Note that there are five, not the four originally planned: `verify_connection` was added when the verification layer of Chapter 6 required a relation-agnostic adjacency test.

Three design decisions inside these operations are load-bearing.

Relations are returned with fanout counts, in both directions. The count is free signal: a relation with a fanout of forty thousand is almost never the edge that answers a specific question, and the scorer can use this without an extra query. Retrieving incoming as well as outgoing edges matters because Freebase’s relation direction is a modelling artefact — a question about

where someone was born may be answerable through `people.person.place_of_birth` outward or `location.location.people_born_here` inward.

Administrative predicates are filtered at the tool layer. Relations whose names begin with `common.`, `freebase.`, `type.`, `kg.`, `user.`, `dataworld.`, `rdf-schema#`, or `owl#` are dropped before the candidate list is built. These carry no factual content, and Section 5.10 shows what happens when they are not filtered: they consume beam slots and drag the agent into the schema rather than the data.

The relation list is capped at 300 entries, sorted by fanout. An earlier cap of 80 was raised after it was found to hide precisely the low-fanout, high-precision relations that answer specific questions — again, Section 5.10.

5.2.3 Mediator Traversal and Relation-Agnostic Adjacency

Section 4.3 established that 63.7% of the environment’s nodes are unnamed mediators. If the agent saw these as candidate entities, most of its frontier would consist of nodes with no surface form and no meaning to a language model.

`get_neighbors` therefore hops through them transparently. When an expansion reaches a node flagged `is_cvt`, the tool immediately expands that node one further step, excluding the origin and excluding other mediators, and returns the entities on the far side. Each returned neighbour carries a `via` chain — one relation for a direct edge, two for a mediator-mediated one. The model reasons about logical neighbours; the log retains the full raw chain, so hop accounting stays exact.

`verify_triple` applies the same principle to checking: it matches *undirected* and accepts a mediator in the middle. Direction-strictness would generate false rejections, because the claims it checks come from generated text that has no notion of which way a Freebase edge points.

`verify_connection` goes further and drops the relation entirely, asking only whether two entities are adjacent directly or through one mediator. It exists because claim decomposition frequently produces a claim whose relation does not correspond to any single Freebase predicate — “X played for Y” may be realised through a roster mediator with no edge literally named for playing. Requiring an exact relation match in such cases rejects true claims; Section 6.3.1 explains where each of the two checks is used.

5.3 Agent State

The state is a typed dictionary threaded through every node. Table 5.2 groups its fields by purpose.

Table 5.2: The agent state. Fields marked *append-only* accumulate across node executions through reducers; all others are overwritten by whichever node returns them.

Group	Fields
Question	qid, question, gold_q_entities
Plan	plan: ordered sub-objectives, each with a status (pending/active/done) and its resolved entities
Search	anchors (current frontier), traversed (<i>append-only</i> triple log), backtrack_stack, banned, last_expanded
Answer	candidate_answers, draft, answer, answer_entities, supporting_triples
Verification	supported_claims, unsupported_claims, unsupported_claim_objs
Control	budget (meter, mutated in place), trace (<i>append-only</i>), and the router scratch keys _eval_decision, _last_n_new, _frontier_max_score

Two disciplines govern this structure and are worth stating because violating either produces bugs that are extremely hard to localise.

Nodes write, routers read. A router is a pure function that inspects state and returns an edge label; it never mutates. Any signal a router needs must therefore be deposited in state by the node preceding it — which is what the three underscore-prefixed scratch keys are for. The explorer records how many triples it added and the best score it saw; the evaluator records its decision; the routers read those and decide.

The budget meter is mutated in place and never returned as a delta. It is a mutable object shared by reference across all nodes, because counters that are merged rather than mutated can be silently lost when two nodes return overlapping state. Enforcement must not be able to drift.

5.4 The Planner

5.4.1 Decomposition into Ordered Sub-Objectives

The planner makes one model call, converting the question into an ordered chain of sub-objectives plus the entity mentions to seed the search. The chain is strictly sequential; later objectives reference earlier results with a #N back-reference. A general dependency-graph planner was not built, because a sequential chain covers the composition structure that dominates CWQ and a DAG planner is a research problem in its own right.

The prompt is reproduced in full as Section A.1. It is few-shot with one example per question archetype — single hop, composition, conjunction, superlative — and the single-hop example exists to demonstrate that *not* decomposing is a valid output. Over-decomposition of simple questions is a real failure mode, and Section 8.7 shows it is not hypothetical: on the shallower benchmark, removing the planner improves accuracy.

The prompt also instructs the model to extract only specific named entities as topic mentions, and explicitly not generic type words such as “celebrity”, “university”, “country”, or “senator”. That instruction was added in response to evidence discussed in Section 5.10: type words resolve successfully to type nodes in the graph, consuming anchor slots with entities that lead nowhere.

5.4.2 Plan Validation and Degenerate-Plan Handling

The planner’s output is validated, not trusted. Four checks run: the plan must contain a non-empty list of sub-objectives; it must contain no more than four; every #N reference must point strictly backwards; and topic mentions must be present. The back-reference check catches the planner’s most common structural error, a step that references itself or a future step.

If any check fails — or if the model call itself raises — the planner falls back to a single sub-objective consisting of the whole question, seeded with the dataset’s topic entities. This is a deliberate design principle: **planning can degrade but can never abort a run**. A degraded plan is a data point that appears in the results; a crash is a lost question that biases them.

Anchor seeding has two modes. In the default benchmark configuration, anchors are seeded from the dataset’s annotated topic entities, which isolates reasoning from entity linking and is the standard protocol in this literature [7, 9]. The alternative seeds from the planner’s own extracted mentions, giving the end-to-end number. Because Section 4.6 showed resolution succeeds on 99.86% of mentions, the two modes differ very little in this environment.

Setting the planner ablation flag bypasses the model call entirely and installs the fallback plan directly, which is what makes “no planner” a one-configuration-line change rather than a separate code path.

5.5 The Explorer

The explorer performs one expansion of the frontier. It is the only node that touches the graph in bulk, and its structure is the heart of the navigation policy.

5.5.1 Frontier Construction and the Single Scoring Pass

The node proceeds in five steps:

1. **Checkpoint.** The current frontier, the current depth, and the best score seen at this point are pushed onto the backtrack stack.
2. **Gather.** For every anchor, `get_relations` is called and the results are collected into a *single* candidate list, each row tagged with the anchor it came from.

3. **Score.** The whole list is scored in one pass (Section 5.5.2). Pooling candidates across anchors before scoring is what allows the hybrid scorer to consult the model *once* per expansion rather than once per anchor.
4. **Beam.** Candidates are taken in descending score order into per-anchor buckets of width three, skipping any (anchor, relation, direction) triple on the ban list (Section 5.6.3).
5. **Expand.** Each selected edge is expanded with `get.neighbors`; the resulting triples are appended to the traversal log, and the neighbours become frontier candidates.

The new frontier is deduplicated by entity identifier, sorted by the score of the edge that produced each entity, and truncated to the anchor cap. Sorting by score rather than by insertion order matters: an insertion-ordered frontier keeps whichever entities happened to be produced first, which causes the agent to drift away from the anchor that mattered.

5.5.2 The Hybrid Scoring Function

Each candidate edge receives a score blending semantic similarity with a model judgement; the prompt carrying the second is Section A.2:

$$\text{score}(c) = \alpha \cdot \cos(\mathbf{v}_{\text{rel}(c)}, \mathbf{v}_{\text{obj}}) + (1 - \alpha) \cdot \text{LLM}(c),$$

where $\mathbf{v}_{\text{rel}(c)}$ is the precomputed embedding of the candidate’s relation name (Section 4.5), $\mathbf{v}_{\text{obj}}$ is the embedding of the *rendered active sub-objective*, and $\text{LLM}(c) \in [0, 1]$ rates how likely following that edge from that anchor is to advance the sub-objective.

Three properties of this design deserve emphasis.

The objective, not the question, is embedded. Hop two of a two-hop question is scored against “find the director of Titanic”, with the back-reference already substituted by the resolved entity name, rather than against the original question text. This is the concrete mechanism by which decomposition is supposed to help, and isolating it is what makes the planner ablation interpretable.

The model scores only a shortlist. Candidates are pre-ranked by embedding similarity and only the top twenty are shown to the model, in one batched call. Without the pre-filter, an expansion from five anchors with hundreds of relations each would be unaffordable.

The blend is applied after the call, not inside it. The scoring prompt contains no configuration value — no α , no τ . This is not cosmetic: because the response cache (Section 5.9) keys on prompt content, a prompt free of configuration means every condition in an α sweep shares the same cached model responses, and the sweep costs one set of calls instead of one per condition. Embedding a configuration value in that prompt would multiply sweep cost by the number of conditions without changing any result.

If the model call fails or the budget is exhausted, the scorer degrades silently to embedding-only scoring rather than aborting the expansion. Setting $\alpha = 1$ disables the model term entirely, which is the embedding-only ablation.

5.6 The Evaluator and Routing Policy

5.6.1 Sub-Objective Completion Judgment

The evaluator makes one model call per expansion. It is shown the active sub-objective, a window of retrieved facts, and the remaining budget, and returns a decision (continue, backtrack, or answer), whether the objective is complete, and which entities satisfy it.

The fact window is *relevance-ranked*, *not recent*. Traversed triples are verbalised, embedded, and the thirty most similar to the current objective are shown, in traversal order. The distinction is not incidental: a large expansion can add hundreds of triples, and a window showing the most recent thirty will push the answer out of view in exactly the situations where the expansion succeeded. Section 5.10 documents the case that forced this change.

5.6.2 The Groundedness Filter on Resolutions

The evaluator's resolved entity names are not accepted as given. Each is looked up against a table built from the tail entities of the *traversed* triples, and any name that does not appear there is discarded.

This filter is small in code and significant in effect. A language model asked which entities satisfy an objective will readily list entities it knows from training but never retrieved. The filter refuses to launder that parametric recall into the agent's evidence. Section 5.10 records a case in which the evaluator confidently resolved nine correct countries that appeared in no retrieved triple, and the filter rejected all nine — costing a point on that question while preserving the property that every entity the system proposes came from the graph.

Resolutions that survive the filter are accumulated into `candidate_answers` on *every* evaluation, whether or not the objective was judged complete. An earlier design discarded resolutions when the objective was incomplete, which threw away correct answers found mid-run.

When an objective completes and further objectives remain, the plan advances, the decision is forced to continue, and the resolved entities become the next frontier. When the last objective completes, the decision is forced to answer.

5.6.3 Backtracking Triggers and the Backtrack Stack

Backtracking fires on the first of three conditions, evaluated in this order:

1. **Dead end** — the last expansion produced no new triples.
2. **Low score** — the frontier’s *signal maximum* fell below the threshold τ .
3. **Evaluator veto** — the model judged the direction a dead end.

The second trigger deserves a precise statement, because it is easy to assume it tests the blended score and it does not. The quantity compared against τ is

$$\sigma = \max\left(\underbrace{\max_{c \in E} \text{score}(c)}_{\text{blended, expanded edges}}, \underbrace{\max_{c \in R} \cos(v(r_c), v(o))}_{\text{embedding, all candidates}}, \underbrace{\max_{c \in R_K} s_{\text{LLM}}(c, o)}_{\text{model, rated candidates}} \right),$$

the largest of three signals: the best blended score among the edges actually expanded, the best raw embedding similarity over every candidate relation the anchors offered, and the best model score over the top- K candidates the model was asked to rate. The trigger therefore fires only when *all three* signals are weak. A frontier where the blend is depressed — because α happens to discount whichever signal is confident here — does not backtrack if either underlying signal still points somewhere. This makes the trigger deliberately conservative: it abandons a branch only on unanimous evidence of hopelessness, and Section 8.1 shows the consequence, which is that τ barely fires at the values swept.

The order matters. The first two are deterministic and cost nothing; checking them before deferring to the model’s judgement means a mechanically hopeless frontier is abandoned without an argument about it.

The backtracker does three things. It pops the *highest-scoring* snapshot from the stack — not the most recent one, which would make backtracking a simple undo. It restores that snapshot’s anchors and depth. And, critically, it adds every (anchor, relation, direction) triple expanded since that snapshot to a **ban list** that the explorer consults on its next pass.

The ban list is what makes backtracking a search operation rather than a loop. Without it, restoring a frontier is pointless: a deterministic scorer facing an identical frontier selects identical edges, producing identical facts and an identical decision to backtrack, until the backtrack budget is exhausted. With it, a backtrack forces the scorer to its next-best alternatives, which is what “exploring an alternative branch” actually requires. Section 5.10 shows both behaviours in the logs.

5.6.4 Routing

`route_after_eval` is pure Python and enforces budgets before preferences. Wall-clock and depth are checked first; if either is exhausted the agent is routed to answer with whatever it has. If a backtrack trigger fired, the agent backtracks if the backtrack budget permits and otherwise

Table 5.3: Budget configuration. The configuration is hashed and the hash recorded in every run record, so results can always be attributed to the configuration that produced them.

Parameter	Value	Enforced in
max_depth	4	route_after_eval
beam_width	3	explorer (slice)
max_anchors	5	explorer (slice)
max_backtracks	3	route_after_eval
max_llm_calls	25	LLM client
max_verify_iters	2	route_after_verify
max_seconds	300	route_after_eval

answers. Only then is the evaluator’s own decision honoured. Routers are deterministic by construction: no adversarial model output can change what they do.

5.7 The Answerer

The answerer is a finalizer rather than a generator. Which of its three paths runs is decided entirely by what the verification layer left in state, and two of the three are specified in Chapter 6: the *verified* path, where no claim was unsupported and the draft is emitted with no further model call, and the *repair* path, where unsupported claims remain and one rewrite call restates the answer around a deterministically filtered entity list (Section 6.5). On the development set the verified path handled 77 of 80 questions at zero additional cost.

The third path exists for the case where the verifier raised before producing a draft at all. It makes one model call over the traversed facts and the accumulated candidates. If the model budget is exhausted it falls back without a call to the resolved entities of completed plan steps, or failing that to the accumulated candidates, producing a degraded but non-empty answer. If the call raises for any other reason, the error is recorded in the trace and the answer is an explicit failure string.

That last distinction is deliberate. An earlier version caught all exceptions as budget exhaustion, which silently misreported genuine errors as expected behaviour — exactly the kind of defect that inflates apparent robustness.

5.8 Budgets and Termination Guarantees

Budgets do three jobs: they bound cost, they guarantee termination, and they produce the efficiency data of Section 8.6. Table 5.3 lists them.

Each budget has exactly one enforcement site, which is what keeps enforcement auditable. The model-call budget is checked inside the client before every call, so no node can bypass it;

exceeding it raises an exception that each node catches into a degraded path rather than a crash. Depth, backtracks, and wall-clock are checked only in the post-evaluation router. Verification iterations are checked only in the post-verification router. Beam width and anchor cap are not checks at all but slices inside the explorer.

Together these give a termination guarantee that does not depend on model behaviour: every cycle in Figure 5.1 passes through a router that decrements or checks a monotone counter, so no sequence of model outputs, however adversarial, can produce a non-terminating run. This property is verified directly by the mechanics tests of Section 7.10, which script an evaluator that always says continue and assert that the depth budget halts it.

5.9 Instrumentation and Reproducibility

Two logs are written per run. The *tool log* records every graph operation with its arguments, result size, and latency. The *run log* records, per question: the answer and its extracted entities, the draft, the verification outcome and any unsupported claims, the count of supporting triples, every backtrack with its trigger reason and restored depth, the full decision trace, the budget snapshot, wall-clock time, and — for provenance — the backbone description, the budget configuration hash, the run configuration, and the current git commit.

The response cache deserves separate description: it underwrites the thesis’s reproducibility claims. Every model call is keyed by a hash over the model identifier, temperature, reasoning-effort setting, and the full prompt text; hits are served from disk. The subtle part is that a cache hit must be *indistinguishable* from a live call inside the agent: the budget check still runs, the call counter still increments, and the original token counts are replayed into the meter from the stored record. A fully cached rerun therefore reproduces the original run’s budget snapshots exactly, rather than reporting zero tokens. A separate counter records cache hits so that live and replayed runs can still be told apart in analysis.

This has three consequences. Repeated experiments cost nothing. A code change that is not supposed to alter behaviour can be verified by confirming that a rerun is served entirely from cache. And because the key includes the dated model snapshot, a silent server-side change to a model alias cannot poison previously cached results.

5.10 Design Validation on the Development Set

The first end-to-end run of the assembled framework, over a twenty-question development set drawn from the training splits, answered roughly three to four questions correctly. The final configuration answers thirteen to fourteen. The interval between those numbers is not incidental

Table 5.4: Root causes of failure in the first end-to-end run, and the mechanism each one motivated. Every fix is described in the section indicated.

Observed failure	Mechanism introduced	Section
Backtracking was a deterministic no-op: the restored frontier produced identical scores, facts, and decisions until the budget drained	Ban list on abandoned (anchor, relation, direction) triples	Section 5.6.3
The evaluator’s fact window showed the most recent thirty triples, so a large successful expansion pushed the answer out of view	Relevance-ranked fact window	Section 5.6
The relation list, capped at 80 by descending fanout, hid low-fanout precise relations behind hub relations	Cap raised to 300, plus the administrative-predicate blocklist	Section 5.2
Entities resolved while an objective was still incomplete were discarded	candidate_answers accumulated on every evaluation	Section 5.6
The frontier was truncated in insertion order, so the agent drifted away from the anchor that mattered	Frontier sorted by edge score before truncation	Section 5.5

tuning; it is where several of the mechanisms described above were shown to be necessary, and it is documented here because each fix is evidence that a component earns its place.

The development set was constructed to exercise specific machinery rather than to sample the benchmark distribution: six single-hop questions, eight two-hop compositions, three conjunctions, two whose shortest path traverses a mediator, and one question about a deliberately non-existent entity whose correct answer is a refusal. Questions were drawn from the training and validation splits only, never from test, so that everything tuned against them remains outside the evaluation.

The first run produced no crashes, terminated cleanly on every question, and abstained rather than confabulated in almost every failure — which was the actual exit criterion for the framework’s mechanics. But its thirteen failures collapsed into five systematic causes, summarised in Table 5.4.

Three observations from this exercise are worth carrying forward, because they bear directly on later chapters.

The two confabulations in the first run were caused by a retrieval defect, not a generation defect. Two questions about national currencies produced a confident “United States Dollar”. The relation cap had hidden the currency_used relation; the beam surfaced triples about GNI per capita denominated in dollars; and the answerer pattern-matched. No retrieved triple supported the answer. After the cap was raised both questions resolved correctly — and at roughly a third of the token cost. These two traces are the motivating example for Chapter 6: a claim-level check against the traversed triples would have caught them regardless of the retrieval defect, which is precisely the argument for verifying before emitting.

The groundedness filter on resolutions was validated by a failure. On one question the evaluator listed nine correct countries; none appeared in any retrieved triple, because the beam had gone down an unrelated branch. The filter rejected all nine, and the system abstained. This cost a point. It also demonstrated, one layer earlier than intended, that a language model inside an agentic scaffold will assert entities it never retrieved — and that catching this requires an explicit check, because nothing about the model’s confidence distinguishes the nine countries it recalled from entities it actually found.

One instrumentation defect was found and removed. A counter reporting how many banned expansions had been skipped was computing the number of anchors instead. It was reporting a plausible-looking number that was unrelated to what it claimed to measure. It was corrected rather than deleted, but the episode motivates a standing rule adopted for the remainder of the work: a metric that has never been checked against a case where its correct value is known independently should not be trusted, and should not be analysed.

5.11 The Complete Navigation Algorithm

Algorithm 1 states the loop.

Algorithm 1 AGR navigation (verification layer abstracted; see Chapter 6)

Require: question q , graph G , budgets B , blend α , threshold τ

Ensure: answer A with supporting triples S

```

1:  $P \leftarrow \text{PLAN}(q)$ ; if invalid then  $P \leftarrow [q]$ 
2:  $F \leftarrow \text{SEEDANCHORS}(P)$ ;  $T \leftarrow \emptyset$ ;  $C \leftarrow \emptyset$ ;  $stack \leftarrow \emptyset$ ;  $ban \leftarrow \emptyset$ 
3: while budgets permit do
4:   push  $(F, depth, \sigma)$  onto  $stack$   $\{\sigma$  from the previous pass; 0 initially $\}$ 
5:    $R \leftarrow \bigcup_{e \in F} \text{GETRELATIONS}(e)$ 
6:    $scored \leftarrow \alpha \cos(\cdot) + (1 - \alpha) \text{LLM}(\cdot)$  over  $R$  against the active objective
7:    $E \leftarrow \text{top-}\beta$  of  $scored$  per anchor, excluding  $ban$ 
8:    $T \leftarrow T \cup \text{GETNEIGHBORS}(E)$ ;  $F \leftarrow \text{top-}m$  new entities by score
9:    $\sigma \leftarrow \max(\max_{c \in E} scored(c), \max_{c \in R} \cos(\cdot), \max_{c \in R_K} \text{LLM}(\cdot))$   $\{\text{signal maximum}\}$ 
10:   $(d, done, \rho) \leftarrow \text{EVALUATE}(\text{objective}, \text{TOPFACTS}(T))$ 
11:   $\rho \leftarrow \{x \in \rho : x \text{ appears in } T\}$ ;  $C \leftarrow C \cup \rho$   $\{\text{groundedness filter}\}$ 
12:  if  $done$  then
13:    advance  $P$ ; if no objective remains then  $d \leftarrow \text{answer}$  else  $F \leftarrow \rho$ 
14:  end if
15:  if  $|T_{\text{new}}| = 0$  or  $\sigma < \tau$  or  $d = \text{backtrack}$  then
16:     $ban \leftarrow ban \cup E$ ;  $(F, depth) \leftarrow \text{pop highest-scoring snapshot}$ 
17:  else if  $d = \text{answer}$  then
18:    break
19:  end if
20: end while
21:  $(A, S) \leftarrow \text{VERIFYANDANSWER}(q, T, C)$   $\{\text{Chapter 6}\}$ 
22: return  $(A, S)$ 

```

Chapter 6

The Structural Verification Layer

Chapter 5 described a navigation loop that ends with a set of traversed triples and a set of accumulated candidate entities. Nothing in that loop constrains what the final answer text *asserts*. This chapter specifies the layer that does, together with the repair, filtering, and evidence-reporting policies built on it. Every quantity reported here is recomputed from the archived development-set run at the frozen configuration ($\alpha = 0.7$, $\tau = 0.20$, `verify_claims = true`, $n = 80$), and where a behaviour is established on a handful of questions rather than a distribution, the count is given in the sentence that makes the claim.

6.1 Motivation: Verifying Before Emitting, Not After

Section 1.3 argued that agentic KGQA systems are typically evaluated on whether the correct answer is *among* what they assert, leaving the precision of the assertion unmeasured and unconstrained. The navigation loop makes that concrete: its final act is a language-model call over a fact window and a candidate list, the model is free to name an entity that appeared in neither, and no downstream component objects.

Two episodes from the first end-to-end development run (Section 5.10) sharpen the point. In one, a retrieval defect hid the `currency_used` relation, the beam surfaced triples about income denominated in dollars, and the answerer confidently asserted “United States Dollar” for two different countries — an assertion no retrieved triple supported. In another, the evaluator listed nine countries that appeared in no retrieved triple at all. The first was caught only by fixing the retrieval defect; the second by the groundedness filter on resolutions (Section 5.6.2), one layer earlier than intended. Neither was caught by anything examining the answer.

The design response is to check the draft against the environment *before* it is emitted rather than to fact-check an emitted answer afterwards. Three properties follow from placing the check inside the loop. The checker sees the same retrieved evidence the drafter saw, so a disagreement between them is informative rather than an artifact of differing context. The check happens while

budget remains, so a rejected claim can trigger further exploration rather than only a retraction. And the check is symbolic where the graph can answer, so the arbiter of whether a fact holds is the environment and not a second opinion from the model that produced the draft.

What the layer promises should be stated precisely, because the terms are easy to inflate. It certifies that the atomic claims a draft asserts are *supported* by the environment in the sense fixed in Section 2.3.3: the traversed triples, or the graph in the retrieved neighbourhood, exhibit a connection consistent with the claim. It does not certify that the answer is correct. An answer every one of whose claims is supported can still be the wrong answer to the question, and Section 6.8 shows that this is a structural gap rather than a hypothetical one.

6.2 Claim Decomposition of the Draft Answer

Verification begins with a single model call that produces three things at once: a prose draft, the list of entities that draft asserts as the answer, and a decomposition of the draft into atomic claims.

The call sees a fact window of at most sixty traversed triples — ranked by embedding similarity between the triple’s verbalisation and the *question*, with traversal order preserved among the survivors — together with up to twenty-five accumulated candidate entities. Each claim is emitted as a triple of strings $\langle h, r, t \rangle$ over entity *names*, not identifiers, since the model has never seen an identifier.

Four provisions in the prompt are load-bearing, and each exists because its absence produced a specific defect.

Names are to be copied exactly as they appear in the facts. The graph’s surface forms are frequently not the ones a language model would generate unprompted — Freebase renders Yale University as University Yale — and every downstream check is keyed on the name string.

answer_entities names what the draft asserts as the answer, not every entity it mentions. Without this the list drifts toward a bag of mentioned names, including the question’s own subject.

A hedged draft has zero claims. This makes abstention explicit rather than something inferred from the absence of an answer.

Answer entities must be specific entities of the kind the question asks for, never a restatement of the question or an entity type. This provision was added after development runs produced answers of the form “the senators from New Jersey are the United States Senate members associated with New Jersey” — grammatically an answer, semantically empty, and, as Section 6.8 explains, entirely invisible to a claim-level check. It is a drafting-side mitigation of a failure the checking side cannot address.

The relation slot r is deliberately free text. The model is not asked to name a Freebase predicate,

because it has no reliable way to do so and a wrong guess would be indistinguishable from a wrong claim. The consequence is that the structural checks below cannot match on the relation at all; the cost of that choice is analysed in Section 6.3.1 and again in Section 6.8.

Decomposition on the development set was sparse. Across 80 questions the drafter emitted 121 claims — a mean of 1.51 and a median of 1 per question, with a maximum of 5. Ten questions produced no claims at all. For those ten the layer certified nothing, and Section 6.8 returns to what that means for the outcome label they received.

6.3 Two-Stage Claim Checking

Each claim is routed through at most three tests: two structural, one model-based. Algorithm 2 states the procedure; the subsections that follow explain each test and the reason for their order.

6.3.1 The Structural Check

The structural check operates on entity identifiers, so it first needs a mapping from the names the model produced back to graph nodes. That mapping, μ , is built from the traversed triples alone: every head and tail name encountered during navigation is registered against its identifier, first occurrence winning. Its scope is worth stating explicitly, because it bounds everything that follows. **A claim naming an entity the agent never traversed cannot reach either structural route**, regardless of whether that entity exists in the graph. Such claims fall directly to the entailment check.

For claims whose endpoints both resolve, the first route is *traversed adjacency*. The traversed triples are collapsed into an undirected set of endpoint pairs, and a claim is supported if its endpoint pair is in that set. This route is exact, costs nothing beyond a set lookup, and — uniquely among the three — produces citable evidence: every traversed triple joining the pair is added to the supporting-triple set.

The second route is the `verify_connection` tool (Section 5.2.3), a single Cypher query asking whether the two entities are adjacent in the *full* graph, either directly or through one mediator node. It exists because the first route tests the *traversal*, not the graph: the agent may have reached both endpoints along different branches without ever walking the edge between them.

Both routes are relation-agnostic. Since r is free text with no correspondence to a Freebase predicate, matching on it would reject true claims wholesale — “X played for Y” is realised through a roster mediator carrying no predicate named for playing. The design intent is a division of labour: the structural routes provide high recall for “some supporting edge exists,” and semantic precision is delegated to the entailment check. Section 6.8 examines what that division actually buys and what it lets through.

Algorithm 2 Structural verification of a draft

Require: question q , traversed triples T , candidates C , tools, budget
Ensure: draft, answer entities, supported and unsupported claim sets, supporting triples

- 1: $W \leftarrow \text{TOPFACTS}(q, T, 60)$; $\mu \leftarrow \{ \text{name} \mapsto \text{id} \}$ over T {first occurrence wins}
- 2: $(\text{draft}, A, K) \leftarrow \text{DRAFTANDDECOMPOSE}(q, W, C)$ {one call}
- 3: **if** $\neg \text{verify_claims}$ **then**
- 4: **return** $(\text{draft}, A, \emptyset, \emptyset, \emptyset)$ {ablation condition}
- 5: **end if**
- 6: $\text{adj} \leftarrow \{(h, t), (t, h) : (h, r, t) \in T\}$; $S \leftarrow \emptyset$; $P \leftarrow \emptyset$; $E \leftarrow \emptyset$
- 7: **for all** $c = \langle h, r, t \rangle \in K$ **do**
- 8: **if** $h \notin \mu$ **or** $t \notin \mu$ **then**
- 9: $P \leftarrow P \cup \{c\}$ {endpoint never traversed}
- 10: **else if** $(\mu[h], \mu[t]) \in \text{adj}$ **then**
- 11: $S \leftarrow S \cup \{c\}$; $E \leftarrow E \cup \{ \theta \in T : \{ \theta_h, \theta_t \} = \{ \mu[h], \mu[t] \} \}$
- 12: **else if** $\text{VERIFYCONNECTION}(\mu[h], \mu[t])$ **then**
- 13: $S \leftarrow S \cup \{c\}$ {no evidence recorded}
- 14: **else**
- 15: $P \leftarrow P \cup \{c\}$
- 16: **end if**
- 17: **end for**
- 18: $U \leftarrow \emptyset$
- 19: **if** $P \neq \emptyset$ **then**
- 20: $V \leftarrow \text{ENTAIL}(W, P)$ {one batched call; on any failure $V \equiv \text{false}$ }
- 21: partition P into S and U by V
- 22: **end if**
- 23: **if** $U \neq \emptyset$ **and** verify budget remains **then**
- 24: $\text{iters} \leftarrow \text{iters} + 1$; append repair objective for the first $c \in U$; re-anchor on $\mu[c_h]$
- 25: **end if**
- 26: **route:** $U = \emptyset \Rightarrow$ grounded; else verify, depth, and time budget all remain $\Rightarrow$ retry (**goto** explorer); else give_up
- 27: **return** $(\text{draft}, A, S, U, E)$

On the development set the second route was consulted five times, across five distinct questions — four per cent of the 121 claims. All five returned connected, and all five had a mediator path; in three of them there was no direct edge at all, so a mediator-blind adjacency test would have rejected them. Two were direct edges the traversal had simply never walked. Query latency ranged from 1.9 to 75.7 ms. The route is therefore cheap and, on this evidence, correct — but it is also rare, and the structural check is in practice overwhelmingly the traversed-adjacency test.

6.3.2 The Entailment Check

Claims that fail both structural routes — together with those whose endpoints were never traversed — accumulate into a pending set and are checked in one batched model call. The claims are numbered, rendered as $h - r - t$, and presented against a fact block, with the model

asked for a boolean verdict per claim. The prompt defines unsupported explicitly, to include the three cases that would otherwise be resolved charitably: plausible but absent, contradicted, and inferable only through outside knowledge.

Three properties of this check must be stated, because each bounds what its verdicts mean.

The fact block is the same window the drafter saw. It is the same sixty triples, ranked by relevance to the *question* rather than to the individual claim under test. A claim about an entity peripheral to the question may therefore be judged against facts that never mention it. The bias this introduces is directional: it disfavors peripheral claims and favours claims about material the drafter had already been shown.

Failure is conservative and lossy. If the call raises for any reason — budget exhaustion, transport error, malformed output — every pending claim is marked unsupported. This is the right default for a system whose purpose is to avoid unsupported assertions, but it means an infrastructure failure and a genuine rejection produce the same record except in the trace.

The check can only add support, never remove it. Claims accepted structurally never reach it. A claim whose endpoints are adjacent for a reason unrelated to the asserted relation is accepted without any semantic examination.

6.3.3 Why the Structural Check Runs First

Three reasons, in descending order of importance. *Authority*: the thesis’s position, fixed in Section 4.5.3, is that embeddings and language models rank while symbols decide, so the model must be consulted only where the graph is silent — it fills gaps and never overrides a finding of the environment. *Evidence*: only traversed adjacency yields triples that can be attached to the answer, so the order does not change which claims are accepted but does change how many can be justified. *Cost*, least importantly: a set lookup is free, a `verify_connection` probe is one indexed query, and the entailment call is the layer’s only mandatory expense beyond drafting, so ordering cheap-to-expensive runs it on a residue rather than on every claim.

6.4 Repair Policies for Unsupported Claims

The router following the verifier is pure Python and has three outcomes: no unsupported claims routes to grounded; unsupported claims with budget remaining route to retry; otherwise `give_up`.

6.4.1 Targeted Re-Exploration Under Remaining Budget

On retry, the verifier has already re-targeted the agent before the router runs. It takes the *first* unsupported claim, synthesises a repair objective of the form `verify whether  $h \ r \ t$` , marks every

active plan step done, appends the repair step as the new active objective, and — if the claim’s head name resolves — resets the frontier to that single entity. Control returns to the explorer, which now searches for the missing evidence specifically.

Two limitations follow. Only one claim is repaired per iteration, and the one chosen is first by position in the draft’s decomposition rather than by importance to the answer. Both keep the repair path cheap, and neither was revisited because the path was rarely exercised.

The retry route requires verify, depth, *and* time budget. The depth condition was added after a development trace showed a question completing with depth 5 against a maximum of 4: the retry edge re-enters the explorer without passing through the post-evaluation router, which is where the depth check lives, so each retry silently granted one expansion beyond budget. Since Section 5.8 claims budgets are hard guarantees, the hole was closed in the router rather than documented as an exception.

That condition turns out to be decisive, and the honest statement of what this route contributed is that **it never executed**. On all three development questions that produced an unsupported claim, the depth budget was already exhausted when the verifier ran, so the router took `give_up` in every case and no repair exploration occurred. This is not a coincidence of small numbers so much as a structural correlation — the verifier is reached either because the plan completed or because a budget ran out, and all three firings came from the fourteen questions that exhausted depth, none from the sixty-six that finished with search budget in hand. With $n = 3$ that is an observation and not a finding, but it means the targeted re-exploration described above is specified and unit-tested rather than empirically characterised, and no claim in this thesis depends on it.

One instrumentation consequence follows, and matters when reading run records. The verifier increments `verify_iters` and appends the repair objective *before* the router decides, so a record showing `verify_iters: 1` records the verifier’s intent to repair, not an executed repair. Whether exploration actually resumed is visible only in the node sequence of the trace.

6.4.2 Answer Rewriting and Hedging

On `give_up` the answerer makes one rewrite call. It receives the draft, the unsupported claims, and — crucially — the list of entities already decided to survive, with an instruction to keep them named exactly as written. It is asked to remove or hedge the unsupported material and to respond with prose only. That last phrase is the whole of the fix described in Section 6.5: in the original design this call also regenerated `answer_entities`, which turned out to be the single largest defect in the layer.

6.4.3 Iteration Cap and the Draft-Only Fallback

Repair is capped at two iterations, which together with the depth and time conditions guarantees the verify–explore cycle terminates. On the development set the cap was never reached, for the reason just given: the cycle never began.

The grounded path costs nothing. When no claim is unsupported the answerer emits the draft verbatim with no further model call — 77 of 80 development questions took this path. The layer’s aggregate cost is correspondingly small: mean tokens per question rise from 4,858 without claim checking to 4,922 with it, about 64 tokens or 1.3%, and mean model calls from 7.2 to 7.3. Whatever the case for or against the layer, it is not a cost argument.

A distinct path exists when `verify_claims` is false: the verifier returns the draft and its entity list untouched with outcome `draft_only`. This is the ablation condition of Section 7.8, not a runtime fallback — the system never selects it dynamically. A third path, the legacy answerer of Section 5.7, runs only when the verifier raised before producing a draft at all.

6.5 Entity Filtering of the Final Answer

The rule is one line and its justification is the most instructive result in this chapter.

The rule. On the give-up path the final entity list is computed deterministically from the draft’s own list: an entity is kept if it appears in at least one supported claim, or if it appears in no unsupported claim. Strings are copied verbatim. The function can only remove entities, never add one. The disjunction’s second branch is deliberately permissive — an entity the decomposer never mentioned in any claim is retained, on the principle that the layer should punish unsupported assertions and not imperfect decomposition recall.

The scope. This filter applies to the give-up path only. On the grounded path the draft’s entity list passes through unchanged, because there are no unsupported claims to filter against. On the development set the filter therefore ran on 3 of 80 questions.

The Attribution Census

The filter exists because of a paired comparison between the verified condition and its `verify_claims = false` twin at the frozen α . In aggregate, claim checking *cost* accuracy: 64 hits against draft-only’s 66. Aggregate counts cannot distinguish a mechanism working as intended — turning an unsupported lucky hit into an honest hedge — from a mechanism misfiring, so the two runs were diffed per question. Exactly three questions differed. Table 6.1 gives all three.

The classification is unambiguous. *No entailment verdict was wrong*, and no repair exploration was involved — as Section 6.4 records, none ran. All three differences trace to one component:

Table 6.1: The complete attribution census: every development-set question whose answer differed between claim checking and its ablation, at $\alpha = 0.7$, $\tau = 0.20$. Three questions, one run per condition. “Pre-fix” is the original design in which the rewrite call regenerated the entity list; “post-fix” is deterministic filtering.

Question	Draft-only	Verified, pre-fix	Verified, post-fix
Harbaugh’s teams founded before 1996 <i>gold</i> : 4 teams	4 gold teams <i>plus</i> Baltimore Ravens; scores as a hit	[] — hedge	[] — hedge
Party of Hitler and Carl Voss <i>gold</i> : Nazi Party	Nazi Party — hit	Adolf Hitler, Nazi Party; hit, but asserts the question’s subject	Nazi Party — hit
Franco’s university, founded 1701 <i>gold</i> : University Yale	University Yale — hit	Yale University — wrong	University Yale — hit
Whole run — 80 questions, of which 3 are unanswerable and abstained on in every condition			
hits / hedges / wrong	66 / 7 / 4	64 / 8 / 5	65 / 8 / 4

the rewrite call, failing three different ways. On Harbaugh it over-deleted, discarding four grounded-correct team names along with the unsupported one. On Hitler/Voss it laundered the question’s subject into the answer list — an entity the draft never asserted. On Franco it silently respelled the graph’s University Yale as the parametric Yale University, converting a defensible qualified answer into a scored error.

The finding is worth stating in one sentence, because it is the conclusion this chapter’s design rests on:

The verification logic was sound; its repair path free-generates entities from a language model, which reintroduces exactly the ungroundedness the layer exists to prevent.

Replacing generation with deterministic filtering removed two of the three regressions, exactly as predicted: Hitler/Voss and Franco left the diff.

What the Census Does and Does Not Establish

Three qualifications belong here rather than in a limitations chapter, because without them the numbers above would be read as more than they are.

The census is $n = 3$. It is a complete enumeration of the questions on which the two conditions differed, which is exactly why it could be classified by inspection — but three questions support a mechanism claim, not a rate.

Claim checking did not reduce wrong answers on this set. Post-fix verified scores 65/8/4 against draft-only’s 66/7/4. The wrong count is identical; the layer cost one correct answer

and removed none. Its case therefore cannot be made on development-set accuracy, and this thesis does not attempt to make it there. The case rests on the groundedness measurements of Section 8.5, which are the metric the layer was built to move, and on the precision effects that Hits@1 is structurally unable to see.

The residual difference is a single question, and its interpretation involves a label changed after the fact. The sole remaining diff is Harbaugh, where draft-only’s “hit” asserts Baltimore Ravens — supported by no traversed triple, and not a team Jim Harbaugh played for — alongside an unverifiable founding-date filter. Set-intersection Hits@1 scores that as a clean hit; per-question F1 in Chapter 8 does not. The question was additionally relabelled from `cvt_heavy` to `unanswerable_env` after this analysis, on the grounds that it carries a date constraint the environment cannot express (Section 4.7.4). The relabelling is defensible on its own terms and was recorded as a decision, but it was made after seeing the result it affects, and is flagged as such here and in Section 9.9.3. The aggregate figures reported above are not rescored on account of it.

6.6 Output Contract: Answer Paired with Supporting Triples

Every run record pairs the answer with the triples that support it. This is the explainability deliverable named in Section 1.6, and its actual coverage is narrower than the phrase suggests.

Only one of the three routes records evidence. Supporting triples are collected in the traversed-adjacency branch alone. A claim certified by `verify_connection` is accepted with nothing attached, and so is a claim certified by entailment. The output contract is therefore “answer paired with whatever traversed triples happened to justify it”, not “answer paired with justification for every claim”.

The measured consequence on the development set: 13 of 80 answers carry no supporting triples at all. Ten of those asserted no claims — they are hedges, and zero is the honest number. Of the remaining three, one had every claim rejected, so there was nothing to record; the other two had their single claim certified by `verify_connection`, which records no evidence. Across all 80 records the median count is 3 and the mean 4.1, with a maximum of 16.

Two instrumentation defects in this area are recorded rather than quietly fixed, in keeping with the standing rule adopted in Section 5.10.

The first was a Python falsy-empty-list trap. The answerer substituted the entire traversed set for the supporting triples whenever the verifier’s list was empty — treating “the verifier ran and matched nothing” identically to “the verifier never ran”. A development trace consequently recorded 1,927 supporting triples for an answer that asserted nothing whatsoever. The fix distinguishes the two states by testing for `None` rather than for falsity: absent means the legacy path, empty means zero support, and zero is reported.

The second is unrepaired and therefore fenced off. A counter in the verifier’s trace named `n.structural` counts every supported claim, including those certified by the entailment call, so its name misdescribes it. Under the standing rule — a metric that has never been checked against a case whose correct value is known independently should not be analysed — it is not used anywhere in this thesis, and no table or figure derives from it. The route-by-route counts reported in Section 6.3.1 come from the tool invocation log, which records each `verify.connection` call with its arguments and result.

6.7 A Worked Example

Which university, founded in 1701, did actor James Franco attend? Gold: University Yale.

Navigation resolves James Franco as the anchor and expands along `education.education.student` and `people.person.education`. The plan’s first objective — find the university founded in 1701 — is never satisfied, because the environment holds essentially no date literals (Section 4.7.4). Three evaluator-triggered backtracks follow, and the run terminates on the depth budget with 18 model calls, 17 of them cache replays.

The verifier drafts: “*James Franco attended University Yale, which was founded in 1701.*” Its entity list is [University Yale] — copied from the fact window, in the graph’s spelling. It decomposes the draft into an attendance claim and a founding claim.

The attendance claim resolves both endpoints in μ and its endpoint pair is in the traversed adjacency set. It is supported, and four traversed triples joining the pair are recorded as evidence. The founding claim names 1701, which no traversed triple mentions, so it cannot resolve and falls to the entailment check — which correctly reports it unsupported, because the fact is genuinely absent from the environment rather than merely unretrieved. The verifier prepares a repair objective anchored on University Yale, but the depth budget is already spent, so the router takes `give_up` and the exploration never resumes; there was in any case nothing to find.

The answerer now filters. University Yale appears in a supported claim, so it is kept, verbatim. The rewrite call produces prose that asserts the attendance and hedges the founding date, and its own opinion about entities is discarded. Final answer entities: [University Yale] — a hit.

The instructive part is the counterfactual. Before deterministic filtering, this identical trace — same draft, same claims, same verdicts — ended with [Yale University]. The rewrite call, asked to restate the answer, restated it in the model’s own spelling rather than the graph’s, and the answer was scored wrong. Nothing about the verification was different. The claim-level machinery had worked perfectly and the layer still emitted an entity string that appears nowhere in the environment, which is the precise thing it exists to prevent.

6.8 Failure Modes by Construction: Wrongful Rejection and Wrongful Acceptance

This section is analytical. It enumerates the ways the layer can be wrong that follow from its design, independently of how often they occur. Which of these mechanisms actually fired, and how often, is reported in Section 9.5; the vocabulary for the two polarities is fixed in Section 9.5.1 and used here. A claim is *wrongly rejected* when the layer withholds support the environment would in fact provide, and *wrongly accepted* when it certifies a claim the environment does not actually support.

Mechanisms of Wrongful Rejection

Composite claims. The decomposer emits claims corresponding to what the draft asserts, not to every relationship the draft relies on. When a draft asserts a filtered set — “these teams were founded before 1996” — the decomposition is about the founding, and the membership relation that makes each team a candidate may never be claimed at all. Every team’s only claim is then the unverifiable one, the filter drops every entity, and a draft containing four correct answers becomes a hedge. This is the Harbaugh question of Table 6.1, and the mechanism is worth naming: *a composite assertion whose unverifiable component drags down its verifiable component*. The conservative rule is defensible for a hallucination-mitigation system, and it is also the layer’s most expensive single behaviour on the development set.

Endpoints outside the traversal. μ is built from traversed triples only, so a claim about an entity the agent never walked to skips both structural routes regardless of what the graph contains. It is judged solely by a model, against a fact window that by construction does not mention it.

A question-ranked window for a claim-level judgement. The entailment check’s evidence is ranked for relevance to the question, not to the claim under test. Peripheral claims are systematically disadvantaged.

Conservative failure. Any exception in the entailment call marks all pending claims unsupported. Infrastructure failure is recorded as rejection.

Name collisions in μ . The mapping keeps the first identifier seen for a name. Two distinct graph entities sharing a surface form collapse to one, and adjacency is then tested for the wrong node.

Mechanisms of Wrongful Acceptance

Relation-agnostic adjacency. Both structural routes ignore r and ignore direction. Any edge between the endpoints certifies any claimed relation between them. A claim that X is Y’s mother is accepted on an edge recording that X is Y’s child; a claim that a film was directed by a person

is accepted on an acting credit. This is the direct cost of allowing free-text relations, and it is not a small one — it is the layer’s principal acceptance risk.

The mediator hop. `verify_connection` accepts a path through one mediator node. Because a Freebase mediator encodes a single n -ary fact, the endpoints of such a path are genuinely co-participants in one fact — but which role each plays is exactly what the mediator encodes and the test discards. An actor, a film, a character, and a period are mutually “connected” through one performance node, and any claim relating any two of them is certified.

Vacuous certification of empty decompositions. A draft with no claims has no unsupported claims, so it is routed grounded. On the development set this describes 10 of 80 questions. The label is not false — an answer that asserts nothing asserts nothing unsupported — but it must not be read as a positive verification, and any statistic computed over grounded outcomes inherits this.

Entities that appear in no claim are never checked. This is the broadest of the mechanisms listed here and deserves to be stated without softening. Verification operates on claims; the answer’s entity list is a separate output of the same call. An entity the drafter named but the decomposer never wrote into a claim passes through untouched on the grounded path, and is deliberately retained by the permissive branch of the filter on the repair path (Section 6.5). Since 77 of 80 development questions took the grounded path, the layer’s guarantee over emitted entities is in practice mediated entirely by decomposition recall — and decomposition recall is itself a language-model output, subject to no check.

Grounded-but-vacuous answers. This is the boundary of what triple-level verification can do, and it deserves to be stated as such. An answer whose every claim is supported can still fail to answer the question: “the senators from New Jersey are the United States Senate members associated with New Jersey” decomposes into claims the graph genuinely supports, and is certified. No claim-level check can catch this, because the defect is not in any claim — it is in the relationship between the claims and the question. The anti-vacuity provision of Section 6.2 addresses it at drafting, which is mitigation, not verification.

Structural acceptance is final. A claim accepted by adjacency is never examined semantically. The entailment check, which is the only component that looks at meaning, sees only what the structural routes could not resolve.

What the Layer Is

Stated at the level of precision the preceding list requires: the layer establishes that the entity pairs a draft’s claims relate are connected in the environment, and on the repair path it makes the emitted entity list a deterministic function of those verdicts rather than a second generation. It does not establish that the claimed relation is the one the edge records, that an entity absent from every claim was grounded at all, that the answer is responsive to the question, or that a claim

it could not check is false. Those boundaries are properties of the design rather than defects discovered in it, and Chapter [9](#) reports how much of the residual error they account for.

Chapter 7

Experimental Setup

This chapter fixes everything that had to be decided before any test question was answered: which questions, which systems, which numbers, and which rules for reading them. It is written to be sufficient for replication, and it records the decisions that went the other way as well as the ones that stood — a backbone choice that was made, reversed, and then remade under a pre-registered rule; a test-set build that read the wrong split and was caught before harm; a validation threshold that was missed by half a thousandth. Each of those is reported where it belongs rather than collected into a confessional at the end.

Every figure quoted in this chapter and the two that follow is regenerated by `scripts/build_thesis_numbers.py` into a single file, `results/phase4/thesis_numbers.json`, from the scoring, groundedness, judge, and census artifacts. No number in the thesis is transcribed by hand from a console log.

7.1 Mapping Experiments to Research Questions

Each of the three research questions of Section 1.5 is answered by a distinct experiment rather than by one table read three ways. **RQ1** — whether agentic navigation improves multi-hop accuracy — is answered by the main matrix of five systems on two datasets, scored with Hits@1 and F1, broken down by hop stratum, with paired significance tests (Sections 8.2 to 8.4); the per-stratum breakdown, not the aggregate, is what makes the claim about *multi-hop* accuracy. **RQ2** — what pre-generation verification contributes — is answered by three measurements that do not agree with one another, which is itself the finding: the two-tier groundedness metric (Section 8.5), the precision and F1 columns of the main matrix, and the verifier ablation (Section 8.7). It was posed as *what does it contribute* rather than *does it reduce hallucination* for the reason Section 8.8 gives. **RQ3** — component attribution at a token price — is answered by four single-deviation ablations on stratified half-samples, each compared against the unmodified system restricted to the same questions (Sections 7.8 and 8.7).

7.1.1 Pre-Registration and the No-Tuning Policy

One rule governed everything from the moment the test files were built: *nothing gets tuned*. No prompt was edited, no configuration value changed, and no threshold adjusted after test data was touched. Defects discovered mid-matrix were documented rather than fixed, and any fix that did become unavoidable would have been applied identically to every system with all affected files re-run — a situation that did not arise.

Four decisions were fixed in writing before the measurements they govern:

- The frozen configuration $\alpha = 0.7$, $\tau = 0.20$, claim verification enabled, tuned on the development set (Section 7.9) and never revisited.
- The sample size: stratified samples of 400 questions per dataset, named as the fallback before the API budget was known and adopted once it was (Section 7.2).
- The baseline certification threshold: a retrieval baseline is certified if it hedges while the gold answer is visible in its own retrieved facts on fewer than 15% of its hedges (Section 7.4).
- The judge validation threshold: Cohen’s $\kappa \geq 0.7$ against blind human annotation before the semantic-support numbers may be cited as validated (Section 7.6).

The last of these was missed, by 0.0005. It is reported as missed in Section 7.6, because a threshold that is only pre-registered when it is met is not pre-registered at all.

One further discipline, cheaper than it sounds and load-bearing for Section 8.7: the single code change Phase 4 required — a `use_planner` flag to disable decomposition — was certified as a no-op on the frozen path by re-running an already-answered development question and confirming that every language-model call was a cache hit. Byte-identical prompts are byte-identical behaviour, so the reference condition in the ablation table is provably the same system that produced the main matrix.

7.1.2 Metrics Proposed but Not Run

The approved proposal named three evaluation criteria: answer accuracy, groundedness, and *path fidelity* — agreement between the relation chain the system traversed and the chain the benchmark’s gold SPARQL query specifies. Path fidelity is not reported. Computing it requires the gold relation chains, and the RoG distribution used to build the knowledge environment (Section 4.2) publishes name-keyed triples without the originating SPARQL, so the chains would have to be reconstructed from the upstream releases and re-aligned to this environment’s relation vocabulary. That was outside the project’s remaining budget. Section 10.3.7 treats it as future work; the point is not relitigated here.

7.2 Test Sets

7.2.1 WebQSP and ComplexWebQuestions

Two benchmarks are used, in the RoG distribution [9] whose per-question subgraphs also form the knowledge environment (Section 4.2). **WebQSP** [19] is the semantic parses of WebQuestions: natural questions from search logs, mostly one or two hops, with multi-valued answers common. **ComplexWebQuestions** (CWQ) [20] is built by composing WebQSP questions with additional constraints through SPARQL templates, giving deeper chains, conjunctions, superlatives, and comparisons — and, as Section 7.7 documents, a higher rate of machine-generated label defects. The distinction that matters operationally is visible in the gold answer sets: WebQSP averages 7.98 gold entities per question (median 2), CWQ averages 1.78 (median 1). Any metric that rewards naming one correct entity therefore behaves very differently on the two benchmarks, which is one of the reasons F1 is reported alongside Hits@1 (Section 7.5).

7.2.2 Stratified Sampling, Seeds, and Published Question IDs

Only the *test* splits are used. The development set of Section 7.9 was mined from train and validation, and the builder asserts zero overlap between the two pools.

Sampling was a budget decision. The full test splits hold 1,628 and 3,531 questions; five systems over both, at the measured token rates, projected to roughly \$45–55, against a stated ceiling of \$15–20. The pre-registered fallback — stratified samples of 400 per dataset, giving bootstrap confidence intervals of roughly ± 5 points on Hits@1 — was therefore adopted, with the arithmetic recorded before the runs fired. The whole benchmark, including the ablations, ultimately cost \$11.13 in API spend.

Sampling is proportional within hop strata at seed 42, and the resulting question sets are committed as immutable artifacts (`results/phase4/test_webqsp.json`, `results/phase4/test_cwq.json`); every question identifier is listed in Chapter C. They were never regenerated after any system ran.

A near miss worth recording. The first build read `train-*.parquet` rather than `test-*.parquet` — a glob copied from the development-set miner, which legitimately reads train. Pool sizes matching the train splits and eighteen resolved shards were suggestive; the conclusive signal was the development-overlap guard reporting 32 and 45 overlapping questions where a genuine test pool must report zero. The contamination guard, built for hygiene, is what caught the contamination. No system had run against the files; they were deleted and rebuilt, and the rebuilt pools report 1,628 and 3,531 with `skipped={}` — the same evidence now testifying the other way, and the reason a cheap overlap assertion belongs in every benchmark builder.

Table 7.1: The two certified test samples. The unreachable stratum is retained deliberately; the ceiling is the share of questions with at least one gold answer reachable within four hops, and is the upper bound on Hits@1 for every system evaluated here.

Dataset	n	h1	h2	h3plus	unreach.	Ceiling
WebQSP	400	256	128	4	12	97.0%
CWQ	400	137	211	49	3	99.2%

7.2.3 Hop-Count Stratification and the Accuracy Ceiling

The stratification variable is the length of the shortest path in *this* environment between any question topic entity and any gold answer entity, bounded at four raw hops — the same machinery used to certify the development set, so that mediator nodes are counted consistently as the two edges they are (Section 5.2.3). Questions fall into h1, h2, h3plus, or unreachable.

Table 7.1 carries two facts the results chapter leans on repeatedly. WebQSP is 64% one-hop with a four-question h3plus tail — too small to interpret, and labelled as such wherever it appears. CWQ is majority two-hop with a genuine deep tail of 49. The samples preserve the strata proportions of their parent pools, so per-stratum numbers describe the benchmark rather than the sample.

The unreachable stratum was kept rather than dropped. Dropping it would raise every system’s score by the same amount and misdescribe the environment; keeping it means the ceiling is reported honestly and, as Section 8.5 shows, that these questions become the sharpest available probe of parametric leakage — any “hit” on a question with no graph path to gold is an assertion the environment cannot support.

7.3 Backbone Model Selection and Qualification

All systems share one frozen backbone: gpt-5.4-mini-2026-03-17 at temperature 0.0, with `reasoning_effort` explicitly set to “none” and the response cache enabled. The model string, both sampling parameters, and the cache mode are stamped into every one of the 4,000 test-matrix records and every ablation record, so any record found anywhere states what produced it.

Two properties motivated a non-reasoning model: sampling parameters remain available, so temperature can be pinned; and no hidden reasoning tokens contaminate the cost metric. The second was verified rather than assumed — a defensive check logs the reasoning-token count on every call, and it read zero across all runs in this thesis.

Selection between gpt-4.1-mini-2025-04-14 and gpt-5.4-mini-2026-03-17 was made by a qualification script that runs both candidates over the same twenty development questions and reports tokens, calls, latency, reasoning tokens, errors, and a determinism probe. The probe re-runs a subset of questions and counts how often the *decision signature* — a hash of the plan,

the expanded relations, the evaluator verdicts, and the answer, not of timings or scores — repeats exactly.

The first round reported 3/3 signature matches for 4.1-mini against 1/3 for 5.4-mini, and 4.1-mini was chosen on that basis. A second round with the cache disabled reversed it: 2/3 against 3/3. The reversal, not either number, is the finding.

7.3.1 Trajectory Stability Under Temperature-Zero Decoding

Temperature-zero decoding on a hosted API is greedy, not deterministic. The logits are not bit-stable across calls, because floating-point reduction order depends on how a request is batched with other traffic, and when the top two tokens are near-tied a wobble of order 10^{-6} flips the argmax. A sequential agent then *amplifies* that: one divergent token in a planner call changes a sub-objective’s wording, which changes its embedding, which changes the beam, which changes everything downstream. A model that is 98% stable per call can be 70–90% stable per trajectory. With three probes per round, an estimator of a rate in that band bounces between 1/3 and 3/3 by sampling variance alone. That is what happened. A tiebreaker was therefore pre-registered before rounds three and four — *if the two candidates finish within three matches of each other, take gpt-5.4-mini on longevity grounds, the 4.1 line being deprecation-adjacent* — and both rounds returned 2/3 for each candidate. Pooled over all four rounds the score is 9/12 against 8/12: a tie, the rule fired, and the backbone was frozen.

The reported measure of trajectory stability is therefore $\approx 75\%$, and the thesis makes no claim of determinism. One provenance limit belongs with that number: only the first round survives as per-question artifacts, because the qualification script writes its detail file in truncating mode and later rounds overwrote it; the tool logs accumulated across all four rounds, and the console tables for rounds two through four exist in the project record rather than as committed artifacts. Two observations from those four runs matter more to this thesis than the choice they settled. First, each candidate carried a *persistent* ungrounded answer: 4.1-mini asserted “United States Dollar” for a Dominican-peso question in two of four runs and a wrong European Union president in one; 5.4-mini named the wrong Beckham child in all four and a different wrong president in one. Second, both carried *intermittent* ones — the same question, the same graph, the same budget, with the answer moving between grounded-correct and ungrounded-wrong as the sampling weather changed. Backbone selection does not remove this class of error; it relocates it. That is the premise of this thesis, measured over four runs and two frontier-family models, and it is the reason Chapter 6 exists.

7.3.2 Response Caching as the Reproducibility Backstop

Reproducibility here rests on the cache, not on the model. Every completion is stored keyed on model string, temperature, reasoning effort, prompt, and schema hint, and — the detail that makes it work — the original token usage is stored alongside and *replayed into the budget meter* on a hit. The budget check still runs and the call counter still increments, so a fully cached rerun reproduces the original run’s budget snapshot exactly rather than approximately. Every table in Chapter 8 comes from one recorded run per condition, and that run replays byte-identically for as long as the cache survives, independent of API weather or model retirement.

Fresh-run variance is a different problem and is handled the standard way: bootstrap confidence intervals over questions and paired significance testing, both specified in Section 7.5.

The cache leaves one artifact in the record. The WebQSP run of the full system was started, interrupted after seventeen questions, and restarted; the restart replayed those seventeen from cache. Answers, entities, tokens, calls, and traces are unaffected — that is what byte-exact replay means — but their wall-clock readings measure a disk, not a system. The original cold log was recovered and merged back by a script that asserts every field except the timing and cache counters is identical, so the archived file carries true cold latencies throughout; no record in it is a full cache replay. Latency is in any case computed over cold-cache records only, which protects the column regardless.

7.4 Baseline Systems

Four baselines span the space between no retrieval and agentic navigation. All were implemented against the same tools, the same graph, and the same backbone.

7.4.1 No-Retrieval LLM (Parametric-Memory Control)

One call, the question, no facts, with an instruction to return an empty entity list if the answer is unknown. WebQSP and CWQ predate current models and are plausibly memorised, so this system measures the floor above which any retrieval result must be read. Its entity strings come from parametric memory rather than the graph, so surface forms mismatch gold more often than for graph-grounded systems; its score is consequently a *lower bound* on parametric knowledge. That asymmetry is reported rather than corrected, because grading one system leniently and another strictly would corrupt the comparison.

7.4.2 Vector-RAG over Verbalised Triples

Every triple is verbalised into a sentence, embedded with MiniLM [29], and indexed; the question retrieves the top 30 by exact inner product and one call answers from them — dense retrieval [28] applied to a graph. Its limitation is structural rather than incidental: a mediator-reified fact, a marriage or a film role, has no single triple expressing it, so no single retrieved sentence can express it either. Triples with a mediator endpoint are filtered at build time, since a verbalised half-edge is meaningless and leaks raw machine identifiers into answers; filtering the noise does not repair the paradigm, and both facts are reported.

7.4.3 Static GraphRAG

The structure-without-navigation comparison [4]: resolve the question’s topic entities, expand a two-hop neighbourhood under the same relation blocklist and the same mediator pass-through contract every other graph-touching system uses, rerank by embedding similarity to the question, and answer from the top 30 facts in one call. It shares Vector-RAG’s answering prompt and cutoff exactly, so the difference between the two isolates retrieval strategy and nothing else.

Two implementation details fixed on development data are worth naming, because both were comparability repairs rather than improvements: neighbourhood expansion initially passed *through* ordinary entities and stopped at mediator nodes, exactly inverted for Freebase and the reason machine identifiers were reaching answers; and retrieved facts are deduplicated by endpoint pair before reranking, because cosine similarity favours redundant phrasings of one connection and was crowding the informative row out of the top 30. That dedupe has a cost, stated rather than hidden: two distinct facts sharing both endpoints collapse to one, observed but rare.

7.4.4 Think-on-Graph (Agentic Baseline)

The credibility-critical comparison. Think-on-Graph [7] is reimplemented on this environment’s tools: beam width 3, depth 3, language-model relation pruning, language-model entity pruning, and a sufficiency check at each depth. It is a reimplementation, not the authors’ code, and it runs on this graph with this backbone — which makes it a *stronger* comparison than citing published numbers, because the graph, the model, the budget, and the scoring rule are all held constant and only the algorithm differs. Its numbers are therefore not comparable to the published ones and are not presented as such.

One shared-budget consequence is reported as a finding rather than a defect. The cost of beam search is multiplicative in width and depth, so under the same 25-call cap the full system operates under, Think-on-Graph is clipped on 117/400 WebQSP questions (29.2%) and 176/400 CWQ

questions (44.0%). A clipped run still answers, from whatever it gathered, by the same graceful degradation the full system uses — symmetry that fairness requires.

7.4.5 Variables Held Constant Across All Systems

Every system shares the backbone and its sampling parameters, the knowledge environment and its relation blocklist, the entity resolver, the question files, the budget meter and its 25-call cap, the output schema so that scoring reads one field for all systems, and the record schema so that any record self-describes with backbone, budget hash, run configuration, and source commit. Both graph-navigating systems seed from the dataset’s annotated topic entities, which removes entity linking as a confound and is stated as a limitation in Section 9.9.3. One asymmetry is deliberate: the baselines get a plain answering prompt while the full system’s drafting prompt carries the anti-vacuity and grounding provisions of Section 6.2, which are part of the system under test rather than of the shared harness.

7.4.6 Baseline Certification Protocol

Baselines were debugged exclusively on development data and certified before any test question ran. The pre-registered diagnostic targets the one failure mode that would be an implementation defect rather than a paradigm limit: *retrieval found the gold answer and the system hedged anyway*, checkable because both retrieval baselines log a sample of what they retrieved. The rule, written before the numbers: under 15% of hedges certifies the baseline as-is; over 30% means an over-abstinent prompt, to be adjusted identically for both baselines with both files re-run.

Measured: Vector-RAG 0% on both datasets, Static GraphRAG 10% and 14%. Both certified, with two caveats recorded rather than smoothed over. The diagnostic samples the top ten of thirty retrieved facts, so it is a lower bound; and Think-on-Graph’s 0% is *vacuous* rather than clean, since it logs no retrieved-fact sample and the diagnostic therefore compared against an empty string. As a multi-step navigator it was not the diagnostic’s target, but the reading is a null, not a pass. All ten test files completed at exactly 400 records with zero failures and zero duplicate question identifiers.

7.5 Metrics

7.5.1 Answer Accuracy: Hits@1 and F1

Both metrics are computed over *entity sets* after Unicode NFKC normalisation, case folding, and whitespace stripping — no fuzzy matching, no substring containment. *Hits@1* is 1 if any predicted entity matches any gold entity, which is the convention in this literature and is what

makes published numbers comparable at all. *F1* is computed per question between the predicted and gold sets as defined in Section 2.2, and averaged over questions.

Reporting both is not redundancy. Hits@1 rewards a system that names one correct entity among several wrong ones; on WebQSP, where the median question has two gold answers and the mean has eight, that loophole is wide. The precision component of F1 is what closes it, and Section 8.2 shows that the gap between the two metrics is exactly where the verification layer’s contribution becomes visible.

Strict matching raises an obvious worry — that “St. Petersburg” versus “Saint Petersburg” is manufacturing failures. It was measured rather than assumed. Of the full system’s wrong answers, 1 of 65 (2%) on WebQSP and 6 of 99 (6%) on CWQ are surface-form near-misses under aggressive normalisation. Strict matching does not materially understate performance, and the failure census of Chapter 9 is therefore reading real errors.

7.5.2 Accounting for Hedges and Abstentions

A *hedge* is an empty predicted entity set: the system declined to assert. Hedges score zero on both Hits@1 and F1 — an abstention earns nothing on an accuracy metric — and hedge rate is reported as its own column so the abstention–accuracy trade is visible rather than buried. A system can buy precision with abstention, and the reader is given both numbers to see it happen. Hedges are not epistemically uniform, and Chapter 9 keeps them separate from wrong answers throughout for that reason: a wrong answer is a reasoning error, whereas a hedge is usually a coverage gap that never produced a committal claim.

7.5.3 Two-Tier Groundedness and Hallucination Rate

Groundedness is measured at two levels, and the word *tier* in this thesis refers to this metric and nothing else.

Tier 1, structural, is deterministic, free, applies to every asserted entity in all 4,000 records, and operationalises Section 2.3.3 directly: *an asserted entity is graph-grounded if and only if it exists as a node in the knowledge environment and is connected to at least one of the question’s topic entities within four hops*. The entity-level ungrounded rate is the fraction of asserted entities failing this test; the question-level rate is the fraction of answered questions containing at least one such entity. The test is deliberately strict about surface form — a parametric spelling that does not exist as a node is ungrounded *by definition of the environment*, which is the point of defining hallucination relative to an environment at all.

Tier 2, semantic, asks the harder question Tier 1 cannot: is the entity supported *as the answer*? For a stratified sample of 60 asserted answers per system per dataset (600 judgements), the graph’s own shortest connecting paths between topic and asserted entity are retrieved and a

language model judges whether any one of them expresses the relationship the question asks about. The governing instruction is that *mere connectedness is not support*, and that sentence is what separates the two tiers.

Two design choices are pre-registered and consequential. Judging is against **the environment**, not against each system’s own retrieved context: the logs do not preserve per-system evidence uniformly, and environment-judging asks every system the identical question, so the metric applies even to the no-retrieval control. And path retrieval takes the three shortest paths, which means a supporting chain can lose to shorter irrelevant ones. Tier 2 rates are therefore *conservative lower bounds*, uniformly depressed across all systems, and are read as a between-system comparison rather than as absolute support rates.

7.5.4 Efficiency: Tokens, LLM Calls, and Wall-Clock Latency

Tokens and calls are counted by the shared budget meter, identically for every system, and are exact — the cache replays original usage, so they are unaffected by which runs were warm. Wall-clock latency is not: it is computed over **cold-cache records only**, and that restriction is repeated wherever latency appears. Where a condition’s records are entirely cache replays the latency cell is reported as unmeasured rather than as a number, which happens for one ablation condition in Section 8.7.

7.6 The Groundedness Judge and Its Validation

7.6.1 Independence of the Judge from the Answering System

The Tier-2 judge sees a question, one asserted entity, and up to three connecting paths rendered as relation chains with mediator nodes masked. It does not see which system produced the assertion, whether the assertion was scored a hit, the gold answer, or the system’s own reasoning. Its prompt was fixed before any per-system result was inspected. It runs on the same backbone as the systems it judges, which is a limitation stated in Section 9.9.3 rather than a claim of independence from the model family.

7.6.2 Human Annotation Protocol

One hundred judged items were drawn and written to a sheet stripped of the judge’s verdicts, and were labelled by the author *blind* — neither the judge’s output file nor the answer key was opened until the sheet was saved. Blinding is what makes the resulting statistic a validity check rather than a measure of anchoring.

A written annotation guideline was fixed during calibration and is reproduced in full in Section D.1. Each of its eight rules was forced by a case: judge the asked relationship *under the graph's vocabulary*; treat a mediator-bridged pair of edges as one relation; label supported if *any* single path supports; judge relations by *role, not type*, so a normally-generic relation counts when it is itself what the question asks; for conjunctions require every asked relationship covered by some specific path, jointly rather than in one chain; judge each entity independently of answer-set completeness, which is recall and is measured elsewhere; ignore superlative and temporal filters the graph cannot express, judging only the base relation; and treat a path expressing only a question's identifying descriptor, rather than its actual ask, as unsupported.

That last rule is the *composition-echo* case, and it is the same phenomenon the failure census later finds at scale (Section 9.7) — the annotation task and the error taxonomy converging on one mechanism from opposite directions.

7.6.3 Agreement Between Judge and Human Annotator

On $n = 100$ items, observed agreement is 85%, with near-symmetric marginals (54 human-supported against 51 judge-supported) and near-symmetric disagreement (9 human-yes/judge-no against 6 human-no/judge-yes), so neither rater is systematically the more lenient. Cohen's κ [38] is

$$\kappa = \frac{0.8500 - 0.5008}{1 - 0.5008} = 0.6995.$$

The pre-registered threshold was $\kappa \geq 0.7$. Rounded to three decimals this value reads 0.700 and appears to clear; it does not. It falls short by 0.0005, and it is reported as short.

What follows from that is a matter of weight, not of discarding. By the conventional bands [39] a κ of 0.70 is substantial agreement, and the task is genuinely hard — whether a relation chain really expresses an asked relationship has real grey area for a careful human, as the guideline's list of decided edge cases attests. So the Tier-2 numbers are reported, with the κ beside them and the miss stated, and no conclusion in this thesis rests on Tier 2 alone. Where Tier 2 is load-bearing — the between-system semantic-support comparison of Section 8.5 — the finding is also supported by the precision column of the main matrix, which is measured without a judge. The alternative course, tightening the judge prompt and re-judging a fresh sample until the bar was cleared, was available and pre-committed; it was not taken, because the honest number was already in hand and re-running until a pre-registered threshold is met is exactly what pre-registration exists to prevent.

7.7 Gold-Answer Quality Control

7.7.1 The Gold-Noise Problem in WebQSP and CWQ

Both benchmarks carry label defects, and CWQ carries more because its questions and answers are generated by SPARQL templates over the knowledge base rather than written by hand. A question whose gold answer is wrong is not a system failure, and counting it as one corrupts both the accuracy tables and the error taxonomy. Some detection procedure was therefore necessary — but a naive one is worse than none, as the results below show.

7.7.2 Consensus Pre-Pass and Per-Item Adjudication

The detection principle is **independent multi-system consensus against gold**: when at least three of the five systems assert the same non-gold answer, the label is worth examining. Independence is what makes the signal meaningful, so the voters are the five *systems* of the main matrix, spanning two knowledge sources (pretraining and this graph) and four retrieval paradigms. The four ablation conditions would not do: they share the full system’s scorer, drafter, and traversal, so their agreement would describe an architecture rather than a label.

The pass emits one row per (question, consensus answer) pair, so a question with three different non-gold consensus answers contributes three rows. Rows and questions are therefore reported as separate units throughout, and question counts are always the deduplicated ones.

Adjudication assigns each question one of exactly three verdicts — `gold_ok`, `gold_wrong`, `ambiguous_question` — with a free-text subtype for grouping that drives nothing. Automatic triage cleared 15 of 89 rows on WebQSP and 2 of 59 on CWQ; the remainder were adjudicated by hand against the question text and, where a world fact was at issue, an external check.

7.7.3 Two Evidence Signatures of Label Error

The most useful methodological result of this pass is that label errors come in two kinds with *opposite* evidence signatures, and that separating them is what makes the adjudication reproducible rather than ad hoc.

A **world-fact error** — gold contradicts reality — requires *mixed* consensus: the parametric baseline and the graph-based systems agreeing against the label means two different knowledge sources agree. The San Antonio city council question is the worked specimen: four systems converge on Bexar County against a Comal County gold, with mixed evidence, and Section 9.8.2 traces where that label came from.

An **annotation-pipeline error** — gold contradicts the very graph the labels were generated from — has the opposite signature: *graph-only* consensus, where every graph-reading system

Table 7.2: Gold-label adjudication. Rows and distinct questions are separate units because the pass emits one row per consensus answer. Exclusions are the union of `gold_wrong` and `ambiguous_question` questions.

Dataset	Rows	Questions	gold_wrong	ambiguous	Excluded
WebQSP	89	58	12	10	22 (5.5%)
CWQ	59	47	17	2	19 (4.8%)

finds the same right answer while the parametric baseline misses it for unrelated reasons. The specimen is WebQTest-55: “Who had a concert named Crazy Love Tour?” asks for a person, the graph-reading systems all return Michael Bublé, and gold lists his professions — the annotation query having traversed one relation too far.

This distinction was not designed in advance; it was forced. The adjudication scope had initially been narrowed to mixed-evidence flags only, on the reasoning that graph-only agreement is weaker evidence. The Bublé case is graph-only and is a genuine label error, so the scope was widened to cover every flag whose consensus answer was not simply an echo of the question’s own topic. The narrower scope would have missed an entire class.

7.7.4 Census-Based Exclusions and the Dual-Reporting Policy

Two numbers must not be conflated, and Table 7.2 separates them: 58 and 47 questions were *flagged* by the consensus pass, while 22 and 19 were *adjudicated* as defective. The gap is not noise — most flagged questions are all five systems converging on the same wrong answer, which is a finding about the systems rather than the labels and is treated as one in Section 9.7. A policy of rescoring on consensus alone would have corrupted the results with exactly those cases. The excluded questions are removed from the failure census of Chapter 9 — they are benchmark artifacts, not system failures — and `ambiguous_question` is excluded on the same logic, an underdetermined question being no more a system failure than a broken label. Of the excluded questions, 12 of 22 (WebQSP) and 15 of 19 (CWQ) were failures of the full system; those are the ones that shrink the census pool.

The accuracy tables are not rescored. They are reported exactly as measured, with this section as the footnote, for two reasons. Rescoring on self-adjudicated labels would substitute one contestable ground truth for another. And the errors cut both ways: the remaining 10 and 4 excluded questions are ones the full system scored as *hits* against labels now judged broken or ambiguous. That symmetry is what makes the footnote credible rather than defensive, and it means the direction of the net bias is unknown, not conveniently favourable.

Two scope limits belong on the record. The rule that automatically assigned `gold_wrong` to questions with implausibly long gold lists is a heuristic, not a proof, and its outputs were spot-checked rather than individually verified. And the manual adjudication covered every

mixed-evidence flag plus every graph-only flag that was not a topic echo — broad, but not the complete flag population.

7.8 Ablation Conditions

Four conditions, each a single deviation from the frozen configuration, each isolating one mechanism:

- **no-planner** — decomposition disabled; the whole question becomes one objective. Anchor seeding is untouched, so this varies decomposition and nothing else.
- **no-backtrack** — `max_backtracks = 0`; a configuration change, which correctly alters the budget hash stamped on the records.
- **no-verifier** — `verify_claims = false`; grounded drafting with no claim extraction or checking.
- **embedding-only** — $\alpha = 1.0$, removing the language-model term from the hybrid scorer while leaving τ at its frozen value.

The reference row is the *unmodified* system restricted to the same questions. It costs nothing: the main-matrix records are filtered to the half-set question identifiers rather than re-run, which also guarantees the pairing that the significance tests require. Comparing against the full 400-question score would be invalid, since accuracy on 400 questions and on 198 are different numbers for the same system and McNemar’s test needs matched pairs.

Ablations run on stratified halves — 200 WebQSP and 198 CWQ — rather than the full samples. This was the pre-registered response to the benchmark’s actual burn coming in above projection, and it was the ablation *sample* that was cut rather than any system or condition. The odd 198 is floor division within each stratum: CWQ’s strata of 137/211/49/3 halve to 68/105/24/1. Nothing in the analysis assumes a round number, and the consequence — reduced statistical power — is confronted directly in Section 8.7 rather than left implicit.

7.9 Hyperparameter Selection on the Development Set

7.9.1 Development Set Construction and Coverage

The development set is 80 questions mined from the train and validation splits of both benchmarks, selected to span hop counts and to include mediator-heavy and constraint-bearing questions, and certified for answer reachability by the same procedure applied to the test samples. It is disjoint

from both test samples by construction, and the disjointness is asserted by the test-set builder rather than assumed.

7.9.2 The α - τ Sweep Protocol

The two free parameters of the navigation loop are the blend weight α in the hybrid scorer (Section 5.5.2) and the low-signal backtracking threshold τ (Section 5.6.3). They were swept jointly over $\alpha \in \{0.3, 0.5, 0.7, 1.0\}$ and $\tau \in \{0.0, 0.20\}$ — eight conditions, each a complete run of all 80 questions, with two additional draft-only conditions to price the verification layer. Each condition is a constructor argument rather than a source edit, and its values are stamped into every record it writes, so no condition can be mislabelled after the fact.

The chosen values, $\alpha = 0.7$ and $\tau = 0.20$, were frozen on the basis of that sweep and never revisited. Section 8.1 reports the sweep, including the non-monotonicity at $\alpha = 0.5$ that a smoothed curve would have hidden.

7.10 Implementation, Environment, and Reproducibility

The system is implemented in Python against LangGraph for the state machine, the Neo4j Python driver for graph access [21], and sentence-transformers for embeddings [29]. The graph runs in a local Neo4j instance; all Cypher [22] is templated inside the tool layer and never composed by a language model (Section 5.2). Randomised operations use seed 42 throughout.

Three practices carry more reproducibility weight than the dependency list, and Chapter E records them as techniques. *Records self-describe*: each carries the backbone description, a budget-configuration hash, the full run configuration including the system name, and the source commit, so any record found anywhere states what produced it — which is what made the cache-replay incident of Section 7.3 diagnosable. *Frozen artifacts are committed and regenerable ones are not*: the certified question sets, their half-splits, and every result file embody spent money and non-repeatable runs, while the graph import, embedding matrices, and triple index are deterministic products of committed scripts. *Code changes are proved harmless by cache identity* rather than by a passing test suite: re-running an answered question and confirming every call is a cache hit proves byte-identical prompts, hence an untouched frozen path, and is the certificate behind the ablation reference row.

Two limits belong here rather than in a footnote. The archived results carry neither per-folder documentation nor generated manifests, so file provenance rests on the self-describing records and on this chapter. And one generated summary in the development-set archive predates a fix applied to the runs it summarises, so rescoring that directory yields different numbers than the committed summary; the discrepancy is stated where those numbers are used (Section 8.1) rather than resolved by editing a generated file.

Chapter 8

Results

This chapter reports the measurements the previous chapter specified. It is organised so that the weakest claims are visible next to the strongest: the headline accuracy result is large and significant on every comparison, the structural groundedness result is emphatic but attributes to a different cause than expected, the semantic groundedness result is underpowered, and three of the four ablations detect nothing. All four outcomes are reported at the same volume.

Every number comes from `results/phase4/thesis_numbers.json`. Confidence intervals are 95% bootstrap intervals over questions with 10,000 resamples at seed 42 [36]; paired comparisons use the exact-binomial form of McNemar’s test [37] on per-question correctness.

8.1 Development-Set Tuning Outcomes

The blend weight α and the low-signal backtracking threshold τ were swept jointly over the 80-question development set (Section 7.9). Table 8.1 reports all eight conditions plus the two draft-only conditions that price the verification layer.

Three readings, of which only the first is the one the sweep was run to obtain.

$\alpha = 0.7$ is the maximum, and the curve is not clean. Hits and F1 rise from $\alpha = 0.3$ to $\alpha = 0.7$ and fall sharply at $\alpha = 1.0$, which is the expected shape: embedding similarity alone ($\alpha = 1.0$) is a weaker guide than the blend, and weighting the language-model judgement too heavily ($\alpha = 0.3$) makes the score noisy. But the wrong-answer column does not follow the hits column. It reads 7, 8, 5, 6 across the four α values, peaking at $\alpha = 0.5$ — the condition that also has the fewest hedges of the verified rows. At $\alpha = 0.5$ the system commits more often and is wrong more often, and that irregularity is reported rather than smoothed away, because a presented curve that hides a known kink is exactly what a defence finds. It is one sample of 80 questions and the difference is three questions wide; it is noted, not interpreted.

τ does almost nothing on this set, and that is consistent with its design. At $\alpha = 0.7$ and $\alpha = 0.5$ the two τ values produce identical hits, hedges, and wrong counts. At $\alpha = 0.3$ they

Table 8.1: The development-set sweep, $n = 80$. Hits, hedges, and wrong answers partition the set. Verify indicates claim verification enabled; the two draft-only rows disable it. All rows are from the sweep as run, before the answer-entity filter of Section 6.5 was corrected; the frozen condition’s post-correction figures are given in the text.

α	τ	Verify	Hits	Hedge	Wrong	F1	Tokens	Calls	Bktrk
0.3	0.00	yes	61	12	7	0.728	4,999	7.4	16
0.3	0.20	yes	61	12	7	0.728	5,030	7.5	18
0.5	0.00	yes	62	10	8	0.743	4,874	7.2	16
0.5	0.20	yes	62	10	8	0.743	4,874	7.2	16
0.7	0.00	yes	64	11	5	0.769	4,925	7.3	17
0.7	0.20	yes	64	11	5	0.769	4,925	7.3	17
1.0	0.00	yes	59	15	6	0.713	3,408	4.9	22
1.0	0.20	yes	59	16	5	0.713	3,400	4.9	26
0.5	0.20	no	64	9	7	0.770	4,807	7.1	16
0.7	0.20	no	66	10	4	0.797	4,858	7.2	17

differ only in backtrack count (16 against 18); at $\alpha = 1.0$, one wrong answer becomes a hedge and backtracks rise from 22 to 26. This is what the signal-maximum formulation of Section 5.6.3 predicts: the trigger fires only when the blended score, the embedding maximum, *and* the language-model maximum are all below threshold, which is rare when the language-model term is contributing. The $\alpha = 1.0$ row is where it has the most to do, and it is exactly there that its effect is visible. The value $\tau = 0.20$ was frozen on the basis of a sweep in which it changed almost nothing; that is stated plainly rather than presented as a tuned choice.

Verification costs accuracy on this set. Against the frozen condition, the draft-only row scores 66 hits to 64 and F1 0.797 to 0.769. After the answer-entity filter of Section 6.5 was corrected — a change made on development data before any test question ran — the frozen condition scores 65 hits, 11 hedges, 4 wrong, F1 0.785. So on the final code the layer costs one correct answer and removes no wrong ones on this set of 80. Section 8.7 finds the same thing at $n \approx 200$ on test data, and Section 8.8 explains why that is the expected result rather than a disappointing one.

One reproducibility note belongs with this table. Only the frozen condition was re-run after the filter correction, so Table 8.1 is the pre-correction sweep throughout — internally consistent, since all eight conditions ran under the same code. The committed summary file for that directory also predates the correction, so rescoring the archived runs yields 65/11/4 for the frozen condition where the summary records 64/11/5. The archived per-question records are authoritative.

8.2 Main Results on WebQSP

On WebQSP, AGR answers 302 of 400 questions correctly: 75.5% against a reachability ceiling of 97.0%, +9.5 points over Think-on-Graph and +30.2 over the parametric floor. The floor

Table 8.2: Main results, $n = 400$ per dataset. Brackets are 95% bootstrap intervals. **P** and **R** are mean per-question precision and recall; **Hedge** is the share of questions on which the system asserted nothing. The Hits@1 ceilings are 97.0% and 99.2% (Table 7.1).

Data	System	Hits@1	F1	P	R	Hedge
WebQSP	No-retrieval	0.453 [0.403, 0.500]	0.305 [0.266, 0.345]	0.380	0.303	12.2%
WebQSP	Vector-RAG	0.568 [0.517, 0.618]	0.432 [0.389, 0.475]	0.469	0.460	27.0%
WebQSP	GraphRAG	0.340 [0.292, 0.388]	0.262 [0.224, 0.302]	0.279	0.284	55.8%
WebQSP	Think-on-Graph	0.660 [0.613, 0.705]	0.477 [0.436, 0.518]	0.571	0.480	18.8%
WebQSP	AGR	0.755 [0.713, 0.797]	0.642 [0.600, 0.684]	0.697	0.653	8.2%
CWQ	No-retrieval	0.307 [0.263, 0.352]	0.279 [0.236, 0.323]	0.285	0.282	38.0%
CWQ	Vector-RAG	0.203 [0.163, 0.242]	0.168 [0.133, 0.204]	0.171	0.189	48.2%
CWQ	GraphRAG	0.205 [0.168, 0.245]	0.183 [0.147, 0.222]	0.187	0.194	60.8%
CWQ	Think-on-Graph	0.435 [0.388, 0.482]	0.378 [0.332, 0.423]	0.403	0.384	22.5%
CWQ	AGR	0.522 [0.475, 0.570]	0.469 [0.423, 0.514]	0.484	0.481	23.0%

matters. WebQSP is a decade old and plausibly in the backbone’s pretraining data, and the no-retrieval control confirms it: 45.3% of these questions are answerable from memory alone. Any claim about what retrieval contributes has to be read against 0.453, not against zero.

F1 widens the margin that Hits@1 understates. Against Think-on-Graph the Hits@1 gap is +9.5 points; the F1 gap is +16.5 (0.642 against 0.477), and the precision column shows where it comes from (0.697 against 0.571). WebQSP’s questions have a mean of eight gold answers, so a system that asserts a long list collects Hits@1 credit for the one entity that matches and pays nothing for the rest. Precision is what charges for it. The two columns together support the sentence this thesis leads with: AGR answers more questions and asserts fewer falsehoods, at an 8.2% hedge rate — the lowest of the five systems, so the precision is not bought with abstention.

Raw hits mislead on the retrieval baselines, and the correction is a finding. GraphRAG scores 0.340, below the no-retrieval control’s 0.453, which invites the conclusion that the implementation is broken. Put the wrong-answer counts beside the hits and the picture inverts. No-retrieval asserts on 351 of 400 questions and is wrong on 170 of them — 51.6% assertion precision. GraphRAG asserts on 177 and is wrong on 41 — 76.8%. The mechanism is the prompt: the retrieval baselines are instructed to answer only from the facts given and to return nothing otherwise, so when retrieval misses they abstain, while the no-retrieval control has no such constraint and guesses from memory. On a memorised benchmark, guessing pays. *A grounded system scoring below an ungrounded one on raw hits while committing four times fewer falsehoods* is the contamination story the control was built to expose, and it is reported as a result rather than as an anomaly.

Table 8.3: Paired McNemar tests, AGR against each baseline, on per-question correctness over the same 400 questions. Every comparison rejects.

Dataset	Baseline	AGR only	Baseline only	p
WebQSP	No-retrieval	152	31	2.3×10^{-20}
WebQSP	Vector-RAG	117	42	2.2×10^{-9}
WebQSP	GraphRAG	186	20	6.5×10^{-35}
WebQSP	Think-on-Graph	76	38	4.8×10^{-4}
CWQ	No-retrieval	121	35	2.7×10^{-12}
CWQ	Vector-RAG	151	23	2.8×10^{-24}
CWQ	GraphRAG	148	21	1.0×10^{-24}
CWQ	Think-on-Graph	86	51	3.5×10^{-3}

8.3 Main Results on ComplexWebQuestions

On CWQ, AGR answers 209 of 400: 52.2% against a 99.2% ceiling, +8.7 over Think-on-Graph and +21.5 over the floor. Every system scores lower here, and the ordering changes in one important way: *both* static retrieval baselines fall below the parametric control. Vector-RAG drops from 0.568 on WebQSP to 0.203, GraphRAG from 0.340 to 0.205, while no-retrieval falls only from 0.453 to 0.307.

The reason is structural rather than incidental. A CWQ question needs a chain, and one verbalised triple cannot contain a chain — no amount of retrieval quality repairs that, which is the paradigm limit Section 7.4 predicted at implementation time and Section 8.4 demonstrates per stratum.

The number worth leading with on this dataset is assertion precision. AGR and Think-on-Graph commit at effectively the same rate — 308 and 310 of 400 questions answered rather than hedged — and out of those near-identical commitments AGR is right 209 times and wrong 99, against 174 and 136. That is 67.9% assertion precision against 56.1%: on the harder, deeper benchmark, the system with an explicit plan, a guided scorer, and a claim check finds 35 more answers and asserts 37 fewer falsehoods than blind beam search on the same graph with the same model, from the same number of commitments.

Table 8.3 settles the comparisons that matter. All eight reject, including the two against the agentic baseline — the only comparison in the table where both systems navigate the same graph with the same model under the same budget, so that the algorithm is the only difference. The discordant counts make the margin concrete rather than distributional: on CWQ, AGR uniquely solves 86 questions Think-on-Graph misses against 51 the other way.

8.4 Accuracy Broken Down by Hop Count

Table 8.4 is the direct answer to RQ1, and it answers it with a shape rather than with a difference of means.

Table 8.4: Hits@1 and F1 per hop stratum. Stratum sizes are those of Table 7.1. WebQSP’s h3plus has $n = 4$ and is not interpreted.

Data	System	h1	h2	h3plus	unreach.
WebQSP	No-retrieval	0.43 / 0.29	0.52 / 0.36	0.25 / 0.00	0.17 / 0.17
WebQSP	Vector-RAG	0.82 / 0.63	0.12 / 0.07	0.00 / 0.00	0.08 / 0.08
WebQSP	GraphRAG	0.30 / 0.23	0.44 / 0.35	0.25 / 0.07	0.08 / 0.08
WebQSP	Think-on-Graph	0.63 / 0.45	0.78 / 0.58	0.25 / 0.07	0.17 / 0.17
WebQSP	AGR	0.75 / 0.64	0.84 / 0.73	0.00 / 0.00	0.08 / 0.08
CWQ	No-retrieval	0.38 / 0.35	0.30 / 0.27	0.14 / 0.14	0.00 / 0.00
CWQ	Vector-RAG	0.33 / 0.27	0.14 / 0.11	0.10 / 0.09	0.67 / 0.67
CWQ	GraphRAG	0.34 / 0.30	0.16 / 0.14	0.02 / 0.02	0.33 / 0.33
CWQ	Think-on-Graph	0.53 / 0.48	0.37 / 0.32	0.45 / 0.32	0.33 / 0.33
CWQ	AGR	0.46 / 0.42	0.55 / 0.49	0.57 / 0.50	0.33 / 0.33

On CWQ, AGR’s accuracy rises with hop count. Hits@1 goes $0.46 \rightarrow 0.55 \rightarrow 0.57$ across h1, h2, h3plus, and F1 goes $0.42 \rightarrow 0.49 \rightarrow 0.50$. Every other system on that dataset decays: Think-on-Graph falls from 0.53 to 0.37 before partially recovering at 0.45; Vector-RAG collapses $0.33 \rightarrow 0.14 \rightarrow 0.10$; GraphRAG $0.34 \rightarrow 0.16 \rightarrow 0.02$. A system that gets *better* as the required chain gets longer is not merely ahead on average — it is the only one in the table whose profile is consistent with actually traversing the chain, and that monotonicity is a stronger form of the multi-hop claim than any aggregate could carry.

Think-on-Graph beats AGR on CWQ’s one-hop stratum (0.53 against 0.46, $n = 137$), and this is reported rather than buried because it makes the rest of the story more credible, not less. Section 8.7 identifies the mechanism: decomposition costs accuracy on questions that do not need decomposing, and CWQ’s h1 stratum is full of them.

Vector-RAG’s WebQSP profile is the paradigm in two cells. It scores 0.82 at one hop — the best single-hop number anywhere in the table, better than AGR’s 0.75 — and 0.12 at two. When one triple suffices, dense retrieval over verbalised triples is excellent; when a chain is required, it has nothing to offer. GraphRAG’s WebQSP inversion, 0.30 at h1 against 0.44 at h2, is the complementary artefact: popular one-hop entities have large neighbourhoods, and hub noise drowns the answer in the reranked window.

The unreachable stratum is a hallucination probe, not an accuracy measurement. By construction no path to gold exists within the hop bound for these questions, so every “hit” is an assertion the environment cannot support that happened to match the label. Vector-RAG scores 0.67 on CWQ’s three unreachable questions and no-retrieval 0.17 on WebQSP’s twelve. The counts are tiny and no weight is placed on them, but the direction is exactly what Section 8.5 measures at scale: Hits@1 will happily reward what RQ2 is about the absence of.

Table 8.5: Two-tier groundedness. Tier 1 is deterministic over every asserted entity in all 4,000 records; Tier 2 is a language-model entailment judgement over a stratified sample of 60 assertions per system per dataset, validated at $\kappa = 0.6995$ (Section 7.6).

Data	System	Tier 1: structural			Tier 2
		Entities	Ungrounded	Questions	Supported
WebQSP	No-retrieval	661	179 (27.1%)	37.6%	63.3%
WebQSP	Vector-RAG	1040	38 (3.7%)	6.2%	61.7%
WebQSP	GraphRAG	800	6 (0.8%)	2.8%	55.0%
WebQSP	Think-on-Graph	832	0 (0.0%)	0.0%	61.7%
WebQSP	AGR	1235	0 (0.0%)	0.0%	66.7%
CWQ	No-retrieval	340	42 (12.4%)	13.7%	36.7%
CWQ	Vector-RAG	289	7 (2.4%)	3.4%	50.0%
CWQ	GraphRAG	224	2 (0.9%)	1.3%	36.7%
CWQ	Think-on-Graph	433	0 (0.0%)	0.0%	43.3%
CWQ	AGR	474	0 (0.0%)	0.0%	48.3%

8.5 Groundedness and Hallucination Rate Across Systems

8.5.1 Tier 1: Structural Hallucination Is a Property of the Paradigm

The differential is emphatic. The parametric control asserts 661 entities on WebQSP of which 179 — 27.1% — have no basis in the knowledge environment, and 37.6% of the questions it answers contain at least one such entity. Both navigating systems sit at exactly zero, over 1,235 and 832 asserted entities. Parametric answering hallucinates structurally at double-digit rates; graph-navigated answering eliminates structural hallucination outright.

But Think-on-Graph reaches zero as well, and that changes the attribution. Structural groundedness is a property of the *navigation paradigm* — any system that copies its answer entities out of triples it actually traversed inherits it — not of this thesis’s verification layer specifically. The honest claim is therefore narrower and more testable than the one the work set out to make: *both navigating systems achieve zero structural hallucination; they differ at the level of semantic support*, where Think-on-Graph’s precision of 0.571 and 0.403 against AGR’s 0.697 and 0.484 (Table 8.2) shows it asserting substantially more entities that exist, connect to the topic, and were copied from real triples — and are not the answer. Section 1.3 builds on exactly this gap.

Two readings of the middle band belong on the record. The no-retrieval control’s CWQ rate (12.4%) is less than half its WebQSP rate (27.1%) not because it fabricates less on harder questions but because it hedges more — 38.0% against 12.2% — and abstention suppresses the measured hallucination rate. That is the abstention trade appearing in a third metric. And Vector-RAG’s 38 ungrounded entities are mostly not fabrications: its index is global, so retrieval can surface facts about a similarly-named subject that exists in the graph but has no connection to the question’s topics, and the model copies faithfully from them. The metric correctly flags those

as unanchored to the question, but the mechanism is retrieval error rather than generation error. Structural ungroundedness conflates the two, and the distinction is recoverable from whether the entity appears in the system’s own retrieved facts.

8.5.2 Tier 2: A Narrow Band and an Underpowered Comparison

Tier 2 asks whether the asserted entity is supported *as the answer*, and under the instruction that mere connectedness is not support, every system lands in a 36.7–66.7% band. AGR is highest on both datasets, by 5.0 points over Think-on-Graph on WebQSP and 5.0 on CWQ.

That gap is not resolvable at this sample size. With $n = 60$ per cell the 95% binomial interval has a half-width of roughly 12–13 points, so a 5-point difference is inside the noise, and no conclusion in this thesis rests on it. What the tier does establish is the level: semantic support is *hard* for every system, including the two that are structurally perfect. Zero structural hallucination coexists with roughly a third to a half of assertions failing a strict semantic-support test — which is the single most useful thing the metric says, and it says it about the paradigm rather than about any one system.

Two properties of the measurement bound its interpretation, both pre-registered. Path retrieval takes the three shortest connecting paths, so a genuinely supporting chain can lose to shorter irrelevant ones; the rates are conservative lower bounds, depressed uniformly across systems, which preserves the between-system comparison while making the absolute levels untrustworthy. And the judge was validated at $\kappa = 0.6995$ against a pre-registered bar of 0.7 (Section 7.6) — substantial agreement, marginally short of the threshold, and a further reason to treat this tier as corroborating rather than decisive.

8.6 Efficiency and Cost

8.6.1 Token and Call Budgets per System

Agency costs roughly an order of magnitude over one-shot retrieval: 4,511 and 6,818 tokens per question against ~ 500 , and 11–17 seconds against one to two. That is the price of RQ1’s accuracy, and it is stated as a price.

The comparison that carries information is against the other agentic system, since the two run under an identical 25-call cap. AGR uses *half* the calls — 6.2 against 12.8 on WebQSP, 8.9 against 18.2 on CWQ — with more tokens per call, because each call carries a richer prompt: a plan, a scored candidate set, a fact window. Fewer, better-informed decisions rather than more, blinder ones.

The clip column is the sharpest form of that. Beam search costs are multiplicative in width and depth, so under the shared budget Think-on-Graph exhausts its calls before finishing on 29.2%

Table 8.6: Cost per question. Latency is computed over cold-cache records only. Clip rate is the share of questions on which the shared 25-call budget was exhausted mid-search.

Data	System	Tokens	Calls	Seconds	Clipped
WebQSP	No-retrieval	113	1.0	1.2	—
WebQSP	Vector-RAG	527	1.0	1.4	—
WebQSP	GraphRAG	531	1.0	2.1	—
WebQSP	Think-on-Graph	3,615	12.8	12.9	29.2%
WebQSP	AGR	4,511	6.2	11.2	0.0%
CWQ	No-retrieval	118	1.0	1.0	—
CWQ	Vector-RAG	535	1.0	1.4	—
CWQ	GraphRAG	470	1.0	2.2	—
CWQ	Think-on-Graph	5,572	18.2	15.2	44.0%
CWQ	AGR	6,818	8.9	17.2	0.0%

of WebQSP questions and 44.0% of CWQ questions, while AGR never does. On nearly half the multi-hop benchmark the baseline is answering from a truncated search. This is a fair constraint — both systems live under the same cap, and a clipped run still answers from what it gathered by the same graceful degradation — and it is the efficiency finding rather than a caveat on it: under an identical budget, AGR converts its calls into correct answers at roughly twice the rate.

8.6.2 The Accuracy–Cost Frontier

Reading Table 8.2 against Table 8.6, the frontier has three points and no interior ones. Vector-RAG on WebQSP is the cheap corner: 0.568 Hits@1 for 527 tokens, and 0.82 on the one-hop stratum, which makes it the right choice for shallow questions and useless for deep ones. AGR is the accurate corner on both datasets. Think-on-Graph sits between them on accuracy and above AGR on calls, so it is dominated on this environment — lower accuracy at higher call cost. The no-retrieval control is off the frontier entirely once precision is counted, though it remains the cheapest way to be right about half the time on WebQSP, which is the contamination point restated as an economic one.

8.7 Ablation Study

Four single-deviation conditions on stratified halves ($n = 200$ WebQSP, $n = 198$ CWQ), each against the unmodified system restricted to the same questions. Table 8.7 reports the conditions and Table 8.8 summarises the component attribution. Throughout, Δ is *ablated minus reference*, so a positive Δ means removing the component *improved* the metric.

Table 8.7: Ablation conditions on stratified halves. The reference row is the frozen system restricted to the same questions, at zero additional cost. Latency is unmeasured for no-verifier because every record in that condition is a full cache replay.

Data	Condition	Hits@1	F1	Hedge	Tok.	Calls
WebQSP	reference	0.755 [0.695, 0.810]	0.659 [0.599, 0.717]	8.5%	4,562	6.2
WebQSP	no-planner	0.830 [0.775, 0.880]	0.742 [0.687, 0.794]	5.5%	3,159	4.0
WebQSP	no-backtrack	0.745 [0.685, 0.805]	0.646 [0.586, 0.705]	9.5%	4,261	5.8
WebQSP	no-verifier	0.760 [0.700, 0.820]	0.663 [0.604, 0.721]	8.0%	4,460	6.1
WebQSP	embedding-only	0.740 [0.680, 0.800]	0.643 [0.583, 0.702]	13.5%	3,413	4.4
CWQ	reference	0.495 [0.424, 0.566]	0.445 [0.379, 0.511]	23.2%	7,105	9.1
CWQ	no-planner	0.434 [0.369, 0.505]	0.398 [0.333, 0.464]	26.8%	5,617	6.7
CWQ	no-backtrack	0.490 [0.419, 0.561]	0.445 [0.379, 0.511]	28.3%	5,792	7.4
CWQ	no-verifier	0.490 [0.419, 0.561]	0.436 [0.371, 0.503]	20.2%	6,704	8.7
CWQ	embedding-only	0.475 [0.409, 0.545]	0.437 [0.370, 0.502]	28.3%	4,925	5.9

8.7.1 Without the Planner: A Stratum-Dependent Effect

This is the one condition that reaches significance, and it points the opposite way to the design intent.

On WebQSP, *removing* the planner improves every axis at once: Hits@1 $0.755 \rightarrow 0.830$, F1 $0.659 \rightarrow 0.742$, hedge rate $8.5\% \rightarrow 5.5\%$, tokens $4,562 \rightarrow 3,159$, calls $6.2 \rightarrow 4.0$. McNemar gives $p = 5.9 \times 10^{-3}$ on 6 against 21 discordant pairs. The per-stratum breakdown locates the damage precisely: h1 rises $0.75 \rightarrow 0.85$ and h2 rises $0.84 \rightarrow 0.89$, so decomposition is hurting simple *and* two-hop questions on this dataset. The planner prompt’s first worked example instructs the model not to decompose single-hop questions; the ablation proves that instruction under-triggers, with a p -value.

On CWQ the sign reverses and significance is not reached: Hits@1 $0.495 \rightarrow 0.434$, 27 against 15 discordant pairs, $p = 0.088$. The stratum table shows a coherent mechanism rather than noise — h1 *improves* ($0.49 \rightarrow 0.54$) while h2 collapses ($0.48 \rightarrow 0.34$) and h3plus moves little ($0.62 \rightarrow 0.54$) — which is precisely “decomposition helps genuine composition and hurts questions that do not need it”. But it does not clear $\alpha = 0.05$ at $n = 198$, and the honest statement carries both halves: *the planner’s benefit is stratum-conditional and significant on WebQSP* ($p = 0.006$); *the CWQ pattern points the same way but does not reach significance at this sample size* ($p = 0.088$).

The finding is therefore not “the planner helps” or “the planner hurts” but that its value is conditional on question structure, and that applying decomposition unconditionally is the defect. Section 9.3 reads the 21 and 15 questions where removing the planner flipped a failure into a success and names the mechanisms; Section 10.3 turns them into an adaptive gating proposal that this ablation earned rather than motivated.

8.7.2 Without Backtracking

Hits@1 moves -0.010 on WebQSP and -0.005 on CWQ; F1 -0.013 and 0.000 . Discordant pairs are 5 against 3 and 6 against 5, giving $p = 0.73$ and $p = 1.00$. There is no detectable effect at this sample size.

Two readings are available and the data does not separate them: backtracking’s contribution may be small because the scorer and the verifier already prevent most of the excursions it exists to correct, or $n \approx 200$ may simply be too few questions to resolve a small effect. This is reported as “no detectable effect at $n \approx 200$ ”, never as a confirmed null. The mechanism is not idle — the full runs record 63 backtracks on WebQSP and 199 on CWQ, dominated by evaluator-triggered ones (47 and 170) — so the condition removes real activity whose accuracy consequence this experiment cannot measure.

8.7.3 Without the Verification Layer

The cleanest null in the table, and the most informative one. Discordant pairs are 0 against 1 on WebQSP and 1 against 0 on CWQ: across 398 paired questions, removing claim verification changed the correctness of exactly two. Hits@1 moves $+0.005$ and -0.005 ; F1 $+0.004$ and -0.009 . Both $p = 1.00$.

This reproduces the development-set result of Section 8.1 exactly, and it is what Chapter 6 predicted by construction: on 77 of 80 development questions the layer certified the draft and changed nothing, and the repair path never executed. A mechanism that mostly passes cannot move an accuracy metric.

The conclusion this licenses is narrower and sharper than “the verifier helps”, and it is the one Section 8.8 adopts: *claim verification does not measurably change how often AGR is right; it changes how often AGR is right for a reason it can exhibit*. Hits@1 cannot see that distinction — it is what the groundedness metrics were built to see, and what the supporting-triple output contract of Section 6.6 exists to make checkable. The evidence for the layer is Table 8.5, the precision column of Table 8.2, and the specimen analyses of Section 9.5; it is not this row, and this row is reported at full length rather than omitted.

8.7.4 Embedding-Only Scoring

Removing the language-model term from the hybrid scorer costs -0.015 and -0.020 Hits@1 and -0.016 and -0.008 F1 — consistently negative on both datasets and both metrics, and consistently short of significance ($p = 0.66$ and $p = 0.48$).

Two corroborating details make the direction credible without over-claiming from it. The condition is the cheapest of the four (3,413 and 4,925 tokens, 4.4 and 5.9 calls), so the language-model scoring term is buying accuracy with tokens as designed. And it carries by far the most

Table 8.8: Component attribution, RQ3. $\Delta F1$ is *ablated minus reference*, so positive means removing the component improved F1. Only the planner condition reaches significance.

Component	WebQSP $\Delta F1$ (p)	CWQ $\Delta F1$ (p)	Verdict
Planner	+0.083 (0.006)	−0.047 (0.088)	Stratum-dependent
Backtracking	−0.013 (0.73)	0.000 (1.00)	No detectable effect
Verification	+0.004 (1.00)	−0.009 (1.00)	No accuracy effect
α -blend	−0.016 (0.66)	−0.008 (0.48)	Directional, underpowered

backtracks — 57 and 143 against the reference’s 38 and 108 — which is the τ mechanism of Section 5.6.3 visible in the logs: with the language-model term removed, the signal maximum clears τ less often, so the search machinery works harder to compensate. The primary evidence for $\alpha = 0.7$ remains the development sweep of Table 8.1, where the $\alpha = 1.0$ condition lost five hits on 80 questions; this ablation corroborates it at test scale rather than independently establishing it.

8.7.5 Paired Significance Testing and Statistical Power

One of four conditions reaches significance. That is the correct outcome to report for four components tested on half-samples, and it is reported as such rather than rewritten into four confident claims.

The power limitation is arithmetic, not rhetorical. McNemar’s test reads only discordant pairs, and at $n \approx 200$ with a system near 0.5–0.75 accuracy, a component that changes five questions in a hundred produces on the order of ten discordant pairs — not enough to reject at conventional levels. The planner condition cleared the bar on WebQSP because its effect is large (21 against 6), not because that comparison was better powered than the others. The halves were adopted as the pre-registered response to the benchmark’s cost coming in above projection (Section 7.8); doubling them is the straightforward remedy and is named in Section 10.2.

Three conditions are therefore reported as “no detectable effect at this sample size”. That phrase is not a hedge for “no effect”: for the verifier the point estimates are near zero *and* the mechanism’s contribution was never expected to appear in accuracy, whereas for backtracking and the α -blend the point estimates are consistently signed and the mechanism is visibly active in the logs. Those are different epistemic situations, and collapsing them into one word would lose the distinction.

8.8 Findings Against RQ1, RQ2, and RQ3

RQ1 — does agentic navigation improve multi-hop factual accuracy? Yes, and the multi-hop qualifier is doing real work. AGR leads every cell of Table 8.2, and all eight paired comparisons

reject, including both against the agentic baseline running on the same graph with the same model under the same budget. The stratum profile is the substance of the answer: on CWQ, AGR’s accuracy *rises* with hop count ($0.46 \rightarrow 0.55 \rightarrow 0.57$) while every other system decays, and static retrieval collapses from 0.33 to 0.10. Two boundaries are reported with it — Think-on-Graph is better on CWQ’s one-hop stratum, and Vector-RAG is better than everything on WebQSP’s.

RQ2 — what does pre-generation verification contribute beyond navigation? The answer has three parts that a yes-or-no question would have merged, which is why it was not asked as one.

First, navigation eliminates structural hallucination, and the verifier is not what does it. Both navigating systems assert zero ungrounded entities, over 1,235 and 832 assertions, against 27.1% for parametric answering. Any system that copies its answer entities out of triples it traversed inherits this; attributing it to the verification layer would be wrong, and Table 8.5 is the evidence that it would be wrong.

Second, the verifier’s measurable contribution is precision, not accuracy. AGR’s assertion precision exceeds Think-on-Graph’s by 12 points on CWQ (67.9% against 56.1%) and its per-question precision by 0.13 and 0.08, at a lower hedge rate on WebQSP — so it is not bought with abstention. The mechanism is visible in Chapter 6: entity-level filtering of the final answer removes assertions the traversal cannot support, which costs recall almost nothing and costs a false assertion every time it fires.

Third, verification shows no measurable Hits@1 effect. Two questions out of 398 change when it is removed. This is not a failure of the layer but a statement about what Hits@1 measures: a metric satisfied by one correct entity among several cannot see a mechanism that operates on the others. The layer’s value proposition is that an answer arrives with the triples that support it (Section 6.6) — an auditability claim, not an accuracy claim — and the honest formulation is that *claim verification does not change how often the system is right; it changes how often the system is right for a reason it can exhibit*.

RQ3 — which components contribute what, at what token cost? One of four reaches significance. The planner is stratum-dependent: removing it improves WebQSP by 0.083 F1 at $p = 0.006$ while saving 30% of tokens and 35% of calls, and costs CWQ 0.047 F1 at $p = 0.088$. That asymmetry, not a pooled average, is the result — and it is a finding about how decomposition should be *applied* rather than about whether it works. Backtracking, verification, and the α -blend show no detectable effect at $n \approx 200$, for the different reasons Section 8.7 separates. On cost, the whole apparatus runs at half the language-model calls of the agentic baseline and never exhausts a budget that clips the baseline on 44% of CWQ questions.

The three answers do not point the same way, and the chapter does not force them to. Navigation is where the accuracy comes from; verification is where the precision and the auditability come from; and decomposition, the component the literature treats as straightforwardly beneficial, is the one component this work can prove is sometimes harmful.

Chapter 9

Error Analysis and Discussion

Chapter 8 measured how often each system fails. This chapter reads the failures and names the mechanism of each one. It is the part of the work no script produced: 259 failure packets — question, gold, answer, plan, explored relations with their scores, backtrack reasons, verifier outcome — read and labelled by hand, one category and one sentence each.

The reading protocol was fixed before it began. For each packet, find where the trajectory *first* left the correct path, and label that point rather than the last visible symptom; when torn, take the earliest plausible cause and record the alternative. That rule is what keeps a run that mis-decomposes, then explores the wrong relation, then hedges from being counted three times under three categories.

9.1 Annotation Protocol and the Failure Taxonomy

9.1.1 Category and Subtype Schema

Ten categories carry the census, each naming a distinct architectural locus rather than a symptom: `relation_selection` (the scorer picked the wrong edge), `composite_claim` (a multi-constraint question handled partially), `kg_gap` (the environment cannot express what the question needs), `decomposition_error` (the planner or draft pipeline caused it), `answer_selection` (the correct candidate was retrieved and the drafter chose another), `echo` (a real, grounded entity one node away from the answer), `premature_termination` (the search stopped before the answer), `verifier_fn` and `verifier_fp` (a right claim rejected, a wrong one accepted), and `gold_noise / ambiguous_question` for the benchmark defects that escaped the exclusion pass of Section 7.7. An other category was kept open for mechanisms the schema did not anticipate, and Section 9.3 reports what fell into it.

Subtypes are a second, open vocabulary that drives nothing and exists to group cases when writing. The full schema, with the worked examples that fixed each boundary, is Section D.2.

Table 9.1: Census provenance. Stage D is the main reading over the full-matrix failures; Stage A is the separate reading of the questions where removing the planner flipped a failure into a success (Section 8.7); the single migrated row is a Stage D finding later promoted to a formal benchmark-defect exclusion (Section 9.8).

Source	WebQSP	CWQ
Stage D — main census (all remaining wrongs and hedges)	65	157
Stage D — row migrated to a benchmark exclusion	0	1
Stage A — planner-discordance census	21	15
Total failures analysed	86	173

One naming decision is recorded here because it was got wrong once and corrected. The verifier’s *positive* class is “the claim is supported”, so a false positive is a claim wrongly *accepted* and a false negative one wrongly *rejected*. The taxonomy’s gloss originally paired those in the opposite order. It was corrected, two mislabelled rows were re-filed (Sections 9.3.2 and 9.5), and the merged histogram was regenerated. Section 9.5.1 states the convention once; everywhere else this thesis says “wrongly rejected” and “wrongly accepted” in plain English, because with outputs labelled supported and unsupported a reader has no reliable way to infer which class is positive.

9.1.2 Population, Not Sample

The census is a *population*, not a sample. Both datasets were read to completion: every wrong answer and every hedge that survived the benchmark-defect exclusions of Section 7.7. Table 9.1 gives the provenance.

An earlier plan drew a stratified $40 + 20$ sample from CWQ’s failures; the census was read to completion instead, so there is no sampling-bias caveat on any number in this chapter. What does remain is a *composition* caveat: wrong answers and hedges are reported separately throughout and never pooled. They are different failure semantics — a wrong answer is a reasoning error that committed, a hedge is usually a coverage gap that never produced a committal claim — and their proportions differ sharply between the two datasets, so a pooled percentage would describe neither.

9.2 Distribution of Failure Categories Across Datasets

9.2.1 The Shape Flip, and What It Is Not

The two datasets fail differently, but not in the way a first reading of Table 9.2 suggests. `relation_selection` is the largest category on *both* — 26 of WebQSP’s 86 and 39 of CWQ’s

Table 9.2: The merged failure histogram, $n = 259$. Wrong and hedge are kept separate. Generated by scripts/synthesize_census.py over the three label sources of Table 9.1.

	WebQSP		CWQ		
Category	wrong	hedge	wrong	hedge	Total
relation_selection	20	6	20	19	65
composite_claim	1	—	23	23	47
kg_gap	8	4	8	24	44
decomposition_error	15	11	10	2	38
answer_selection	1	1	8	5	15
echo	5	2	4	2	13
ambiguous_question	1	1	3	5	10
premature_termination	1	4	1	2	8
gold_noise	1	—	3	3	7
verifier_fn	—	3	1	3	7
other	—	1	—	5	6
verifier_fp	—	—	—	1	1
Total	53	33	81	92	259

173 — so it is not what distinguishes them. Picking the wrong edge is the base rate of graph navigation, and it is dataset-independent.

What distinguishes them is a pair of categories that are nearly absent from one and dominant in the other. `composite_claim` is 1 of 86 on WebQSP and 46 of 173 on CWQ; `kg_gap` is 12 and 32. In the other direction, `decomposition_error` is 26 on WebQSP — tied with `relation_selection`, the two together accounting for nearly 60% of its failures — against 12 on CWQ.

That asymmetry is demonstrable rather than assertable, from two facts already established. First, the stratum distribution of Table 7.1: WebQSP is 64% one-hop with a four-question deep tail, so most of its questions have no composition to get wrong and no constraint to drop, while presenting the planner with exactly the questions that do not need decomposing — which is also the significant ablation result of Section 8.7, appearing here as a category count. Second, the expressiveness limits certified in Section 4.7.4: the environment carries no date literals, no numeric literals, and no ordinals, and CWQ’s SPARQL templates invoke all three constantly while WebQSP’s naturally-occurring questions rarely do. The failure profile of a system on a dataset is, to this extent, a property of the dataset’s construction.

9.2.2 Hedging Where It Should

A second reading of Table 9.2 is worth stating as a positive result rather than only as a deficit. On CWQ, `kg_gap` skews three-to-one toward hedges — 24 against 8 — while on WebQSP it leans the other way, 4 against 8. When CWQ’s numeric identifiers, date literals, and ordinal constraints are unreachable, the system much more often declines than asserts something wrong. That is the behaviour the anti-vacuity and grounding provisions of Section 6.2 were written to

Table 9.3: The entity-extraction bug. In each case the drafted answer text names every gold value correctly and the scored entity list holds only the sentence’s grammatical subject.

Question ID	Gold	Named in text	Scored entities
WebQTest-1215	6 professions	all 6	[Stephen Covey]
WebQTest-704	6 professions	all 6	[Thor Heyerdahl]
WebQTrn-124	4 films	all 4	[Angelina Jolie]

produce, and it is visible in the one place where the environment guarantees the system cannot succeed.

9.3 Decomposition, Drafting, and Answer-Selection Errors

Three families share this section because they share a locus: everything downstream of retrieval and upstream of the answer. The reasoning is often complete and correct by the time these failures occur, which is what makes them both diagnostic and cheap to fix.

9.3.1 The Entity-Extraction Bug

Nine of the 38 `decomposition_error` cases carry the subtype `extraction_bug`, and they are the most actionable finding in the census. On profession-shaped and “what did *X* do” questions, the evaluator resolves every correct gold value, the drafted text states them verbatim, and `answer_entities` then collapses to the sentence’s grammatical *subject*. Table 9.3 gives three instances read from the raw traces.

Table 9.3 understates how complete the drafts are, so one is worth quoting whole. For WebQTest-1215, “who was stephen r covey”, gold lists Author, Manager, Writer, Consultant, Professor, and Motivational speaker; the drafted answer reads “*Stephen Covey was a Writer, Motivational speaker, Author, Consultant, Professor, and Manager*” — all six, correct, in a different order — and `answer_entities` comes back as [‘Stephen Covey’].

This is not a reasoning failure at all. The navigation found the answers, the verifier certified them, and the prose reports them. What fails is the step that turns answer text into the entity list, which consistently reaches for the subject rather than the predicate values.

The consequence has to be stated as a limitation on Chapter 8, not merely as a bug: scoring reads `answer_entities`, so every one of these cases is recorded as a wrong answer or a hedge in Table 8.2. The reported accuracy of this system on “list the professions” and “list what *X* did” question shapes is therefore depressed by an extraction defect independent of any reasoning quality. Nine cases across 259 is not enough to move the headline numbers appreciably, but the correct reading of them is that the measured figure is a floor for that question shape, and Section 10.3 lists the fix among the repairs the census earned.

9.3.2 Right Candidate Retrieved, Wrong One Drafted

`answer_selection` (15 cases, 13 of them on CWQ) is the extraction bug’s more consequential sibling: the exact gold value demonstrably entered the resolved candidate set, and the drafter chose something else. `WebQTest-1555` asks what the parliament of Nepal is called; the evaluator resolves `Parliament of Nepal` itself, and the draft names its two component chambers instead. `WebQTrn-2784` asks which Tupac Shakur film Malcolm Campbell edited; the trace records `resolved=[‘Nothing but Trouble’]`, the literal gold value satisfying both constraints, and the draft substitutes `Gang Related` and then hedges on it.

One case in this family was originally filed as a verifier error and is worth the correction in full, because it is the kind of mistake this taxonomy exists to prevent. `WebQTrn-1294` was labelled `verifier_fn` on the theory that the verifier had checked a claim about a different candidate than the one the evaluator resolved. The run record refutes that: the draft itself reads “Marlon Brando also played in *Joy*”, and the verifier rejected exactly what the drafter asserted. Brando was not in *Joy*; De Niro was, and the evaluator had resolved him. **The verifier was correct.** The error is upstream, in the answerer, and the label was corrected to `answer_selection`. That case is also the clearest positive demonstration in the census of the verification layer doing its job, and Section 9.5 opens by pointing back to it.

9.3.3 Failing to Commit: Six Cases of Hedging on a Verified Fact

The other category was kept open for mechanisms the schema did not anticipate. All six of its cases turned out to be the same one, and it is not in the schema because nothing predicted it: *the evaluator resolves the exact gold value, the verifier returns grounded, and the drafted text hedges anyway*. No rejection is involved, so it is not a verifier error at all.

`WebQTest-689` states “Spanish Language was spoken in Spain” and then that the first language spoken in Spain could not be determined, in one answer. `WebQTrn-125` resolves `Memphis`, matching gold exactly, and the draft says the location could not be determined. `WebQTrn-1392` names both of Eleanor Roosevelt’s schools correctly and then declares the school undeterminable. `WebQTest-989` (Battle of Dunkirk), `WebQTest-1923` (*The Last Song*), and `WebQTrn-2721` (Alois Hitler) are the same shape. In every case the entity list comes back empty and the question is scored as a hedge.

The contrast with the extraction bug is exact and worth holding: there the text is right and the entity list is wrong; here the text contradicts a fact it has just stated. Six of six instances share the mechanism, which makes it a defect rather than a distribution.

9.3.4 Context-Stripping: When Decomposition Discards the Question

paraphrase_drift (9 cases) and over_decomposition are the expected decomposition failures — a rewritten sub-objective scoring worse against the target relation than the raw question text, or a redundant step resolving an already-unambiguous entity. WebQTest-1829 is the clean over-decomposition case: the plan splits “what type of religion did massachusetts have” into [‘find Massachusetts’, ‘find the religion type associated with #1’], burns 14 of 16 calls on three evaluator backtracks, and hedges, while the undecomposed run finds the same fact in three calls with no backtracks.

context_stripping (3 cases) is the mechanism this chapter would feature if it could feature only one, because it is a failure mode of decomposition that decomposition’s own design cannot see. Splitting a question into steps *removes* a qualifier that appears only in a later clause, so an early sub-objective resolves the wrong sense of an ambiguous term before the disambiguating information ever arrives.

WebQTest-1367 asks where Glastonbury, England is. The plan’s first step anchors on ‘find Glastonbury’ alone, dropping the question’s own disambiguator, and the explorer resolves toward the identically-named Connecticut town.

WebQTrn-2615 asks what instruments the author of *Fela!* plays. The first sub-objective, “find the author of *Fela!*”, is scored with no mention of instruments, so it resolves the wrong sense of an ambiguous authorship relation before the constraint that would have disambiguated it appears in step two. The undecomposed run answers all five instruments correctly, in fewer calls.

WebQTrn-2570 is the strongest specimen in the census, and its trace shows every link of the mechanism. The question — “This 33rd president was the nation’s leader during WW2” — is a single entity under two descriptions. The planner emits [‘find the 33rd president’, “find the nation’s leader during WW2”], with no #1 reference between them, modelling two descriptors of one entity as though they were a chain. Because sub-objective one is the active scoring context, the anchor World War II scores 0.128–0.133 — junk tier — while presidency-hub relations score 0.46–0.54. Decomposition actively *suppressed* the one anchor that could have discriminated. Objective one then never completes (objective_done: false at every round), so objective two never activates and the WW2 constraint is never used at any point in the search. The agent spends its depth budget wandering the presidency hub and drafts a plausible-sounding president: Woodrow Wilson, the 28th, and nowhere near the war. The undecomposed run answers Harry S. Truman correctly in three calls, because “WW2” stayed in the scoring context.

Two further observations belong to that trace. No ordinal relation appears anywhere in it, so the “33rd” constraint played no part either — consistent with the environment’s lack of ordinal encoding (Section 9.6). And the verifier returned grounded on the Wilson claim, which Section 9.5 takes up as a boundary case rather than a defect.

Three instances of one mechanism, two of them surfaced by the ablation contrast and one by

ordinary reading, is what makes context-stripping a named finding rather than an anecdote — and it is the concrete evidence behind the adaptive-gating proposal of Section 10.3.

9.4 Relation-Selection and Navigation Errors

`relation_selection` is the largest category at 65 and carries no subtypes, because the mechanism is uniform: a wrong relation was explored, or the right one was explored and abandoned. Its interest is not in the individual cases but in the fact that several of them are the *same* gap seen from different angles.

WebQTest-1730 asks where the Paraná river flows. The system answers with the rivers it flows *into*, having explored `geography.river.mouth` and `origin`; a location-containment relation was never tried, and gold is the continent.

Three separate questions across both datasets — WebQTrn-1758, WebQTest-1226, WebQTest-314 — all reach for `government.governmental_jurisdiction.government`, the organisation entity, when the question asks for a form of government and the answer lives on `government.form_of_government.countries`. One relation confusion, reproduced three times on unrelated questions.

The sharpest instance is a triplet on CWQ. WebQTrn-1770 appears three times under different perturbations — via a Beyoncé documentary, via the actor who played Etta James, via the composer of a Beyoncé track — and all three resolve Beyoncé correctly, all three never once try `people.person.children`, the single relation that answers each of them, and all three hedge. Three independent entry points into the same resolver hitting the identical dead end is the cleanest evidence in the corpus that a relation gap is *systemic* rather than an artefact of one question’s phrasing. It is a property of the scorer’s ranking over that entity’s relation set, not of the question.

9.4.1 The Evaluator Abandons a Good Relation

`premature_termination` is small (8 cases) and splits into two mechanisms that must not be merged.

Five carry the evaluator subtype and are the *same* systemic pattern: the correct relation is explored with a solid score and the evaluator backtracks away from it and never returns. WebQTest-1226 (0.451), WebQTest-517 (0.428), WebQTest-1635 (0.631), WebQTrn-710 (0.702), and WebQTest-314 (0.731) all show the identical shape across entirely unrelated domains — government form, constructed languages, sports drafts, mascots. A relation scoring 0.73 being explored and discarded is not a reasoning gap; it reads as a threshold or backtracking-policy defect, and it is one of the most directly fixable findings in the census.

The other three carry `budget` and are genuine exhaustion: WebQTest-1170 exhausts its backtrack budget wandering person-attribute relations after the direct edge returns nothing

usable; WebQTest-1736 hits two real dead ends before the plan’s second hop can run. The third, WebQTest-1630, is instructive because it belongs to two families at once: all five gold names surfaced across repeated evaluator rounds with a grounded verifier, and the run never transitioned from continue to answer before exhausting its step budget — a commitment failure of the kind Section 9.3 documents, expressed through the budget.

9.5 Verifier Rejection and Acceptance Errors

Before the errors, the correct rejections. Section 9.3.2 narrates one in full: the verifier rejected a claim the drafter had fabricated about the wrong actor, and the layer worked exactly as designed. The 52 questions on which the verifier fired unsupported are not a population of mistakes.

The two error polarities are, however, very unevenly evidenced — five cases against one — and that imbalance is not a fact about the verifier. It is a fact about the instrumentation, and it is the finding of this section.

9.5.1 Defining the Error Polarities

The verifier’s positive class is “the claim is supported”. A *false positive* is therefore a claim wrongly **accepted**, and a *false negative* a claim wrongly **rejected**. Because the verifier’s own outputs are labelled supported and unsupported, “positive” has no stable referent for a reader, and the rest of this thesis uses the plain-English terms.

9.5.2 Wrongly Rejected: Semantically Correct, Structurally Unmatched

Five cases, all sharing one mechanism and all carrying the subtype `structural_not_semantic`: the claim is true and the answer correct, but no single triple states it in the phrasing the drafter used, so a transitively-true or differently-anchored fact is rejected and the retry loses the answer.

WebQTest-1133 resolves William Henry Smith across every backtrack round; the verifier flags the claim unsupported both times and the run hedges on what is very likely the right answer. WebQTest-38 resolves *all four* gold campaign entities in one attempt; the rejection forces a retry that narrows to two names and fails anyway. WebQTest-725 resolves both gold states, California and Nevada, then loses them across re-exploration and is rejected. WebQTest-1348 resolves Houston Oilers correctly and grounded; the verifier rejects the phrasing “Peyton Manning’s dad played for Houston Oilers” because the underlying triple is stated in terms of Archie Manning, and the retry recovers only the person’s name. WebQTrn-568 resolves Mecklenburg County — the exact gold — through a sound two-hop chain from newspaper to city to county, is rejected, retries, resolves it again, and is rejected again.

The mechanism is one thing: the structural check demands a single edge asserting the drafted claim, and a transitively-true multi-hop fact does not present one. This is the same limitation Section 6.3.1 described by construction, now observed at census scale, and Section 10.3 names the semantic-level check that would address it.

9.5.3 Wrongly Accepted: One Specimen and a Boundary Case

Exactly one clean specimen exists in the whole census. WebQTrn-1597 asserts a Seth MacFarlane connection to *ThunderCats*, the verifier returns grounded, and a direct Cypher query confirms that no edge from MacFarlane to Lion-O or *ThunderCats* exists in the environment. The claim was accepted with nothing behind it.

The instructive companion is *not* a second defect. WebQTest-1620 asks when President Wilson was in office; the environment exposes inauguration events but no date literal, so the system answers with two inauguration mentions. The asserted entities exist, the edges are real, and the structural check passed *correctly*. The answer is nonetheless wrong. The same holds for the WebQTrn-2570 case of Section 9.3: Woodrow Wilson was retrieved, the claim’s endpoints genuinely connect in the graph, and the verifier was right to say so — while the answer was wrong for the question.

That contrast is the most useful point in this section, and it is the same argument Section 2.3.3 makes and Section 1.3 builds on: structural grounding is a claim about edge existence, not about answer adequacy. A verifier that checks whether a claim is *in* the graph cannot, by construction, check whether it is *the answer*. Table 8.5’s Tier-2 column is the measurement of that gap; these two cases are what it looks like in a single trajectory.

9.5.4 A Rate for One Polarity, and a Blank for the Other

The verifier fires unsupported on 16 of 400 WebQSP questions and 36 of 400 CWQ questions — 52 firings against 748 grounded outcomes. The five wrongly-rejected cases are drawn from those 52, so that polarity has a denominator: roughly one firing in ten cost a correct answer. Even that is a lower bound, since a wrongful rejection that did not change the final answer never entered the census, which reads only failures.

Wrongful acceptance has *no logged population at all*. Accepted claims are never persisted — Section 5.9 records that the verifier logs only the claims it rejects — so the wrongly-accepted cases sit undifferentiated inside the 748 grounded outcomes with nothing marking them. The single specimen surfaced only because the ablation-discordance reading happened to walk that trajectory by hand.

The two polarities are therefore reported asymmetrically and deliberately: a rate for rejection, an explicit blank for acceptance, and no average of the two. A combined “verifier error

rate” computed from these numbers would be an artefact of what the logs happen to record. Section 9.9.3 treats this as the instrumentation gap it is, and Section 10.3 gives the one-line fix that would convert the blank into a rate.

9.6 Knowledge-Graph Gaps: Literals, Temporal Qualifiers, and Ordinals

`kg_gap` is the third-largest category at 44 and CWQ’s single largest hedge category at 24. It invokes the unanswerable-in-environment class defined in Section 4.7.4, and its five subtypes cluster by mechanism.

`ordinal` (9 cases) — superlatives such as “first”, “last”, “smallest”, “earliest”. Every instance shares one shape: the relevant relation is explored, candidates come back, and the minimum-or-maximum comparison across them simply never happens, because no component performs it. WebQTest-245, on the year of the first Miss America pageant, is the clearest illustration — the system answers 1925, then 1980, then 1929 across successive rounds, genuinely different years each time, because nothing sorts and takes the extreme.

`numeric_literal` (16 cases, the largest subtype) — internal identifiers such as `tvrage_id`, `thetvdb_id`, ISO alpha-3 codes, and Freebase’s own located identifiers, which CWQ’s templates invoke constantly and which no relation in the explored schema exposes. Several of these hedges are unusually honest: WebQTrn-849, asking which country bordering Germany has ISO code CHE, names Switzerland in the answer text and then states explicitly that the code itself could not be confirmed from the available facts — a correct inference, correctly caveated, rather than a false assertion.

`date_literal` (4) and `temporal_qualifier` (4) — the environment exposes inauguration and election *events* but not the date values themselves, so a question about when an office was held returns event mentions (WebQTest-1620), and a question about who was president in a given year substitutes `government.election.winner` for a term-range check it cannot perform. WebQTest-749 answers “George H. W. Bush” for 1988: the winner of that November’s election, who took office the following January.

`data_error` (6) — genuine coverage holes rather than expressiveness limits. WebQTrn-261 returns four consecutive rounds of `explored=[]` (max_score 0.0) because the image reference the question names resolves to nothing in the graph at all.

The distinction between the first four subtypes and the last matters for what follows from them. `data_error` is a gap in this particular import; `ordinal`, `numeric_literal`, `date_literal`, and `temporal_qualifier` are gaps in what a triple-and-traversal architecture can express at all, and no larger graph fixes them. They need an operator, which is why Section 10.3 lists literal and ordinal support alongside the architectural items rather than among the repairs.

9.7 The Echo Attractor: Answering with the Topic or an Intermediate Entity

echo names a mechanism that recurs independently of the two mechanical categories above: the system resolves a real, well-grounded entity that sits *near* the correct answer in the graph’s structure, and the verifier passes it because it genuinely is grounded. The fact is true. It is not what was asked.

Three shapes appear, and their boundary is not always crisp. *topic* (6 cases) is confusion between entities sharing a name or a tight cluster: WebQTest-1707 asks who plays Lex Luthor on *Smallville* and answers “John Glover plays Lionel Luthor” — a different character in the same fictional cluster, with Lionel, Alexander, and Lucas Luthor variants all surfacing in the trace before the run gives up. *granularity* (7 cases) is the right entity family at the wrong level: WebQTest-86 answers “Kingdom of Denmark” where gold is “Denmark” — a genuinely distinct node, the formal sovereign-state entity rather than the common-name one. WebQTrn-1405 asks which school founded in 1636 President Kennedy attended, has Harvard College in its very first candidate list, and narrows to Harvard University instead — whose founding-date claim the verifier then correctly rejects. *intermediate* — stopping at a hop along the way rather than continuing to the target — has no rows of its own in this census, because the reading pass filed those cases under granularity; the two are best treated as one family.

The throughline is what makes this a named mechanism rather than a residue category. None of these are cases where the system knew nothing. Every one resolves a real, verifiably-true fact through a real edge. The failure is entirely in *which* true fact gets surfaced — architecturally different from picking a wrong edge and from there being no edge at all. The right edge exists, is used, and lands one node away.

That is also why the echo attractor is invisible to any evaluation treating systems as independent. Section 9.8 shows all five systems of the main matrix falling into it together, on the same questions.

9.8 Benchmark Defects: Gold Noise and Ambiguous Questions

Two numbers must be kept apart. The consensus pass of Section 7.7 *flagged* 58 WebQSP and 47 CWQ questions where at least three of the five systems agreed on the same non-gold answer. Adjudication confirmed 22 and 19 as genuine defects. The gap between those figures is not measurement noise — it is the echo attractor of Section 9.7, operating across systems.

Read what the flagged consensus answers have in common. Five systems answer “rick scott” where gold lists his professions, “thor heyerdahl” where gold lists his occupations, “amelia

earhart” where gold is Pilot and Writer — the topic entity returned as the answer. Five answer “belgium” where gold is the containing region, “dominican republic” where gold is Greater Antilles, “san francisco giants” where gold is the 2014 World Series — the composition’s intermediate rather than its final hop. Five answer “maryland” for counties, “nevada” for Las Vegas, “united arab emirates” for Dubai — one level too coarse.

Cross-system consensus against gold is dominated not by label errors but by a shared attractor, and that is a finding about evaluation methodology as much as about these systems. A policy of rescoring whenever three systems agree — which is a natural thing to want — would have silently converted this system’s most characteristic failure mode into apparent correctness. The manual adjudication step is what stands between the two, and it is the reason the pass is reported with both numbers rather than one.

9.8.1 The Confirmed Defects

What adjudication did confirm is nonetheless substantial: 59 questions across the two datasets where the benchmark, not the system, needed correcting. Three disjoint counts make that total — 41 excluded before the census read anything (22 + 19), 17 more still visible in the census as gold_noise or ambiguous_question rows (3 WebQSP + 14 CWQ), and one that began as a census finding and was promoted to a formal exclusion mid-project.

Two specimens carry the point. WebQTest-958 — “what are some famous people from el salvador” — has 116 gold entities, including raw Freebase machine identifiers leaked into the answer strings: a “list some famous *X*” template ballooning gold past anything a finite answer could match under any metric. WebQTest-55 — “Who had a concert named Crazy Love Tour?” — asks for a person and receives gold listing the artist’s professions, the annotation query having traversed one relation too far past Michael Bublé.

The Vicksburg pair is worth reading side by side, because two sibling questions share a gold string and receive opposite diagnoses. WebQTest-1797.5a1c66f asks who was president during the battle of Vicksburg, with gold Ulysses S. Grant — who took office six years afterwards; the president in 1863 was Lincoln. The full system answers that the claim could not be verified and its verifier flags it unsupported; the no-planner ablation asserts Grant confidently, with zero backtracks, and is scored a *hit* purely for restating a gold value that answers a different question. WebQTest-1797.dece4dd asks which government-position holder fought at Vicksburg — coherent as asked, since Grant was a serving general — and the full system genuinely fails, answering with the Confederate commander John C. Pemberton, a real combatant who held no government position. The qualifier is present in the sub-objective text and never filters the candidate frontier.

Same battle, same gold string: one question unanswerable and one a real composite_claim failure. The pipeline that hedged on the first was more epistemically correct than the one that

“succeeded”.

One process observation belongs on the record. WebQTrn-64 was adjudicated `gold.ok` by the consensus pass — several systems converging on “Tupac Shakur” read as a topic echo — and the opposite way by the census reading of the same evidence: the question literally asks for an actor’s name, and `gold` gives a character name from a different role in the same film. The second reading held, and the row was promoted to a formal exclusion. Two independent adjudication passes can legitimately disagree, and keeping a `gold.noise` category open in the taxonomy is what let that be resolved rather than settled by whichever ran first.

9.8.2 Where Bad Gold Comes From: Flattened Multi-Valued Relations

The San Antonio case shows the provenance of one defect precisely enough to generalise. WebQTest-634 asks which county contains the place where the San Antonio City Council sits. Four systems, spanning parametric and graph-based knowledge, answer Bexar County; `gold` says Comal. Adjudication recorded the verdict `gold.wrong`: San Antonio and its city council are in Bexar. But the census reading of the same case recorded something more specific, and more useful — `kg_gap`, with the note that this is a containment ambiguity in the source data, arising from annexed territory or from Freebase carrying multiple imprecise containment edges.

That is neither a pure label error nor a pure graph error. It is a **multi-valued containment relation flattened to a single value**, with the annotation pipeline then selecting the marginal one. A city can lie in more than one county; a benchmark whose `gold` field holds one answer per relation must choose; and the choice is made by a SPARQL query with no notion of which value is the principal one. Framing it this way explains why the consensus diagnostic caught it at all — the systems and the world agree on the principal value while the label holds the marginal one — and it predicts where else such defects will cluster: any relation that is genuinely one-to-many but stored as though it were one-to-one.

9.9 Discussion

9.9.1 What Agency Buys, and What It Costs

Agency buys the deep tail. On CWQ the accuracy of the agentic system *rises* across hop strata while every static system decays, and it buys that at roughly an order of magnitude in tokens over one-shot retrieval. Against the other agentic system it buys the same accuracy for half the language-model calls, and never exhausts a budget that clips the baseline on 44% of multi-hop questions — the difference between a guided search and a blind one, priced.

What it costs, beyond tokens, is a longer failure surface. Every mechanism in this chapter except `kg_gap` and the benchmark defects is a failure the static baselines cannot have, because they have

no plan to mis-decompose, no frontier to abandon, and no draft to check. `decomposition_error` at 38 cases is the clearest instance: a component added to help contributed a failure family of its own, and Section 8.7 shows it is a net negative on the shallower benchmark. The honest summary is that agency converts a retrieval problem into a control problem, and control problems fail in more ways.

9.9.2 When Verification Helps and When It Over-Hedges

The verification layer’s contribution is precision and auditability, and its cost is recall. Both directions are visible in this census at case level.

It helps when a draft asserts something the traversal never supported: the De Niro case of Section 9.3.2, where a fabricated actor was rejected before it could be emitted, and the Vicksburg-president case of Section 9.8, where declining to assert was the epistemically correct act and the metric scored it as a loss.

It over-hedges when a true claim has no single edge stating it in the drafted phrasing — the five wrongly-rejected cases of Section 9.5, three of which had already resolved the exact gold answer before the rejection forced a retry that lost it. That is the same trade the development set measured at Section 8.1, where the layer cost one correct answer out of 80 and removed none that were wrong, and the same one the ablation of Section 8.7 could not detect at $n \approx 200$.

The scope of the claim this evidence supports is therefore narrow and should be stated narrowly. Verification does not make this system more accurate. It makes the answers it does give attributable, at a small and measurable cost in recall, and it provides no protection at all against the failure mode this chapter finds most often — an answer that is grounded, connected, true, and not what was asked.

9.9.3 Threats to Validity

The instrumentation gap is the most serious. The verifier logs only the claims it rejects, never the ones it accepts with their matching triples. Wrongful acceptance is therefore not self-diagnosing: the single specimen in this census surfaced by manual trace reconstruction, and there is no way to know how many others sit inside the 748 grounded outcomes. One polarity is reported at census scale and the other anecdotally, and that asymmetry is a property of the logging decision rather than of the verifier. Section 10.3 gives the fix.

The judge fell short of its pre-registered bar. Cohen’s $\kappa = 0.6995$ against a threshold of 0.7 (Section 7.6). Agreement is substantial by conventional reading, and the miss is 0.0005, but the Tier-2 numbers are reported as corroborating rather than decisive, and no conclusion here rests on them alone. The judge also runs on the same backbone as the systems it judges.

Topic entities are given, not linked. Both graph-navigating systems seed from the dataset’s annotated topic entities. This isolates navigation from entity linking, which is the intent, but it means the reported accuracies assume a linking step that a deployed system would have to perform and that would introduce errors of its own.

Ablations are underpowered. Three of four conditions detect nothing at $n \approx 200$, and “no detectable effect” is not “no effect” (Section 8.7).

One relabelling was made after seeing the result it affects. A development question originally classified as mediator-heavy was reclassified as unanswerable-in-environment after its outcome was known; Chapter 6 flags it in place, and the aggregates were not rescored.

The reported accuracy has an unmeasured floor. The extraction bug of Section 9.3 converts correct reasoning into a wrong entity list on a specific question shape, so the measured accuracy on that shape is a lower bound rather than an estimate.

Benchmark defects were adjudicated by the author. The exclusion set of Section 7.7 rests on single-annotator judgements, made with the consensus signal visible. The scope limits are recorded there; the accuracy tables are not rescored, and the defects run in both directions.

Chapter 10

Conclusion

10.1 Summary of Contributions

This thesis asked what an agent gains by navigating a knowledge graph rather than retrieving from it, and what a claim-level check against the traversed subgraph adds on top. The answers are not uniform, and the value of the work lies partly in where they diverge.

Chapter 1 located the problem: hallucination in knowledge-grounded question answering is not merely a matter of asserting false things but of asserting things with no basis in the knowledge the system was given, and the standard evaluation convention — Hits@1 over a multi-valued gold set — cannot see the difference. Chapter 2 fixed the vocabulary the technical chapters assume, including the operational definition of ungroundedness this thesis measures against, and withdrew the proposal’s characterisation of the navigation strategy as MCTS-inspired: what the method is, is guided best-first search with a bounded beam and backtracking, and it is described that way throughout.

Chapter 3 positioned the work against the agentic KGQA literature and identified the attribution gap it addresses: published systems bundle decomposition, search, and verification into single reported numbers, so the field has no evidence about which component earns its cost.

Chapter 4 built the knowledge environment and, more importantly, *certified* it: 2.59 million entities and 8.31 million triples with a measured answer-reachability ceiling of 97.3% and 99.7% over the full test splits; a three-stage entity resolver; and an explicit inventory of what it cannot express — no date literals, no numeric literals, no ordinals — which Chapter 9 later shows accounts for the third-largest failure category in the census. Certifying the environment before measuring on it is what allows a failure to be attributed to the system rather than to the substrate.

Chapter 5 specified the framework: a state machine over a constrained, deterministic graph tool API in which the program orchestrates and the language model only decides, with a hybrid scoring function blending embedding similarity against a batched language-model judgement, a signal-maximum backtracking trigger that fires only when every available signal is weak, and an

explicit budget meter that makes cost a first-class, logged quantity.

Chapter 6 specified the Structural Verification Layer, and reported its behaviour honestly enough to bound its own claims: a draft is decomposed into atomic claims, each checked first against traversed adjacency and then, if necessary, by batched entailment, with unsupported claims triggering targeted re-exploration or deterministic removal from the answer. The chapter also records that the repair path never executed on the development set, that ten of eighty answers were certified with no claims at all, and that entities appearing in no claim are never checked — the layer’s guarantees stated as they are rather than as designed.

Chapter 7 fixed the experimental design in advance and reports the decisions that went the other way: a backbone selected, reversed, and reselected under a pre-registered tiebreaker after the first determinism estimate proved to be noise; a test-set build that read the wrong split and was caught by a contamination guard before any system ran; and a judge-validation threshold missed by 0.0005 and reported as missed.

Chapter 8 reported the measurements. AGR leads every cell of the main matrix and every one of the eight paired comparisons rejects, including both against a reimplementing of the agentic baseline running on the same graph with the same model under the same budget. On ComplexWebQuestions its accuracy *rises* with hop count while every other system decays — the strongest available form of the multi-hop claim. It runs at half the language-model calls of the agentic baseline and never exhausts a budget that clips that baseline on 44% of multi-hop questions. Structural hallucination is eliminated: zero ungrounded assertions across 1,709 asserted entities, against 27.1% for parametric answering. And three of four ablations detect nothing, which is reported at the same length as the one that does.

Chapter 9 read all 259 remaining failures and named each mechanism. The findings that generalise beyond this system are three. *Context-stripping*: decomposing a question can discard a qualifier that appears only in a later clause, so an early sub-objective resolves the wrong sense of an ambiguous term before the disambiguating information arrives — and in the strongest specimen, decomposition actively suppressed the one anchor that could have discriminated, driving its score into the junk tier. *The echo attractor*: the most common failure of a graph-navigating system is not to invent an entity but to surface a real, well-grounded, verifiably-true entity that sits one node away from the answer, which the verifier passes because it genuinely is grounded. And *shared attractors break consensus-based evaluation*: five independent systems converge on the same wrong answer often enough that a policy of rescoring on majority agreement would have converted this failure mode into apparent correctness.

Four contributions follow from that body of work.

The framework and its verification layer. An agentic KGQA system that returns its answer paired with the triples supporting it, built so that the supporting evidence is a byproduct of the control flow rather than a post-hoc reconstruction.

A component-level ablation. Four single-deviation conditions with paired significance testing,

closing the attribution gap of Chapter 3 for this architecture — and finding that only one of the four components has a detectable accuracy effect.

Stratum-dependent decomposition. The literature treats decomposition as straightforwardly beneficial. It is not: removing the planner *improves* WebQSP accuracy by 0.083 F1 at $p = 0.006$ while cutting tokens by 30%, and trends toward harming CWQ. The asymmetry, not the pooled average, is the result, and it says that decomposition should be gated on question structure rather than applied unconditionally.

Quantified benchmark defect rates. Fifty-nine questions across the two test samples where the benchmark, not the system, needed correcting — with the two evidence signatures that distinguish a world-fact error from an annotation-pipeline error, and the provenance argument that explains where one class of them comes from.

10.2 Limitations

The limitations that most constrain what may be concluded are, in order of severity:

Wrongful acceptance is unmeasured. The verifier persists only the claims it rejects. One polarity of its error is therefore reported at census scale and the other from a single specimen that surfaced by manual trace reconstruction, and no rate exists for the polarity that matters most to a hallucination claim. This is an instrumentation decision constraining a scientific conclusion, and it is the first item in Section 10.3.

Three of four ablations are underpowered. At $n \approx 200$, a component that changes five questions in a hundred cannot be resolved. “No detectable effect at this sample size” is what the data supports, and it is not the same statement as “no effect”.

Structural grounding is not answer adequacy. Both navigating systems reach zero structural hallucination, so that property belongs to the navigation paradigm rather than to this thesis’s verification layer, and the semantic-support metric that would separate them is validated at $\kappa = 0.6995$ on samples of 60 — underpowered for the 5-point gap it measures. The strongest evidence for the layer is therefore the precision column of the main matrix and the case-level analyses, not a groundedness differential against the agentic baseline.

The evaluation is single-environment, single-backbone, and single-annotator. One knowledge graph built from one distribution, one frozen backbone at $\approx 75\%$ trajectory stability, two benchmarks sharing a lineage, and a failure census and benchmark-defect adjudication performed by the author. Each of those is a defensible scoping decision and each bounds generality.

Entity linking is assumed. Topic entities are given by the datasets, so the reported accuracies presume a linking step a deployed system would have to perform.

One measured defect depresses the reported accuracy. The entity-extraction bug of Section 9.3

converts complete, verifier-certified reasoning into an empty or wrong entity list on a specific question shape. The headline numbers are a floor for that shape.

10.3 Future Work

The census makes it possible to separate proposals that are backed by a counted, repeated pattern from proposals that are backed by an argument. Both are listed; they are not the same kind of claim.

10.3.1 Repairs the Census Earned

Three defects each recur with a uniform mechanism across unrelated questions, and each is small.

Entity extraction (nine instances): the step converting drafted answer text into the scored entity list defaults to the sentence's grammatical subject rather than its predicate values. Fixing it requires the claim decomposer to be told which argument of the drafted sentence is the answer — an instruction, not an architecture.

Evaluator abandonment (five instances): relations scoring as high as 0.73 are explored and then backtracked away from, never revisited, across four unrelated domains. This reads as a threshold or backtracking-policy defect and would be diagnosed by logging the evaluator's decision against the score of the relation it discards.

Drafting commitment (six instances, all six of the other category): when the evaluator has resolved the exact gold value and the verifier has returned grounded, the drafter should commit to it rather than stating the fact and declaring it undeterminable in the same sentence.

10.3.2 Logging Accepted Claims to Make Wrongful Acceptance Measurable

Persist each accepted claim together with the triples that matched it. This converts the wrongly-accepted class from an anecdote into a rate with a denominator, makes grounded-but-wrong answers self-diagnosing from the logs rather than by manual trace reconstruction, and would let the entity-level and claim-level guarantees of Chapter 6 be audited rather than argued. It is the highest-value change in this list relative to its cost, and it is a logging change.

10.3.3 Adaptive Decomposition Gating

The planner ablation and the context-stripping cases together specify the fix rather than merely motivating it. A decomposition step should be gated on two checks: does the rewritten sub-

objective preserve the question’s actual interrogative — the Phoenician case lost the word “where” entirely — and does the step resolve an entity that was already unambiguous, which is what over-decomposition does. A third check falls out of the strongest specimen: sub-objectives that carry no reference to each other are descriptors of one entity rather than a chain, and should not be sequenced as one.

10.3.4 Semantic-Level Verification

All five wrongly-rejected cases share one mechanism: the structural check requires a single edge asserting the drafted claim, and a transitively-true multi-hop fact does not present one. A check that accepts a bounded relation chain expressing the claim — rather than requiring one literal triple — would recover those answers. The Tier-2 judge of Section 7.5.3 is a prototype of exactly this judgement, which suggests folding a bounded form of it into the loop; the κ of 0.6995 is also a caution about how reliable such a check would be.

10.3.5 A Set-Intersection Operator for Compound Questions

`composite_claim` is 47 cases and CWQ’s largest category. Its `no_set_intersection` subtype names the gap precisely: the architecture has no AND-join primitive, so a “find the overlap” sub-objective degrades into another single relation-scoring pass over an incomplete first-hop set. An explicit operator that resolves each clause of a compound question into a candidate set and intersects them is a well-specified addition to the tool API rather than a change to the agent.

10.3.6 Ordinal and Literal-Value Support

`kg_gap` is 44 cases and CWQ’s single largest hedge category. Two of its subtypes need an operator rather than a bigger graph: a minimum-or-maximum-by-value operation for superlatives, and a literal-extraction path for numeric identifiers, ISO codes, and dates that the environment exposes as event or entity nodes rather than as plain values. Neither exists in the current tool API, and every instance in the census fails for that reason alone.

10.3.7 Path Fidelity Against Gold SPARQL Relation Chains

The third evaluation criterion of the approved proposal, deferred for the reason given in Section 7.1.2: it needs the benchmarks’ gold SPARQL relation chains, which the distribution used here does not carry. Reconstructing them from the upstream releases and aligning them to this environment’s relation vocabulary would measure whether the system reaches the right answer *by the right path* — a question this thesis’s groundedness metrics can only approximate,

and one that bears directly on the echo attractor, where a right-looking answer is reached by a path one hop short of the correct one.

10.3.8 Beyond This Environment

Four directions extend the work rather than repairing it. *LLM-constructed knowledge graphs* as a controlled comparison would separate what the framework contributes from what a curated environment contributes, by holding the agent fixed while varying the substrate’s provenance and noise profile. *Rollout-based value estimation* would supply what Section 2.5.3 explicitly disclaims: the current scorer evaluates each candidate once and never revises it in light of what is found later, and genuine tree search over this same tool API is a well-defined next system rather than a rebranding of this one. *Temporal and multi-source environments* address the failure family that recurs throughout Chapter 9 — questions with a time qualifier the graph cannot express — and raise the question of what a claim check means when sources disagree. And *transfer to high-stakes domains* is where the auditability property, rather than the accuracy, is the point: an answer paired with the triples supporting it is worth more in a setting where a reader must be able to check it than in one where the reader only wants the answer.

10.4 Closing Remark

The result this work would most want carried forward is not the accuracy margin. It is the shape of the gap that remains. A system that traverses a real graph and copies its answers out of real triples can be made to assert nothing ungrounded — that problem is solved, and it is solved by the navigation paradigm rather than by any checking layer built on top of it. What remains, and what this thesis’s census found more often than any other mechanism, is an answer that is grounded, connected, and true, and is not the answer to the question that was asked. Fact verification against a knowledge graph turns out to be the easier half of the problem; *relevance* verification is the half still open.

References

- [1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM Transactions on Information Systems*, vol. 43, pp. 1–55, 2025.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [3] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, and J. X. Yu, “A survey of graph meets large language model: Progress and future directions,” *arXiv preprint arXiv:2311.12399*, 2023.
- [4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
- [5] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Advances in Neural Information Processing Systems*, vol. 37, pp. 59532–59569, 2024.
- [6] D. Li, Y. Niu, Y. Ai, X. Zou, B. Qi, and J. Liu, “T-GRAG: A dynamic GraphRAG framework for resolving temporal conflicts and redundancy in knowledge retrieval,” in *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 11880–11889, 2025.
- [7] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, and J. Guo, “Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph,” in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.07697.
- [8] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, and H. Xiong, “Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024. arXiv:2410.23875.

- [9] L. Luo, Y.-F. Li, G. Haffari, and S. Pan, “Reasoning on graphs: Faithful and interpretable large language model reasoning,” in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01061.
- [10] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, and J.-R. Wen, “KG-Agent: An efficient autonomous agent framework for complex reasoning over knowledge graph,” *arXiv preprint arXiv:2402.11163*, 2024.
- [11] Y. Xu, S. He, J. Chen, Z. Wang, Y. Song, H. Tong, K. Liu, and J. Zhao, “Generate-on-graph: Treat LLM as both agent and KG for incomplete knowledge graph question answering,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. arXiv:2404.14741.
- [12] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, and J.-R. Wen, “StructGPT: A general framework for large language models to reason over structured data,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.09645.
- [13] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, “Unifying large language models and knowledge graphs: A roadmap,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.
- [14] C. Ma, Y. Chen, T. Wu, A. Khan, and H. Wang, “Unifying large language models and knowledge graphs for question answering: Recent advances and opportunities,” in *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*, pp. 1174–1177, 2025.
- [15] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, “Chain-of-verification reduces hallucination in large language models,” in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3563–3578, 2024.
- [16] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [17] Y. Hu, W. Xuan, Q. Zhou, Z. Li, Y. Li, J. Hu, and F. Fang, “A self-correcting agentic graph RAG for clinical decision support in hepatology,” *Frontiers in Medicine*, vol. 12, p. 1716327, 2025.
- [18] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, “Freebase: A collaboratively created graph database for structuring human knowledge,” in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pp. 1247–1250, 2008.

- [19] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh, “The value of semantic parse labeling for knowledge base question answering,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 201–206, 2016.
- [20] A. Talmor and J. Berant, “The web as a knowledge-base for answering complex questions,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 641–651, 2018.
- [21] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*. O’Reilly Media, 2nd ed., 2015.
- [22] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor, “Cypher: An evolving query language for property graphs,” in *Proceedings of the 2018 International Conference on Management of Data*, pp. 1433–1445, 2018.
- [23] J. Berant, A. Chou, R. Frostig, and P. Liang, “Semantic parsing on Freebase from question-answer pairs,” in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pp. 1533–1544, 2013.
- [24] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
- [26] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [28] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781, 2020.
- [29] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992, 2019.

- [30] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [31] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.
- [32] L. Kocsis and C. Szepesvári, “Bandit based monte-carlo planning,” in *Proceedings of the 17th European Conference on Machine Learning*, pp. 282–293, 2006.
- [33] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, “A survey of monte carlo tree search methods,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.
- [34] H. Luo, H. E, Y. Guo, Q. Lin, X. Wu, X. Mu, W. Liu, M. Song, Y. Zhu, and L. A. Tuan, “KBQA-o1: Agentic knowledge base question answering with monte carlo tree search,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2501.18922.
- [35] T. Shen, J. Wang, X. Zhang, and E. Cambria, “Reasoning with trees: Faithful question answering over knowledge graph,” in *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 3138–3157, 2025.
- [36] B. Efron, “Bootstrap methods: Another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [37] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [38] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [39] J. R. Landis and G. G. Koch, “The measurement of observer agreement for categorical data,” *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
- [40] X. Tan, X. Wang, Q. Liu, X. Xu, X. Yuan, and W. Zhang, “Paths-over-graph: Knowledge graph empowered large language model reasoning,” *arXiv preprint arXiv:2410.14211*, 2024.
- [41] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” in *International Conference on Learning Representations (ICLR)*, 2024.

- [42] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [43] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, “FactScore: Fine-grained atomic evaluation of factual precision in long form text generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12076–12100, 2023.
- [44] F. F. Bayat, L. Zhang, S. Munir, and L. Wang, “FactBench: A dynamic benchmark for in-the-wild language model factuality evaluation,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33090–33110, 2025.
- [45] Y. Gu, S. Kase, M. Vanni, B. Sadler, P. Liang, X. Yan, and Y. Su, “Beyond I.I.D.: Three levels of generalization for question answering on knowledge bases,” in *Proceedings of the Web Conference 2021*, pp. 3477–3488, 2021.

Index

best-first search, 12

hallucination, 1
1, 10

in-context learning, 10

KGQA, 9

knowledge graph, 3

mediator node, 9

MID, 8

multi-hop question, 2

property graph, 9

ReAct, 10

triple, 8

Appendix A

Prompt Templates

Seven prompts constitute the whole of AGR’s interface to the language model. Two more serve the baselines. All nine are reproduced here verbatim from the source that sends them, in the order a question encounters them. Braces written `{like this}` are Python format fields; doubled braces `{{ . . . }}` are literal braces in the emitted text.

This appendix carries a checker rather than a promise. `scripts/check_appendix_prompts.py` extracts each constant from the source and asserts that it still appears here. Nothing in the LaTeX build could detect the divergence on its own, and the reproducibility argument of Section 7.10 depends on the two not diverging: the response cache keys on prompt text, so a prompt edited in one place and not the other invalidates every cached run as well as this appendix.

Two properties of this set matter more than any individual wording. First, no prompt contains a configuration value. Neither α nor τ nor any budget number appears in any template, which is what allows one cached response to be shared across every condition of the α – τ sweep (Section 8.1). Second, no prompt asks the model to respect a budget. Budgets are counters in code (Section 5.8); the model is never asked to stop, only prevented from continuing.

A.1 Planner

Sent once per question, unless the planner is ablated. Section 5.4 describes the four-archetype few-shot design and the topic-mention restriction; this is the template that implements both. The instruction to use as few sub-objectives as possible is the provision Section 8.8 finds to have been insufficient on the shallower benchmark.

```
You decompose questions into a minimal ordered chain of sub-objectives for
↪
navigating a knowledge graph. Each sub-objective must be answerable by
following one relation (possibly through an event node). Use #N to
↪ reference
the result of sub-objective N. Use as FEW sub-objectives as possible.
```

For `topic_mentions`, extract ONLY specific named entities that appear literally in the question (people, places, organizations, works, products) $\hookrightarrow$.

Do NOT extract generic type words such as "celebrity", "university", "country", "senator", "airport", or "currency" -- these describe what $\hookrightarrow$ kind of answer is wanted, not where to start searching.

Example 1 (single hop -- do NOT decompose):

Question: "who is the president of France?"

```
{"sub_objectives": ["find the president of France"],
  "topic_mentions": ["France"]}
```

Example 2 (composition):

Question: "who directed the film that won Best Picture in 1998?"

```
{"sub_objectives": ["find the film that won Best Picture in 1998",
                    "find the director of #1"],
  "topic_mentions": ["Best Picture"]}
```

Example 3 (conjunction):

Question: "which countries border France and have Spanish as official $\hookrightarrow$ language?"

```
{"sub_objectives": ["find countries that border France",
                    "find which of #1 have Spanish as an official
 $\hookrightarrow$  language"],
  "topic_mentions": ["France", "Spanish"]}
```

Example 4 (superlative):

Question: "what is the largest city in the country where the Nile ends?"

```
{"sub_objectives": ["find the country where the Nile ends",
                    "find the largest city in #1"],
  "topic_mentions": ["Nile"]}
```

Question: "{question}"

The plan this returns is validated rather than trusted. Four checks run before it is accepted, and any failure installs the single-objective fallback described in Section 5.4: the sub-objective list must be non-empty, it must hold no more than four entries, every #N reference must point strictly backwards, and at least one topic mention must be present.

A.2 Relation Scorer

Sent once per expansion step, scoring all candidate edges across all anchors in a single call. This is the prompt that carries the property discussed in Section 5.5.2: it mentions neither α nor the anchor ordering, so its response is reusable across every sweep condition.

```
Sub-objective: {objective}

Candidate knowledge-graph edges (anchor entity --relation-->):
{candidates}

For EACH candidate, rate from 0.0 to 1.0 how likely following that edge
↪ from
that anchor leads toward completing the sub-objective. 1.0 = directly
answers it, 0.0 = irrelevant. Judge the RELATION MEANING in context of
↪ the
anchor, not surface word overlap.
```

The final instruction exists because of the failure it names. Judging surface word overlap is what an embedding does, and the blend of Section 5.5.2 already supplies that signal; the model is asked for the part the embedding cannot give.

A.3 Evaluator

Sent once per navigation cycle. The fact window is the thirty triples ranked most relevant to the current sub-objective, not the thirty most recent — the change forced by the failure recorded in Table 5.4.

```
Current sub-objective: {objective}
Recently retrieved facts:
{facts}
Remaining budget: {d_left} more expansions, {b_left} more backtracks.

Decide:
- "answer" if the facts complete the sub-objective (list satisfying
↪ entities
in "resolved", exactly as named in the facts),
- "continue" if this direction is promising but incomplete,
- "backtrack" if this direction is a dead end.
```

The remaining budget is stated for context, not for enforcement. A model that ignores it changes nothing: the counters in Section 5.8 terminate the run regardless of what the evaluator decides.

A.4 Drafter and Claim Decomposer

One call produces the draft answer, its asserted entity list, and the atomic claims the verification layer will check. Section 6.2 explains why these are one call rather than three: a draft and a claim set produced separately can disagree about what was asserted, and the layer would then verify something the reader never sees.

Four provisions here are load-bearing, and Section 6.2 gives the case that forced each. Copying entity names exactly is what keeps the graph's spelling — University Yale rather than Yale University — in the answer. The distinction between entities the draft *asserts* and entities it merely mentions is what stops a hedge from being scored as an assertion. The anti-vacuity instruction is what stops the model answering “a university” to a *which university* question. And requiring a hedge to carry zero claims is what makes an abstention verifiable rather than merely unverified.

```

Question: {question}
Facts retrieved from the knowledge graph:
{facts}
Entities identified as promising during exploration:
{candidates}

1. Draft an answer using ONLY the facts and candidates above. Use entity
   names exactly as written. If insufficient, say what could not be
   determined.
2. List the answer entities themselves in "answer_entities", copied
   ↪ EXACTLY
   as they are named in the facts above -- these are the entities your
   ↪ draft
   asserts as the answer, not every entity it mentions. If your draft is
   ↪ a
   hedge ("could not be determined"), "answer_entities" MUST be [].
   IMPORTANT: "answer_entities" must be specific entities of the kind the
   question asks for (a person for "who", a place for "where", etc.).
   Never restate the question or name entity types/categories as the
   answer. If you cannot name specific answer entities from the facts,
   your draft must be a hedge and "answer_entities" must be [].
3. Decompose your draft into atomic factual claims, each as
   {"h": entity name, "r": short relation phrase, "t": entity name}}.
   Only include claims your draft actually asserts. A hedged draft has
   ↪ zero
   claims.

```

A.5 Entailment Check

Sent only for claims that the two structural routes of Section 6.3.1 could not settle. It is the semantic fallback, and the definition of *unsupported* is given explicitly because the default reading of the word is too permissive: a model asked merely whether a claim is supported will accept anything it believes to be true.

```
Retrieved knowledge-graph triples:
{facts}

For each claim below, answer whether it is directly supported by the
↪ triples
above (true) or not (false). Unsupported includes: plausible but absent,
contradicted, or only inferable via outside knowledge.
Claims:
{claims}
```

The three enumerated cases are the three ways a fluent model fails this task. *Plausible but absent* is the hallucination the layer exists to catch. *Contradicted* is rarer and easier. *Only inferable via outside knowledge* is the one that matters most for the thesis’s claim, because a model drawing on parametric memory here would certify exactly the content the graph was supposed to constrain.

A.6 Rewriter

Sent when verification rejects at least one claim but at least one entity survives. It is the mechanism by which an unsupported claim is removed rather than merely flagged.

```
Question: {question}
Draft answer: {draft}
The following claims in the draft are NOT supported by the knowledge
↪ graph:
{unsupported}
The following entities remain verified and MUST be kept in the answer,
named exactly as written: {kept}

Rewrite the draft: keep the verified entities and their supported claims,
and remove or explicitly hedge the unsupported claims. If the kept list
↪ is
empty, state that the answer could not be verified against the knowledge
graph. Respond with prose only.
```

The kept list is computed in code by the deterministic filter of Section 5.7, not proposed by the model. The rewrite call may reword the answer but cannot change which entities it asserts, and Section 6.7 reads the case in which that separation decided whether a question was scored

correct.

A.7 Answerer

Sent on the path where verification is skipped, which is the draft-only ablation of Section 8.7, and on the give-up path where the budget drained before a draft existed.

```

Question: {question}
Facts retrieved from the knowledge graph:
{facts}

Entities identified as promising during exploration:
{candidates}

Answer using ONLY these facts and candidates. Give the answer entity name
↪ (s)
exactly as they appear above. If the information is insufficient, state
↪ what
could not be determined.
Also list in "answer_entities" the entities your answer
asserts (exactly as named above); [] if it asserts nothing.
```

Comparing this against the drafter of Section A.4 shows what the draft-only ablation removes. The anti-vacuity provision is absent here, and Section 8.7 reports the hedge-rate consequence.

A.8 Baseline Prompts

The baselines share the backbone, the budget, and the answer-entity contract of the full system, so that the comparison isolates retrieval rather than output format. All three emit `{"answer": "...", "answer_entities": ["..."]}`.

No-retrieval

The parametric control. It receives the question and nothing else, which is what makes it a measurement of memorisation rather than of retrieval (Section 7.4).

```

Answer this question with the name(s) of the answer entity or
entities. Be concise. If you do not know, say so and return [].

Question: {question}
```

Vector-RAG and GraphRAG

Both use the template below; GraphRAG imports it from the Vector-RAG module precisely so that the two differ only in how `{facts}` was retrieved. Section 7.4 records the consequence noted in Section 7.4.5: this prompt is more abstinent than the full system’s drafter, and the resulting hedge rates are a property of the prompt as much as of the retrieval.

```
Facts (retrieved from a knowledge graph):  
{facts}  
  
Question: {question}  
Answer using ONLY the facts above. Name entities exactly as written in  
↪ the  
facts. If the facts are insufficient, say so and return [].
```

Appendix B

Tool API Specification and Cypher Templates

Section 5.2 argues that the model must never author a query. This appendix gives the five operations that follow from that decision, with the Cypher each one executes. Every query below is a fixed template. The model supplies values for the \$-parameters and nothing else, so a malformed model output produces a failed lookup rather than a syntax error, and the error taxonomy of Chapter 9 never has to separate reasoning failures from query-authoring failures. Two limits appear throughout. `max_relations`, set to 300, caps the relation list returned for an entity; `max_fanout`, set to 200, caps the neighbours returned for one edge. Table 5.4 records why the first was raised from its original value of 80.

Every invocation is appended to a JSONL log with its arguments, its result, and its wall-clock duration in milliseconds. That log is the per-call record the latency figures of Section 8.6 are computed from, and for the determinism probes it is the only surviving per-question artifact.

B.1 `search_entity`

Resolves a surface form to graph entities. This is the only entry point from text to graph, and the only operation that consults an embedding. It delegates to the three-tier resolver of Section 4.6 and returns the tier that produced the hit, so that a downstream failure can be attributed to resolution rather than to navigation.

```
search_entity(surface_form, k=5)
  -> [{id, name, score, tier}, ...]      up to k candidates

Tier 1  exact:      MATCH (e:Entity {name:$m}) RETURN e.id, e.name
Tier 2  lexical:    CALL db.index.fulltext.queryNodes('entity_name', $q)
                        YIELD node, score
                        RETURN node.id, node.name, score LIMIT $k
```

```

Tier 3  vector:      CALL db.index.vector.queryNodes('entity_embedding', $k,
↪ $v)

                YIELD node, score
                RETURN node.id, node.name, score

```

Tier 2 accepts its top hit only when the full-text score exceeds a threshold of 1.0. Without that gate the index returns a ranked list for almost any input, including inputs with no correct answer in the graph, and tier 3 would never be reached.

B.2 *get_relations*

Returns the relation types incident to an entity, in *both* directions, each with its fanout count. Direction is carried explicitly because Freebase edges are asserted in one direction only, and the answer to a question is as often reached by traversing an edge backwards as forwards.

```

get_relations(entity_id)
  -> [{rel, dir, n}, ...]      sorted by n descending, capped at
↪ max_relations

MATCH (e:Entity {id:$id})-[r]->()
RETURN r.fb_name AS rel, 'out' AS dir, count(*) AS n
UNION ALL
MATCH (e:Entity {id:$id})<-[r]-()
RETURN r.fb_name AS rel, 'in' AS dir, count(*) AS n

```

Administrative predicates are filtered before the cap is applied, by prefix: *common.topic.*, *common.*, *freebase.*, *type.*, *kg.*, *user.*, *dataworld.*, *rdf-schema#* and *owl#*. These are schema bookkeeping rather than facts about the world. They also have very high fanout, so filtering before sorting is what keeps them from consuming the whole list.

B.3 *get_neighbors*

Expands one edge from one entity. This is the operation that makes Freebase's mediator nodes invisible to the agent: where a neighbour is a mediator, the tool hops through it in the same call and returns the entity on the far side, carrying the via chain that reached it.

```

get_neighbors(entity_id, relation, direction='out')
  -> {neighbors: [{id, name, via[, cvt]}, ...], truncated: bool}

MATCH (e:Entity {id:$id})
MATCH (e)-[r {fb_name:$rel}]->(n)      -- or <-[r]- when direction='in'
RETURN n.id, n.name, n.is_cvt LIMIT $cap

```

```

-- for each neighbour with is_cvt = true, one further hop:
MATCH (c:Entity {id:$cid})-[r2]-(m:Entity)
WHERE m.id <> $orig AND m.is_cvt = false
RETURN m.id, m.name, r2.fb_name AS rel2 LIMIT $cap

```

The *via* chain is what makes the traversal auditable. A two-element *via* means the result was reached through a mediator, and the *cvt* field names the mediator that was crossed. Section 4.3 explains why a mediator counts as two edges for the reachability gate but as one step for the depth budget: the agent should not be charged twice for an artifact of the encoding.

The truncated flag reports whether the fanout cap was hit. It is recorded rather than silently absorbed, because a truncated expansion is a plausible cause of a later failure, and Chapter 9 needs to be able to distinguish it from a relation that was simply never explored.

B.4 *verify_triple*

Asks whether a specific relation holds between two specific entities. It is the relation-aware structural route of the verification layer (Section 6.3.1).

```

verify_triple(head_id, relation, tail_id)
-> {direct, via_cvt, supported}

MATCH (h:Entity {id:$h}), (t:Entity {id:$t})
RETURN
  EXISTS { MATCH (h)-[fb_name:$rel]-(t) } AS direct,
  EXISTS { MATCH (h)-[fb_name:$rel]-(c:Entity {is_cvt:true})-[]-(t) }
    AS via_cvt

```

Both patterns are undirected. A claim is supported if either holds. The *via_cvt* arm exists for the same reason as the mediator hop in *get_neighbors*: a fact stated through a mediator is still a fact, and a verifier that missed it would reject true claims about exactly the compositional questions the thesis is about.

B.5 *verify_connection*

Asks only whether two entities are adjacent, directly or through one mediator. The relation is not named.

```

verify_connection(a_id, b_id)
-> {direct, via_cvt, supported}

MATCH (a:Entity {id:$a}), (b:Entity {id:$b})
RETURN EXISTS { MATCH (a)-[]-(b) } AS direct,
  EXISTS { MATCH (a)-[]-(c:Entity {is_cvt:true})-[]-(b) } AS via_cvt

```

This is the fifth operation, and the one not in the original design. Section 5.2 records that it was added when the verification layer of Chapter 6 needed a relation-agnostic adjacency test. Its permissiveness is deliberate and is also its weakness: Section 6.8 analyses wrongful acceptance as a direct consequence of asking whether two entities are connected at all rather than whether they are connected in the way the claim asserts.

Appendix C

Sampled Question IDs and Run Manifests

Every number this thesis reports is computed over the question sets listed here. Section 7.2 describes how the two test samples were drawn and certified; this appendix records their contents, so that a reader can reproduce a reported figure over exactly the questions it was computed from. The identifiers are the dataset’s own. A CWQ identifier carries the WebQSP identifier it was derived from as its prefix, which is why some of them are long.

Table C.1: The three question sets. The test samples are stratified by hop count, the development set by question shape.

Set	n	Strata
test_webqsp	400	h1 256, h2 128, h3plus 4, unreachable 12
test_cwq	400	h1 137, h2 211, h3plus 49, unreachable 3
dev80	80	conjunction 13, cvt_heavy 10, one_hop 18, two_hop 32, unanswerable_env 5, unanswerable_fake 2

C.1 WebQSP test sample ($n = 400$)

Source: results/phase4/test_webqsp.json.

```
WebQTest-0 WebQTest-100 WebQTest-1003 WebQTest-1007 WebQTest-1010 WebQTest-1011 WebQTest-1014
WebQTest-1020 WebQTest-1022 WebQTest-103 WebQTest-1035 WebQTest-1047 WebQTest-105
WebQTest-1057 WebQTest-1059 WebQTest-1061 WebQTest-1063 WebQTest-1067 WebQTest-1069
WebQTest-1077 WebQTest-1079 WebQTest-108 WebQTest-1080 WebQTest-1090 WebQTest-1095
WebQTest-1097 WebQTest-1099 WebQTest-1108 WebQTest-1110 WebQTest-1111 WebQTest-1114
WebQTest-1115 WebQTest-1120 WebQTest-1121 WebQTest-1130 WebQTest-1133 WebQTest-114
WebQTest-1147 WebQTest-1151 WebQTest-1169 WebQTest-1170 WebQTest-1177 WebQTest-1178
WebQTest-118 WebQTest-1187 WebQTest-1189 WebQTest-1199 WebQTest-121 WebQTest-1215 WebQTest-122
WebQTest-1226 WebQTest-1233 WebQTest-1239 WebQTest-124 WebQTest-1242 WebQTest-1259
WebQTest-1267 WebQTest-1269 WebQTest-127 WebQTest-1271 WebQTest-128 WebQTest-1281
WebQTest-1289 WebQTest-129 WebQTest-1291 WebQTest-1292 WebQTest-1296 WebQTest-1304
WebQTest-1309 WebQTest-1313 WebQTest-132 WebQTest-1321 WebQTest-1322 WebQTest-1324
WebQTest-1327 WebQTest-133 WebQTest-1330 WebQTest-1334 WebQTest-1339 WebQTest-1350
WebQTest-1353 WebQTest-1354 WebQTest-1362 WebQTest-1367 WebQTest-1369 WebQTest-1370
```

WebQTest-1373 WebQTest-1376 WebQTest-1381 WebQTest-1387 WebQTest-1399 WebQTest-1416
 WebQTest-1422 WebQTest-1426 WebQTest-1445 WebQTest-1452 WebQTest-1456 WebQTest-1457
 WebQTest-1458 WebQTest-1461 WebQTest-1464 WebQTest-1465 WebQTest-147 WebQTest-1470
 WebQTest-1471 WebQTest-1477 WebQTest-1484 WebQTest-1485 WebQTest-1490 WebQTest-1493
 WebQTest-150 WebQTest-1501 WebQTest-1520 WebQTest-1524 WebQTest-153 WebQTest-1534
 WebQTest-1548 WebQTest-1550 WebQTest-1555 WebQTest-1556 WebQTest-1557 WebQTest-1560
 WebQTest-1561 WebQTest-1564 WebQTest-1566 WebQTest-1567 WebQTest-1572 WebQTest-1575
 WebQTest-1584 WebQTest-1585 WebQTest-1595 WebQTest-16 WebQTest-161 WebQTest-1611 WebQTest-1618
 WebQTest-1620 WebQTest-1628 WebQTest-1629 WebQTest-1630 WebQTest-1633 WebQTest-1635
 WebQTest-164 WebQTest-1646 WebQTest-165 WebQTest-1650 WebQTest-1655 WebQTest-1658
 WebQTest-1667 WebQTest-1668 WebQTest-1669 WebQTest-1673 WebQTest-1674 WebQTest-1679
 WebQTest-1680 WebQTest-1682 WebQTest-1687 WebQTest-1688 WebQTest-1698 WebQTest-1707
 WebQTest-1711 WebQTest-172 WebQTest-1723 WebQTest-1724 WebQTest-1725 WebQTest-1730
 WebQTest-1733 WebQTest-1736 WebQTest-1737 WebQTest-1744 WebQTest-1752 WebQTest-1758
 WebQTest-1759 WebQTest-176 WebQTest-1760 WebQTest-1763 WebQTest-1764 WebQTest-1767
 WebQTest-1768 WebQTest-177 WebQTest-1770 WebQTest-1782 WebQTest-1791 WebQTest-1793
 WebQTest-1794 WebQTest-1795 WebQTest-1796 WebQTest-1813 WebQTest-1822 WebQTest-1824
 WebQTest-1829 WebQTest-1835 WebQTest-1836 WebQTest-1840 WebQTest-1847 WebQTest-1857
 WebQTest-186 WebQTest-1864 WebQTest-1868 WebQTest-1876 WebQTest-1877 WebQTest-1897
 WebQTest-1898 WebQTest-1902 WebQTest-1907 WebQTest-1909 WebQTest-191 WebQTest-1914
 WebQTest-1915 WebQTest-1919 WebQTest-1920 WebQTest-1923 WebQTest-1928 WebQTest-1929
 WebQTest-193 WebQTest-1937 WebQTest-194 WebQTest-1940 WebQTest-1942 WebQTest-195 WebQTest-1954
 WebQTest-1962 WebQTest-1967 WebQTest-1968 WebQTest-1969 WebQTest-1971 WebQTest-1972
 WebQTest-1976 WebQTest-1978 WebQTest-1979 WebQTest-1981 WebQTest-1985 WebQTest-1993
 WebQTest-200 WebQTest-2002 WebQTest-2003 WebQTest-2006 WebQTest-2017 WebQTest-2020
 WebQTest-205 WebQTest-206 WebQTest-209 WebQTest-211 WebQTest-215 WebQTest-225 WebQTest-23
 WebQTest-231 WebQTest-233 WebQTest-234 WebQTest-239 WebQTest-245 WebQTest-254 WebQTest-256
 WebQTest-262 WebQTest-264 WebQTest-273 WebQTest-284 WebQTest-286 WebQTest-291 WebQTest-296
 WebQTest-298 WebQTest-3 WebQTest-301 WebQTest-304 WebQTest-308 WebQTest-310 WebQTest-311
 WebQTest-314 WebQTest-316 WebQTest-329 WebQTest-334 WebQTest-336 WebQTest-341 WebQTest-342
 WebQTest-359 WebQTest-362 WebQTest-371 WebQTest-377 WebQTest-379 WebQTest-38 WebQTest-383
 WebQTest-388 WebQTest-397 WebQTest-403 WebQTest-409 WebQTest-410 WebQTest-417 WebQTest-419
 WebQTest-425 WebQTest-43 WebQTest-431 WebQTest-432 WebQTest-433 WebQTest-440 WebQTest-441
 WebQTest-447 WebQTest-462 WebQTest-464 WebQTest-469 WebQTest-477 WebQTest-479 WebQTest-480
 WebQTest-487 WebQTest-490 WebQTest-513 WebQTest-514 WebQTest-516 WebQTest-518 WebQTest-519
 WebQTest-523 WebQTest-537 WebQTest-542 WebQTest-543 WebQTest-545 WebQTest-555 WebQTest-567
 WebQTest-579 WebQTest-585 WebQTest-59 WebQTest-599 WebQTest-605 WebQTest-608 WebQTest-617
 WebQTest-618 WebQTest-624 WebQTest-636 WebQTest-637 WebQTest-642 WebQTest-658 WebQTest-66
 WebQTest-661 WebQTest-663 WebQTest-665 WebQTest-670 WebQTest-689 WebQTest-698 WebQTest-701
 WebQTest-703 WebQTest-704 WebQTest-710 WebQTest-721 WebQTest-725 WebQTest-734 WebQTest-735
 WebQTest-737 WebQTest-738 WebQTest-74 WebQTest-742 WebQTest-748 WebQTest-749 WebQTest-769
 WebQTest-776 WebQTest-783 WebQTest-784 WebQTest-788 WebQTest-8 WebQTest-812 WebQTest-83
 WebQTest-830 WebQTest-832 WebQTest-838 WebQTest-848 WebQTest-86 WebQTest-861 WebQTest-865
 WebQTest-866 WebQTest-867 WebQTest-869 WebQTest-873 WebQTest-875 WebQTest-88 WebQTest-881
 WebQTest-884 WebQTest-893 WebQTest-896 WebQTest-901 WebQTest-910 WebQTest-911 WebQTest-915
 WebQTest-919 WebQTest-923 WebQTest-925 WebQTest-93 WebQTest-933 WebQTest-936 WebQTest-942
 WebQTest-958 WebQTest-959 WebQTest-96 WebQTest-960 WebQTest-966 WebQTest-969 WebQTest-973
 WebQTest-975 WebQTest-978 WebQTest-980 WebQTest-983 WebQTest-985 WebQTest-991 WebQTest-994
 WebQTest-996

C.2 CWQ test sample ($n = 400$)

Source: results/phase4/test_cwq.json.

WebQTest-1000_0adacc5f8d85e317c77fbf2ffa83dae9 WebQTest-1000_1d3a86f839023bb35d439f56a673dbe6
 WebQTest-1000_6099ea03f4fd2476605c4a513318d29c WebQTest-1000_7457bb008a1da743b19ff9ce3a5cac63

WebQTest-1002.806620bbf6ca7cf3ced5d4ff130c714d WebQTest-1002.9a330187030f30935208376dd179f40d
WebQTest-1012.131eaa9f6b360a7166467b5784470d4 WebQTest-1012.73741811f34519b29f7d19ccfd4d9553
WebQTest-1012.872253e47dd6ddaa213ff31eeda8783b WebQTest-106.8a7ae3422867fd4b2d40b06930915468
WebQTest-106.c8290c381954f40b4b920252a7df33da WebQTest-1072.92f27159a96f716c1691371e64e5a604
WebQTest-1091.61efa0b17cc53c2187422d88bf670f22 WebQTest-1120.5fd34c6d0f1cf393b2bd58d6e9690993
WebQTest-1120.c78405f6f2157645f1098f8a871f1bc2 WebQTest-1168.1591d6a0e6cef93c6863edb2eb0e198f
WebQTest-1171.a8ed8aa8e8eaf11e4b00b81b2ac2ac23 WebQTest-1178.09e61020934818b85641a22821e7a455
WebQTest-1251.cbff2f20f6caf754bc49d672ca7b150b7 WebQTest-1251.cf6cc4cc9ed790243a390f155ae72256
WebQTest-1260.018dc0c070e53555fa37f63d4b83ca87 WebQTest-1260.c4e06c3a9e4b4f10bd1ae97f1742c198
WebQTest-12.5373d9d04da65179a41e5d4669db121e WebQTest-12.7b54b31f3e5a6273f4fd2a20e565ec6d
WebQTest-12.974c964a1a2810facfeb4b1a996e878a WebQTest-1301.5714a7a5f1feec1348045800b0c0c28c
WebQTest-1320.2492cfb926882a4bf9c580ba4f7c700a WebQTest-1348.3f44e0c4fd23a8334a972ebb812e0e35
WebQTest-1348.9cd662e0ee5c2ffbbbbb0dc86053b1680 WebQTest-1348.b7598df908bf8cbe941f82e1cefaec28
WebQTest-1379.8b1ff0551fb22f1d4e54d33d3656b8e8 WebQTest-1384.9440ac3bd6d8053ca4d553da408a7cb0
WebQTest-1387.e519712761951b558bb7cbf4ea39198a WebQTest-1470.888604e15ae965565efd2aa1eb615bab
WebQTest-1473.3d4ecc1165ae1c21499afb8f7b9e25a3 WebQTest-1513.5422fa602469c38418171ad342583759
WebQTest-1528.04810196a28df9032811889ca112f28b WebQTest-1528.3c737d4a4dbf3b5acb082a4a1e43d792
WebQTest-1528.505f2eda6be64caf7b48a166de833564 WebQTest-1528.6ecf5c92092ae9080307178a28b8d589
WebQTest-1528.ac9d054b50d6d55401844da4807076b2 WebQTest-1547.9a301711fdf36a2e2d6fdd7b21d65d3c
WebQTest-1560.210d8463ee35ff65448a711c4461f122 WebQTest-1560.9b121a33fc0ee97ee3a3c43ee4e02a78
WebQTest-1569.0b5333d98ef87008aa02d1fbc1554b05 WebQTest-1603.6f7bafc08148c28d022ab04b0e6a4aaf
WebQTest-1603.7ca9a8a41caa3634b454484965413fd3 WebQTest-1686.554811ebe1463287ee640a214683ea57
WebQTest-1686.80cf1e206617287917f0af7538f69ab8 WebQTest-1686.c374fe555611a2e3aba1ba9ffd739d13
WebQTest-1705.35c94dc6944d964963b44949d8ed512c WebQTest-1736.125140bfa1a60527bde6e40dce7fe54a
WebQTest-1785.23fbfbc07989820c1a44f16de0e5291be WebQTest-1785.2fd3a482f02db22067954809fe7b222b
WebQTest-1785.6e1e55b0316051b2aec513f70a70b4e2 WebQTest-1785.759908f37fd43c40a532a73779dd793d
WebQTest-1785.98806527b045c387792db297c407c023 WebQTest-1785.9ecb1e0e419ee56c63da7b3da5e40ac9
WebQTest-1785.bd2684abe96767b0564cea78024983f2 WebQTest-1785.e0f5bd8c1ccde7ddfa3be8bdf3122ebc
WebQTest-1785.ee9df68984cde21c3be056caefd6613f WebQTest-1785.fdb7a181aeebb9a12bea9e01a41dd606
WebQTest-1797.194b220adab0abb76bdf49354cfd3fff WebQTest-1797.202ded8e5df1c80ea9770154e24e5fffb
WebQTest-1797.2fb9e2823ccf35d2103fa8846d6f2ca8 WebQTest-1797.345a0887c7012fd7647ad0c6f9783e67
WebQTest-1797.5a1c66f408118c6342c4a0eb350b58de WebQTest-1797.dece4ddacc57788e31bf70edc99f1f37
WebQTest-1817.1bbd9ffd7b604510a3d6751b9f8ef796 WebQTest-1823.a8cd75f8533d96e3be293401d63fad4b
WebQTest-1840.3d474acada1d3230d94e79aad9206043 WebQTest-1923.037ebc0e903ab6eb5ef8ce44ecd3bd64
WebQTest-1923.29c81279ed9a982e12f82e764083db76 WebQTest-1923.2c5591f1dbe7402405004d7a0836dd3b
WebQTest-1923.3dac7aa955af6ee2ede38f6d42711a00 WebQTest-1923.44adab808bc1863fab0f07e6556afd67
WebQTest-1923.7084416ab9f72f1f1f8fc3ce7871ee4a WebQTest-1923.93e2fc83e55c4e0deaa4671a531ca523
WebQTest-1923.963f10fbb72072fc718bd8cc3e0ec12d WebQTest-1923.a64ef0f5ce397a5e1ef6fcd550ebfcfb
WebQTest-1923.a90c08a839ac4d0ac22ebbf436bb578b WebQTest-1923.b10bde702bb6c68ce3cdced85d5aff02
WebQTest-1923.eaeaff16b51e52669edc90501a373c61 WebQTest-212.7905d5d52ac1a17f68996d4a2245e682
WebQTest-213.024fd6ca0b4cb30927c22e93a552ae6c WebQTest-213.05bdda30187b7cfb104ec6e7d5ecec6a
WebQTest-213.3ec496c94ff4f83167e7d83479fa7082 WebQTest-213.4d64427814a804819d7bd73a4187dbe2
WebQTest-213.71621b15ae9777fca8c1becaa188a3bb WebQTest-213.7debc6a8247700de6db4ac7b0f4cc0e1
WebQTest-213.d9993d3e6f665d0c937cf7c0209ff9f8 WebQTest-302.88f3f649eed28a53b8a5479dc2c98cb1
WebQTest-361.5f8c05737061d5501b6a23a978b5589b WebQTest-389.27f6987805010183edbeba6b91a3cafd
WebQTest-389.544dcbf5d728f23ac97aa05f025a646a WebQTest-43.a29af73bf225f568206dcf810358236f
WebQTest-450.b68e8f1a3f19f80d7dfc10bf3796b71e WebQTest-484.db24dc739b7b00cdab7cf3d825975aed
WebQTest-493.3ce332b703593e71708a5604865c5758 WebQTest-517.e7d8b401e84db19b4965bb23aa43871d
WebQTest-537.4439b7c167b6ccf30c53b610934c4521 WebQTest-537.560ee86f4cb9196c8d757880602e8433
WebQTest-537.b7771109a84a9b56518460dc9e8878ff WebQTest-537.de17dd1afec3743e98d327e6b1912e99
WebQTest-537.e5da8cda32fb1aa37028f9f7f7b1d3a8 WebQTest-538.4767846f70218c1178e6fb0c4937e547
WebQTest-538.485b6c9f1bd7bd7972f3dd4fcc11b1e9 WebQTest-538.5f5da3e1d4ca7df9f19ce3fdcf5790e2
WebQTest-538.65e7de970f6844e8520d7785799ec19e WebQTest-538.d37989c422fef8a98e7289ca0144159c
WebQTest-538.fc5392eefef107aa88084b05ddd2e246 WebQTest-55.54e856d3c44e0ac4a33f0f0ebb7a67d6
WebQTest-576.495085ad9dd6274cce483df63474ca21 WebQTest-576.99c43635b648023af901dc341b3bab6e
WebQTest-61.5b52c9e057d0f0e6704f0b264e864dfc WebQTest-61.7bd6b37d01a372aa5af2a8d5ef53d827
WebQTest-61.9be8c18e016b713cbb7f7feb6badebc7 WebQTest-634.4f2c0d9eb022ed67ec2cc356d4d5113a
WebQTest-654.f4f38507bbec5d30bec54c8c8dd9a72b WebQTest-712.4e9134eaaafee4fd4023d41d4770f5eb

WebQTest-719_2883cc73d6e7cc887da948a32c5fd2bb WebQTest-719_402552b61d68fcf116111585da32583b
 WebQTest-719_f1a83a32bf04983a5a5f9da914f867c8 WebQTest-743_2b39afadc21f0a2a4602121616db1f8c
 WebQTest-743_4d2f11c63fa0611b0e29bc00f41bf7a9 WebQTest-743_e8f21e0f0c4375d30317d26f9b95cd91
 WebQTest-759_33c0a2138ef3f4a4f61dbee8dc9714e5 WebQTest-783_b181354faec01d121c5f050e0d0da2fd
 WebQTest-832_87afb2481df60df8476b920f00c4247 WebQTest-832_c1d5c9c777c291a80c29c70ec43c45ad
 WebQTest-832_c334509bb5e02cacae1ba2e80c176499 WebQTest-837_8c90d5499a627a3478313407b1404ecf
 WebQTest-837_c4e06c3a9e4b4f10bd1ae97f1742c198 WebQTest-918_a6b0e48e01f77546d20952acd67b396
 WebQTest-924_6d72eal1b1b38e146c28c87fee57e7f96 WebQTest-924_ad15b96e5de6c60b06f2295638c2e471
 WebQTest-965_e91a95bd503f0704b9cc40c6f2139524 WebQTest-983_1f8b6ebf20d1119eba090d8d0257bdc5
 WebQTest-983_5117d407df6e21d30cd4893bcab84281 WebQTest-989_4b6636a049fa767111eef6ce82bf7054
 WebQTest-989_6d0a1bf895925a752370575740368440 WebQTest-998_47e95c73ec76579a8d66b6e1607df389
 WebQTrn-1001_33d9f1ae3911d4f69a9993902924107d WebQTrn-1001_6bcfac70e63efec42212159c644a593a
 WebQTrn-1023_2ed287b0ef57067bd3a79f71c2fa849e WebQTrn-1023_6b6f19d9abc98d7eda3c6da9a66f1176
 WebQTrn-1023_efcc9747755b72a5f927c5ccf984d9e4 WebQTrn-1053_4c8649801d197e6eb5377d292c0b6ca8
 WebQTrn-105_8b3815ec57389b13892dd5abb108cc4e WebQTrn-1069_1eb6f5acf20855a1a2ef0d5b38673175
 WebQTrn-1069_3c89af72801c057504e78b3a263a9b77 WebQTrn-1077_7bd6b37d01a372aa5af2a8d5ef53d827
 WebQTrn-1077_d9bc2c5e35761bbc475b7900393fcc97 WebQTrn-1155_cb9805c4c3bedc59995289ea0f7dbf7f
 WebQTrn-1180_67f5f717564163742ca2588b7e186fd8 WebQTrn-1180_d234333cfe0037241bacdcbbc5a74317
 WebQTrn-1232_cb253aaf69993423e0691acalad6cafa WebQTrn-124_0782789f35ce79d56102e26e54e5b700
 WebQTrn-124_609120802c32c3608fc379700186f76c WebQTrn-124_8a391bb9366c22ce0aad00cfecf7e08
 WebQTrn-124_92133c60417a74da674a0807a5a56eff WebQTrn-124_cb9e908c93b1f44b95c2119a3de0ab04
 WebQTrn-125_003c6a046c47526d922683f22cf0e983 WebQTrn-1278_025fdafad914ff922ab8144f527c06ec
 WebQTrn-1278_51fc729996448b82cb36b98e70cf1498 WebQTrn-1278_d8e43a02200cfdff82052f8cc5395b27
 WebQTrn-1283_623b8370871966e2e8aeb301482e1558 WebQTrn-1294_a4b2006ad5ffa1964eb8aa93149cba5a
 WebQTrn-1297_a0da13a7b70a064b8cfff58007f6739d WebQTrn-1392_d372995c4cada937a6f201d316698ad5
 WebQTrn-1394_75a63b2d78bab35a97e07d53149a29e0 WebQTrn-1405_32fd6bcd379a666c1b479eb00ba3dfc6
 WebQTrn-1405_b5513034c35cc49fda740f1a0ae5b7b9 WebQTrn-1405_b98a27e21e904173168eb7517b123e51
 WebQTrn-1405_ced20d844a472e7fafced76fae0a1c7c WebQTrn-1484_1fbd766e1b1bf978da23e8646cadf3a1
 WebQTrn-1484_256d41885c1839681311ca57f4d7b6c4 WebQTrn-14_f56f0b81a5d0fd41e793f70c0b468455
 WebQTrn-1521_8a0b4e1cf3027533ff8a45c91970ad01 WebQTrn-1557_25aa66737549cccec679a88f7ec71c350
 WebQTrn-1557_573d4efe36c8edcac99c7247408e53c2 WebQTrn-1557_a26c076b0c074bdd9c4de45e28dfc752
 WebQTrn-1560_3dba296e57fcc898668201e992da8b6b WebQTrn-1577_1948beb9c3dfa0c0861f67f8d27c503f
 WebQTrn-1577_1fad167a955dd126cc7ca38ed92d28f0 WebQTrn-1577_5d3641e018eabc9011ca0d03911487d3
 WebQTrn-1577_68d3456de4f00782e827bacf13846d5e WebQTrn-1577_ffdf9e623c3c4aa623a95045b61c29e9
 WebQTrn-1597_8adc06ba12a3254370bdb3ff8e8820af WebQTrn-1629_5b23295775632ec566475a1ab8748962
 WebQTrn-1646_5abd17edee676db9f12bbbe84edfb575 WebQTrn-1646_9dd9cd027e4fb356ec7f48576c94b9a8
 WebQTrn-1665_4293d22de1b977d7bacb0d5896623115 WebQTrn-1665_5b2c4b0d5faeffa7e6ed3767a4a57c89
 WebQTrn-1677_15a55e684b4c9a686e6865cfcad59905 WebQTrn-1677_22c4d321fc60b69e1d966c77d4107a74
 WebQTrn-1677_2fe630fbb46b32aa9774a9417e843503 WebQTrn-1677_35504fa84f9be86a31a22c167e7c17ef
 WebQTrn-1677_80066cc2617f4e1301adab9b3e951711 WebQTrn-1677_93664d8718c70b3379fbb31bf4743598
 WebQTrn-1677_a7b0673d818a426f402b6a2c2573d137 WebQTrn-1677_bdfdca39c58846e51247fd78bc7683eb
 WebQTrn-1677_c006ead7cde93a8aff111388eaa455c1 WebQTrn-1677_c606d81a0b9a8fe9f1a8a17239ebe8e2
 WebQTrn-1679_25b39fc2f9ae169e151a869f436f594b WebQTrn-1706_97097ed7f8b2c898c800e8717998cb4c
 WebQTrn-1722_c2ff19afb275737b0309f11e46a2d691 WebQTrn-1758_477d7040a0ed5bfecb155b8b3d3082fb
 WebQTrn-1758_7365436f41a9104b900bd24685d394a3 WebQTrn-1762_ea65a065bc9b8f5eda6ea8c0716c84b8
 WebQTrn-1770_540abec8ff3d2f4e81bfc5be9ea8e816 WebQTrn-1770_6325ee890e333f0d4fe222c15a95906f
 WebQTrn-1770_6a7c160ace84e7908302f805739ad06d WebQTrn-1812_04dbed66ba17d35bf79f72e04bbcc776
 WebQTrn-1812_688d69d484eeaffda36f06fbb9a9fe9fb WebQTrn-1812_82e069064d66663276c549f47e5e179d
 WebQTrn-1812_af6e94960d6ecddfb9e8bee3ca654f5 WebQTrn-1841_5d1cba94c816bec220b9f98e97493b0f
 WebQTrn-1938_7322a2a4d46bf36b95bfab4418c9a32b WebQTrn-2006_4c7647e8d0b7435d37296e8103bcb33f
 WebQTrn-2006_54c677761ff06a9b6e223ccb4c72a2a6 WebQTrn-2006_7993a79baca778d4e8e6cb2b1882bca4
 WebQTrn-2006_e258f4c46fa686cf9c12150935e9211f WebQTrn-2023_139d69ce55d2d9feed41587eb4aa1b54
 WebQTrn-2023_8349bb144acf1f6bb43d68bfa35e97c1 WebQTrn-2026_de4e12cff67645e8580d6a495196c768
 WebQTrn-2026_de5bb2b6c6727d40089ae17f2650f97d WebQTrn-2047_01c02992dccc4a886742ee8f9411d939
 WebQTrn-2047_38077debd523fe1b08b3d6e13538402b WebQTrn-2047_3d5f855275265f74d81b862231578e33
 WebQTrn-2057_989ef8c8e3ea409e00989ea93579d5ff WebQTrn-2069_0b65dea73fbce4f9f616eb5f69e9aa73
 WebQTrn-2069_14d783de97238bd1ef6eb28fe7221306 WebQTrn-2069_c1d589c22e4bcb71dd3832d4a8a1dbe9
 WebQTrn-2069_cf3638075fd9204c82c8adf9ae47925e WebQTrn-2100_faefa67fba5946fa87b438a6df0b8e63

WebQTrn-2152_3cdf60c15a8355981dd92e3c57ac2eed WebQTrn-2152_92fba37c9723caee68665ad9a5e4a468
WebQTrn-2152_c9be001aaf5f69c9067ba4b530ca0a93 WebQTrn-21_beeb032ce88fb3abfb8bf8bd2b657e62
WebQTrn-2209_7bc37f5ea0bc419b7ee5510daff240df WebQTrn-2209_beb5507aba5b006b3b2e0598989448c8
WebQTrn-2292_8c30eccd3cde88b0a0d389f34d086641 WebQTrn-2292_ea0bc3bb340865025c534c91eca19be9
WebQTrn-2311_a5f644b5210b936ed83ddcae101ef3ea WebQTrn-2311_a82b389c06081907ebdd12acd9382ab0
WebQTrn-2319_14ab2260ec77ebd5b3b8f666efc08fc3 WebQTrn-2319_16bcc3dca8a1c20229f4341ad409bb04
WebQTrn-2335_7cd339eefa681cdeb907de4bdf65f8aae WebQTrn-237_f43785599ba51bf943f98f8d9bbf102a
WebQTrn-241_353fe30c00fbc6d3d872a689d17b240d WebQTrn-241_7alf9047f5100e2853d54d69c6793f2d
WebQTrn-241_b96b735d1d754f27bc8986b614669f7b WebQTrn-241_f0ffa14797f7486fb0510a740e1cc8d6
WebQTrn-2429_4449720a6e25f3093a8065ae980f0221 WebQTrn-2478_3e09b108c3448248556d4c34acae3cf7
WebQTrn-2478_a887b1442ee30456f5de89006e1a6c72 WebQTrn-2540_1af583970c492fc3682ae363bbfca3d3
WebQTrn-2540_5e56c7691b80c044d23d2f29efc3d94f WebQTrn-2569_6b6fb0182356d92873272424688be466
WebQTrn-2569_acebb33e831a74414284fba7938e01c8 WebQTrn-2570_6f059734e774504c68f628670516e72c
WebQTrn-2570_99dba9274e8d6c826fdb25827fe613b3 WebQTrn-2570_ac52c51debd358141a23acc7ae7ec938
WebQTrn-2570_c47133a5a4595621bf114ffa8ed4c3e3 WebQTrn-2570_d63877a40200d18c7c4e39215a3c7019
WebQTrn-2570_ff6becc9074132acf2f300dbba1c1e4e WebQTrn-25_23b0132339644b875bf6a966c5b0c131
WebQTrn-25_2a2de50d3b65cc5d2c88f54e283a4b8a WebQTrn-25_3a7068fa96cd9e08dc2b1dabe458c85d
WebQTrn-25_403eec031cd613bfe527504362411ca0 WebQTrn-25_4d68c1f4906c41ef2f0ea4e416e2615b
WebQTrn-25_62939f05e216712e4700e993437a980a WebQTrn-25_d315c4815af961d22882675a61b35b81
WebQTrn-25_db9695c22f0f88f0733e1286c7fffe30 WebQTrn-25_ee5981878443f689128ca9ea8a269f29
WebQTrn-25_f65ceb2f4ef98392b0177705932740a5 WebQTrn-2615_48a6ac5681b2e67e07d84708115db614
WebQTrn-261_30a384ed1f686b2338e963e4a2a963d3 WebQTrn-261_7bc18657d5150879fc643779b6ad5e48
WebQTrn-2653_6c19bc1fe78f2c0015bfd29e5c12767c WebQTrn-2653_ea8442cb2c503dcbd06d9b7a77fed60d
WebQTrn-2664_4001558ccc529debde3a908a6f22b018 WebQTrn-2664_9c30227e437d561a395bf8370530b6e6
WebQTrn-2721_2d207ed91313bd46c4ffd81c9e26a912 WebQTrn-2748_4a22e87b9e3af2f4de74d5017c09de12
WebQTrn-2748_4d6c65004ee28d487bc3921f625106b5 WebQTrn-2748_5394f28651892e7656afeda366a6af04
WebQTrn-2754_7b7ab19b5492a3184356f9305cdd3f9f WebQTrn-2779_14a5adac3764601c1967451c79cbdb79
WebQTrn-2784_025fdfafd914ff922ab8144f527c06ec WebQTrn-2784_1ef455d28d4c19f07ec1bf5ca5bad8c3
WebQTrn-2784_699eef038aebc4af8f113ca8cd4081a0 WebQTrn-2784_abedc8acd0fdcd2e595d7ec6d58d6058
WebQTrn-2784_e35bbe9c7fc2677853b0df65b7b656f6 WebQTrn-2809_8deb69518817ab56417b04d421af85b3
WebQTrn-2815_3a3a861fe35922a25bb5991497496887 WebQTrn-2818_db09fc9949516c3b8e34002f5507514d
WebQTrn-2834_61efa0b17cc53c2187422d88bf670f22 WebQTrn-2834_d3301dedbdcb2d171a75be1631e8cec5
WebQTrn-2859_37bf80e89387cf09fca66eb7915a52b0 WebQTrn-2871_4c5dc1332d9eba8fde58e7b9561d4ae3
WebQTrn-2871_98925752e1e60abb73dc775f15ee38af WebQTrn-2904_0b2295cf692f73863c4d317fe2713132
WebQTrn-2904_13765c6b33acdcb614acbd7e8d2c5ea1 WebQTrn-2904_99f2c6359f65cb116bf380f01af55d6b
WebQTrn-3034_2fd33d22c32307ac6536d5c5644a1bde WebQTrn-303_2ee96aa7464485d80d214b61773f4a5c
WebQTrn-303_770773a5150d1cdd3cfadfc25022720b WebQTrn-3057_5a7cd2677ed97e520a124ba9c2da6ad4
WebQTrn-3084_62ea63f7954eccb68d685d3695c0b84d WebQTrn-3084_ca8d129cc42cf94b681a91fb5ed94522
WebQTrn-3084_d7a8cbddd43733cfb78c3d41af704745 WebQTrn-3084_ec0835b1ad2d181fa3b4195d539d862a
WebQTrn-3100_d059b24adec4064377b957ca598769be WebQTrn-3123_5968b1cb8e15d4f7819fde8b72714100
WebQTrn-3141_9521cdd0a498eb745a7f49c67f6a999f WebQTrn-3170_8c4cd2a8dd5064dcd1e88389796138c7
WebQTrn-3249_0b9c49912f43238136954d463b417194 WebQTrn-3249_4fddfec17159cdf7caa87c1e3e1f34cd
WebQTrn-3249_6dbfa476f0510018f470c29993293e68 WebQTrn-3335_c55ccf5445d560b373944df6540f9b31
WebQTrn-3376_0619d288bbbed0ca782e60c6f841a6051 WebQTrn-3376_3cfac4b90586c13b7348805674c750e1
WebQTrn-3376_ba923c7252439706e2733b02103b7ba5 WebQTrn-3400_675e2a842506fdbbcf3ccdec8f51cee3
WebQTrn-3515_76ac752ffcee1b2759600cbec55107c7 WebQTrn-3515_d1a0c35fff4a3a4dec452ce9b7793244
WebQTrn-3527_1661062ba428b2ec2a00e2f84257682f WebQTrn-3543_3f03848605c6758ff2230a955cd92d65
WebQTrn-3543_b5eabb006685da5755fdf968500b7cd4 WebQTrn-3543_ca8d129cc42cf94b681a91fb5ed94522
WebQTrn-3543_d7db4cdab1589b7c97a81236310a67f7 WebQTrn-3673_b0f2d7005d48b548c7022ad46eccfd31
WebQTrn-3744_6dcb413fa7f7fe52b9345f95bda76752 WebQTrn-3766_2458404b2726ce40f3d525263dc9f74c
WebQTrn-3769_6e0a31334a35147f868ae514f9eb9ab8 WebQTrn-412_47b2be470a658a605c3d174a78135849
WebQTrn-423_135747a75bf43cdc1e0a1cb41f49965e WebQTrn-42_021690b43db9ed877de0fa9d66751617
WebQTrn-436_f7cfe105d2830ecc77ff720612084de8 WebQTrn-452_006b1f1ed800c636a01f3888644cc1eb
WebQTrn-452_e1d6f53f49be4abf9b71b6f18c67ac13 WebQTrn-452_f79ffe93341047ed2d8a39e5bb7d9156
WebQTrn-452_f8c717f06ff656fba2fabfb354030958 WebQTrn-464_cab1228be4e8c16ee35a67b7ac63b264
WebQTrn-484_0bd50f38d8c9b7904b5a1187a904cd73 WebQTrn-484_d35194e42e8dfb2250164a3a10b93131
WebQTrn-484_dd6bb253556f96cc801f1b16e60a1cf WebQTrn-493_590fff78c30981ef9566ad54a453d002
WebQTrn-493_cf7833e237b8cda1de396587129515c1 WebQTrn-497_3b4a316ec5790c8b3b2ec3c6aa23b29d

WebQTrn-513.5a15eae658cc54d81698d2e5e68e82c0 WebQTrn-557.12cad9d9eb020bdd9bfd11099ac0a07c
 WebQTrn-567.1e6dd9ec51a40b6c0ec2105ed84574c5 WebQTrn-567.2c5591fldbe7402405004d7a0836dd3b
 WebQTrn-567.5b0f028745dcda2aac706f1a6b8a965c WebQTrn-567.693feb48c0515cd069014e7ca2846b37
 WebQTrn-567.724a3b769671b1ea52a76af3a90687f4 WebQTrn-567.8e46718c3fc1361ff1c02b62a853b402
 WebQTrn-567.94ad90ab85ded4cd5d0680cdd5c121aa WebQTrn-567.b63fe0fde281c871990cc523c2df1e9b
 WebQTrn-567.bfcde480dde9e03c320b2d2cc1d67743 WebQTrn-568.9fa2b7684e5de3793c6eb46ac7025b64
 WebQTrn-568.f4dd5e4885d4e73788e2f0e9a63d6035 WebQTrn-60.085ced56c2b8a859253d1cd5749c9a28
 WebQTrn-60.1649fb7df8600ed128b0db6c53a483a2 WebQTrn-60.47c68c12c2afd9ae65f31a10982c43db
 WebQTrn-60.4faaf65aellcc1b138d41253c3fc0e493 WebQTrn-60.6b8ef173b0fe9eb52a18df50991df028
 WebQTrn-60.7abf3620549d6d7aaf194ce37a34bbd8 WebQTrn-60.eae3033643523fd3a54b6c147fdfa728
 WebQTrn-62.b2f9ab8521bc316c4c97cfbba4229932 WebQTrn-64.017078a590fa5382bab9986c451d8d08
 WebQTrn-64.025fdfafd914ff922ab8144f527c06ec WebQTrn-64.08a3071aec88af141fc20ed22cfff0e2
 WebQTrn-64.d8e43a02200cfdff82052f8cc5395b27 WebQTrn-683.7584ba13fff133af09542211fd90a33d
 WebQTrn-710.c1d5c9c777c291a80c29c70ec43c45ad WebQTrn-710.e3d40457273785e46c5b71732713a5f4
 WebQTrn-750.aa7f855c24369c18d0bfdb21bbf9d6d1 WebQTrn-750.bb35e9b8023fbf9c05df55b4245a4775
 WebQTrn-831.c191674d835a2284701f16d70a146360 WebQTrn-831.f2d789d28c11bc5b682263b07195d208
 WebQTrn-837.690725e14115ca059d1bcbfb022bbf1f WebQTrn-846.50383833393ddda52ad949cebc1877f
 WebQTrn-846.e852443b88aebb56535c3fb8390109ca WebQTrn-849.29d62c66b3b2bc4938023788455b5bc7
 WebQTrn-849.82e069064d66663276c549f47e5e179d WebQTrn-849.8bb131f101e71d906d4a6dd2dbed06f7
 WebQTrn-849.eba7931ddf28af4dfa20fb099196c91f WebQTrn-849.f0ffa14797f7486fb0510a740e1cc8d6
 WebQTrn-849.falffffe7995213b6528da57ea4c8d226 WebQTrn-857.d585a4c2e04a9cd40dd5ec98e2456515
 WebQTrn-886.36e0f3794f873b19127b97c24fcb3743 WebQTrn-894.022431f6dfa30cf2e715eb171e0437ec
 WebQTrn-894.dc126021d39290ceecabd0361b65c4b WebQTrn-945.121437233d5ec5331edb2a957e079759
 WebQTrn-95.9cefb57084da782f843b2c57e373f0a7 WebQTrn-962.f69b9d7b30f2ef81193ae7b0a39c78c5

C.3 Development set ($n = 80$)

Source: results/phase3/dev80.json.

WebQTest-1267.9eeb2f38ee223b499967ac117f641e3b WebQTest-1356.903876a283f3ce8764412a6cfd6a2c44
 WebQTest-1520.5f93548592ef4987720d1e3f3fe6b3d7 WebQTest-1554.b0412b83228508271932b11e338e48c6
 WebQTest-1673.c10cd072338ca267c3ec5cd1333030f4 WebQTest-1781.455af05d577248799d41f84b34d67d8e
 WebQTest-181.9dabd74992f88c62c2152b48ab1ee8cd WebQTest-1907.5abd17edee676db9f12bbbe84edfb575
 WebQTest-1910.1e6d4aefe769586d47767d86ffdd20b5 WebQTest-2025.c99c8cba93f683f3e37f2e175659c925
 WebQTest-2029.761df3934c8a087ee86d3ab6317d2d9d WebQTest-207.7bf178e825bb55452806d2e708f9ed50
 WebQTest-410.67d6828a804cdeef38928c85cb93bb03 WebQTest-657.0818b36d353fffb39f85b4606b1230866
 WebQTest-673.44a4fd4e33816608544b1091f2bd81ea WebQTest-673.5ecefd82a413a79be541a1bd16967800
 WebQTest-693.dd11343b9c28a21731d2ce4bbdc2630d WebQTest-77.8176959b7600428c7cdb7aacd77fb5ea
 WebQTest-851.9f924908bb36bf7d331fa1b933af9f58 WebQTrn-1
 WebQTrn-1002.30f1a4f034fe9f3f6df3327d69fb669b WebQTrn-1052.b9b4dbcf721e6ef7838dcd3082761330
 WebQTrn-1066 WebQTrn-1067.ed395c43c14a93f3e41c1758ec7e1b8a
 WebQTrn-1214.ebb01712f3eb1443b2895961ec14272 WebQTrn-1298 WebQTrn-144 WebQTrn-153 WebQTrn-1548
 WebQTrn-1549 WebQTrn-1566.63779e80b19cde0010c566ec0b4a0ed6 WebQTrn-1589
 WebQTrn-1603.169d8c870fa88566celf62da123323d6 WebQTrn-1719.124a834e764c099d727ab87c1f80ccee
 WebQTrn-1719.96914535d2c1ae6120a090361e60c965 WebQTrn-1751 WebQTrn-2027 WebQTrn-2028
 WebQTrn-2134 WebQTrn-2400 WebQTrn-2436 WebQTrn-254.01e2da60a2779c4ae4b5d1547499a4f8
 WebQTrn-2584.0c8bcc717d50c92c31ff802641504b43 WebQTrn-2617
 WebQTrn-262.c1b166cd48ca0675b0bb9815289851f3 WebQTrn-2721
 WebQTrn-274.5b8fa743161c465927586720934bef48 WebQTrn-279.fd6e419f422eb640f1195f506522023e
 WebQTrn-2825.5bf6906c26a34660ad81a9e0506d36fc WebQTrn-2840.abf44b12ba7b11ea7e71a11cb540d258
 WebQTrn-3009.a453d9b2a3a3816e8ba70b1a240a4bdd WebQTrn-304.4ec9be4c828025d788bc5fb5ebca9e6e
 WebQTrn-3083 WebQTrn-3085 WebQTrn-3109 WebQTrn-334 WebQTrn-3375
 WebQTrn-3381.44d67b357829d662abelc6dc637e219d WebQTrn-3457.b08f65da81574b8397d5f76ea4138c3d
 WebQTrn-3460.f6e8b7149f2e422640a49beb753be4ef WebQTrn-3468 WebQTrn-3502 WebQTrn-3525
 WebQTrn-3604.fa4b1fbc5cc9db1e3164c9b382fcdc4e7 WebQTrn-3617.258527684b01adblfef0f02a9081ebe7
 WebQTrn-3631 WebQTrn-3710 WebQTrn-430.aa25cef57eebcc069b1783e80e760069 WebQTrn-449

WebQTrn-528_9591187bdfe1d3adb757d74382745ce WebQTrn-546
WebQTrn-547_1f0f082cf9a15e8c58c9655386e1363f WebQTrn-559_9e3fc7126b544740b568757fd7ae1e88
WebQTrn-568 WebQTrn-569 WebQTrn-770 WebQTrn-90 dev-fake-01 dev-fake-02 smoke-20

Appendix D

Annotation Instructions and Label Schema

Two annotation tasks in this thesis produced hand labels: the gold-quality adjudication of Section 7.7, and the failure census of Chapter 9. This appendix gives the instructions and the label schema both were carried out against, so that the categories the census reports can be read as definitions rather than as summary words.

D.1 Judge Instructions

The semantic-support judge of Section 7.6 decides one question per claim: is this claim supported by these triples? Eight rules constrain that judgement. Each was forced by a disagreement encountered while validating the judge against a human annotator, and the validation outcome — $\kappa = 0.6995$ against a pre-registered bar of 0.70 — is reported in Section 7.6 rather than here.

1. Judge the question that was asked, not the question the claim would have answered well. A true statement about the topic entity is unsupported if the question asked about something else.
2. Support must come from the triples shown. A claim that is true in the world but absent from the triples is unsupported.
3. A claim inferable from the triples only by adding outside knowledge is unsupported. This is the rule that separates the judge from the model’s own parametric memory, and it is the one that most often decided a disagreement.
4. Surface form is not the test. Where the graph names an entity University Yale and the claim says Yale University, the claim is not unsupported for that reason alone.
5. Direction is not the test either. A triple asserted in one direction supports a claim stated in the other, unless the relation’s meaning is genuinely asymmetric.

6. A composite claim is supported only if every component is supported. Partial support is unsupported.
7. A hedge asserts nothing and therefore has no claims to judge. It is neither supported nor unsupported.
8. Where the triples are ambiguous between two readings and one supports the claim, judge it supported. The metric is intended to detect assertion without grounding, not to penalise underspecified evidence.

D.2 Failure Census Label Schema

Every packet in the census receives one category, an optional subtype, and a one-sentence note. The note pinpoints where the trajectory *first* left the correct path — plan, then explored relations, then backtracks, then verifier — rather than the last symptom. That ordering is the whole discipline of the exercise: a wrong answer usually has several things wrong with it by the end, and only the first one is a cause.

Convention for the verifier categories

The verifier's positive class is *claim is supported*. A false negative is therefore a claim wrongly *rejected*, and a false positive is a claim wrongly *accepted*. This is the standard convention and the one the census labels use throughout. It is stated explicitly because the gloss in the working reference originally paired the two names in reverse, and two Stage A labels were filed against the reversed reading before the error was found. Both were corrected; Section D.3 records them individually.

Gold-defect subtypes

The census itself met only one or two gold defects, too few to support a subtype vocabulary. The adjudication pass of Section 7.7 met roughly 148 flagged rows and made exactly this judgement at volume, so its subtypes are the reference list. Table D.2 gives them with the counts they were derived from.

D.3 Label Provenance and Regeneration

The census is hand labour recorded in files that scripts also write, so which file is authoritative for which field is not obvious from inspection. Table D.3 settles it.

Table D.1: Failure categories and their subtypes. One category per packet.

Category	Subtypes	Meaning
decomposition_error	over_decomposition, paraphrase_drift, context_stripping, extraction_bug	the planner or drafting pipeline caused it
relation_selection	—	the scorer picked the wrong edge
composite_claim	conjunction_uncovered, no_set_intersection	a multi-constraint question handled only partially
premature_termination	budget, evaluator	stopped before reaching the answer
verifier_fn / verifier_fp	structural_not_semantic	hedged a right claim / passed a wrong one
kg_gap	date_literal, numeric_literal, temporal_qualifier, ordinal, data_error	the environment cannot express what the question needs
answer_selection	—	the correct candidate was retrieved and the drafter chose another
echo	topic, intermediate, granularity	the shared-attractor pattern of Section 9.7
gold_noise / ambiguous_question	see Table D.2	a defect in the gold or the question that survived Section 7.7
other	—	fits none of the above

After any relabelling, three scripts must run in order: `pull_noplanner_categories.py`, then `synthesize_census.py`, then `build_thesis_numbers.py`. Skipping the last leaves the thesis quoting the previous histogram.

Corrections made after the first labelling pass

Three changes were made to labels after the initial pass, and are recorded here rather than silently absorbed.

- The false-positive and false-negative gloss in the working reference paired the two names in reverse of the standard convention. It was corrected, and the convention is now stated explicitly above. The census labels themselves had followed the standard convention throughout and were not changed.
- WebQTrn-1294 moved from `verifier_fn` to `answer_selection`. The verifier had been correct; the error was upstream, in the answerer.
- WebQTrn-1597 moved from `verifier_fn` to `verifier_fp`, the original label having used the reversed convention.

Table D.2: Gold-defect and question-defect subtypes, with the counts observed during adjudication across both datasets.

Subtype	Meaning	n
gold_noise		
wrong_gold	the gold answer is factually incorrect	16
incomplete_gold	gold omits valid answers that systems correctly found	16
type_mismatch	gold answers at the wrong entity type or granularity, though the right type is in the graph	6
malformed_gold	gold is structurally corrupted, such as absurd entity counts or machine identifiers leaked into answer strings	3
ambiguous_question		
temporal	the question’s time reference does not uniquely pick out gold’s answer	9
container_vs_member	readable as asking about a container or about its members	3
place_vs_lineage	“where does X come from” answerable as a place or as an ancestry chain	3
malformed_question	two or more sub-questions conflated into one	2
birthplace_vs_upbringing	“where is X from” genuinely ambiguous between the two	1
variety	gold picks one defensible variety or register of an entity over another	1
place_name	a named place has competing referents and gold picks one	1
underdetermined	the question does not determine a single correct answer	1

Table D.3: Where each label lives, and what regenerating will do to it.

File	Status
labels_{ds}.csv	The hand-labelled census. <code>dump_failure_packets.py</code> rewrites the file but carries existing labels forward by question ID, so hand edits survive a re-run.
noplanner_categories_{ds}.csv	Generated, with no carry-forward. Labels belong in <code>noplanner_discordant_{ds}.md</code> , which <code>pull_noplanner_categories.py</code> transcribes into this CSV. An edit made directly to the CSV is overwritten on the next run.
labels_{ds}_dropped.csv	An input to the histogram rather than an archive: <code>synthesize_census.py</code> merges its rows into the totals. Nothing regenerates it, because it is written only at the moment a row leaves the population. If it is lost, recover it from version control.
sampling_manifest.json	Written but never read back. Its carried, to-read and dropped fields are progress counters. The thesis cites only the population and skip counts, which are stable across re-runs.

The histogram was regenerated afterwards. The totals are unchanged; three CWQ hedge rows moved between categories.

Appendix E

Implementation Notes

Section [7.10](#) names three practices that carry more reproducibility weight than the dependency list, and promises they are recorded here as techniques rather than as incidents. Those are the first three sections below. Three further techniques follow, which govern how the agent itself is built and which the earlier three depend on. In each case the defect that forced the practice is given as evidence rather than as narrative.

E.1 Records Self-Describe

Every record written by any run carries, alongside its result, the backbone description, a hash of the budget configuration, the full run configuration including the system name, and the source commit.

The cost is a few hundred bytes per question. The benefit is that provenance never depends on where a file sits or what it is called. A record found in the wrong directory, or copied out of one, still states what produced it. This is what made the cache-replay anomaly of Section [7.3](#) diagnosable: the question was which configuration had produced a set of records, and the records answered it themselves.

The generalisation is that a file path is not metadata. Directory layout is edited, archived and reorganised over a project's life, and every reorganisation is an opportunity to detach a result from the conditions that produced it. Anything a reader must know in order to interpret a record belongs inside the record.

E.2 Frozen Artifacts Are Committed; Regenerable Ones Are Not

The distinction is not about file size. It is about whether the artifact can be recreated.

The certified question sets, their half-splits, and every result file embody spent money and runs that cannot be repeated identically. They are committed. The graph import, the embedding matrices and the triple index are deterministic products of committed scripts run over public data, and are not committed even though rebuilding them takes hours.

Applying the rule requires honesty about the second category. An artifact is only regenerable if the script that generates it is committed *and* its inputs are still obtainable. Where either fails, the artifact is frozen regardless of how it was made, and Section 9.9.3 records the one case in this project where that judgement had to be made after the fact.

E.3 Cache-Identity Verification

This is the technique of most methodological interest, and the one most readily reusable outside this thesis.

The problem it solves is stated easily and answered badly by ordinary testing. A refactor is made to code on the frozen experimental path. Tests pass. Have the reported numbers changed? Re-running the experiment answers the question, but a re-run that produces the same numbers is weak evidence, because it is exactly what a re-run would produce if the change had also altered which questions happened to succeed.

The stronger check exploits the response cache. Every model call is keyed by a hash over the model identifier, the sampling parameters, and the full prompt text. A code change that does not perturb the frozen path therefore issues *byte-identical prompts*, which means every call must be a cache hit. So:

Re-run the affected questions and assert that the number of cache hits equals the number of model calls, and that both are non-zero.

A single cache miss proves that some prompt differed, and therefore that the change was not behaviour-preserving — before any output is compared. The assertion is stronger than output comparison because it tests the input to the non-deterministic component rather than the output of it, and it is cheap because a fully cached re-run costs nothing.

For this to work the cache hit must be indistinguishable from a live call inside the agent. The budget check still runs, the call counter still increments, and the original token counts are replayed into the meter from the stored record. A cached re-run therefore reproduces the original budget snapshots exactly rather than reporting zeros, which is what makes the identity assertion meaningful. A separate counter records hits, so live and replayed runs remain distinguishable in analysis.

Section 5.9 reports the one occasion this certificate was demanded and returned a clean result, for the ablation flag that must be a no-op when disabled.

E.4 Config Injection and the Partial-Edit Hazard

The agent is a graph of nodes whose signatures the framework, not the author, controls. Passing a run configuration into a node therefore means either closing over it at construction time or threading it through the framework’s own injection mechanism. The first is easier and is a trap: a node that closes over a configuration value captures whatever was in scope when the graph was built, which for a sweep means the first condition’s value silently serving every condition.

The practice that follows is narrow and worth stating plainly. *A configuration value must reach the code that uses it by exactly one route.* Where a value can arrive by two routes, an edit that updates one of them leaves the other in place, and the resulting run is neither the old condition nor the new one. This is the partial-edit hazard, and it is not caught by tests, because each route works in isolation.

The defence used here is an assertion at the entry of every node that receives a configuration, checking the type of what actually arrived. It converts a silent wrong-condition run into a crash on the first question. A run that dies immediately costs minutes; a run that completes under the wrong condition contaminates a table.

E.5 Routers Read, Nodes Write

The state object is mutated by nodes and inspected by routers. Allowing routers to write as well produces control flow that cannot be reconstructed from the trace, because the state a node observes was modified by the routing decision that reached it.

The discipline is one sentence: *routers may read state and must not write it; nodes may write state and must not route.* It costs a small amount of duplication — a value a router needs must be written by the node before it, under a name the router agrees on — and buys the property that the trace is a complete account of the run. Every claim in Chapter 9 about where a trajectory went wrong depends on that being true, because the census reads traces, not code.

Two categories of scratch key exist in the state for this reason. Node outputs are the record. Router inputs, prefixed with an underscore, exist so that a router can decide without recomputing what a node already knew.

E.6 Instrumentation Decides What Can Be Concluded

The verifier persists the claims it rejects. It does not persist the claims it accepts.

That decision was made early, on the reasonable ground that rejections are the interesting events: they are what the layer does, and they are few. Its consequence surfaced only at analysis time. Wrongful rejection — a true claim hedged — is recoverable from the logs at census scale,

because every rejection is on record. Wrongful acceptance — a false claim passed — is not, because an accepted claim leaves no trace distinguishable from a claim that was never tested.

The thesis therefore reports one polarity of verifier error as a rate and the other from a single specimen recovered by manual trace reconstruction. Since wrongful acceptance is the polarity that bears directly on a hallucination claim, this is the limitation named first in Section 10.2.

The technique to extract from this is not a fix; the fix is obvious and is the first item of Section 10.3. It is the prior question:

Before freezing an instrument, ask which claim it will be asked to support, and whether the records it keeps can support the negation of that claim as well as its assertion.

Logging the rejections was sufficient to describe what the layer does. It was not sufficient to bound what the layer misses, and only the second supports a claim about hallucination. The cost of logging both was negligible; the cost of not doing so was a limitation that no amount of later analysis could remove.

Generated using Postgraduate Thesis L^AT_EX Template, Version 1.03. Department of Computer
Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka,
Bangladesh.

This thesis was generated on Friday 7th August, 2026 at 6:29am.